

C语言与程序设计

The C Programming Language

第9章 结构与联合

王多强

1097412466

dqwang@hust.edu.cn

华中科技大学计算机学院

9.1 结构概述

简单变量只能描述单一类型的单个数据，如 `int x`。**数组**虽然可以描述一组数据，但这些数据必须属于同一类型，如 `char s[5]`。

现实中，**有些对象需要用多种不同类型的属性数据来描述**：

如**学生对象**，有学号、班级、姓名、性别，以及英语、数学、C语言成绩等属性。

各种属性数据的数据类型不一致：如学号、班级、姓名可以用字符串（字符数组）描述，性别用一个字符型数据描述，而各科成绩则要用浮点型数据描述。

同时，尽管类型不同，但**它们关系紧密**：是学生的不同属性，“组合”在一起才能描述学生对象的“全貌”。

那么如何将这些类型不同但关系又紧密的数据组织在一起呢？

方法：通过“**封装**”，将不同类型的数据“**组合**”起来形成一种“**合成数据**”来描述具有**复杂属性**的对象。

(1) 对“**合成数据**”，首先定义它的**数据类型**，然后再声明这种数据类型的**变量**来存储对象数据。

C语言提供了**结构类型**来声明这种**复杂的合成数据类型**。

(2) 结构类型是一种**构造数据类型**，属于**用户自定义数据类型**。

内部由称为**数据成员**的属性域组成，数据成员也称为**数据项**或**字段**。

9.2 结构类型和结构变量的声明

9.2.1 结构类型的声明

声明结构类型的一般形式：

```
struct 结构类型名  
{  
    成员声明列表  
};
```

最后以分号结束

其中，

- ◆ **struct** 是关键字，用以说明结构类型；
- ◆ **结构类型名**是合法标识符，不同类型的结构通过名字相互区分；
- ◆ **花括号**界定结构的成员列表；
- ◆ **成员声明列表**是数据成员的声明序列。

例：行星信息包括名称、直径和卫星数三个属性。描述行星的结构类型的声明如下：

```
struct plane {           /* 声明结构类型 */  
  
    char name[16];        /* 行星名称数据成员*/  
  
    double diameter;      /* 行星直径数据成员 (km) */  
  
    int moons;            /* 行星卫星数数据成员*/  
  
};
```

说明：这里声明了一个名为 **struct plane** 的结构类型，该结构类型包含三个数据成员：**name**、**diameter**、**moons**，类型分别是包含 16 个字符的字符数组、double、int。

□ 成员声明列表的一般形式为：

数据类型1 成员名₁₁, 成员名₁₂, ..., 成员名_{1k} ;

...

数据类型 n 成员名 _{$n1$} , 成员名 _{$n2$} , ..., 成员名 _{nm} ;

- ◆ 成员的数据类型可以是除**本结构类型**以外的其他任何类型；
- ◆ 数据类型1 ~ 数据类型 n 可以相同，也可以不同；
- ◆ 一条声明可以包含同类型的多个成员，之间用**逗号分隔**。

```
struct plane {  
    char name[16];  
    int name;   
};
```

注1：同一结构里的成员不能重名

```
struct plane {  
    char name[16];  
    struct plane pn;   
};
```

注2：成员类型不能是本结构类型，即**结构不允许递归定义**

如同**枚举类型**定义，**结构类型定义**也是创建一种**用户自定义的数据类型**。

- ◆ 结构类型在语法上仅是“**类型**”的概念；
- ◆ 定义结构类型后，要进一步**声明属于该类型的变量**，称为**结构变量**，结构变量是程序操作的具体对象。
- ◆ 程序运行时，**为结构变量分配存储空间**，**用结构变量保存结构数据**，**用结构变量参与相关运算**。

9.2.2 结构变量的定义

结构类型的变量称为**结构变量**，简称**结构体**。

1、结构变量的声明 有两种方式：

方式1：先声明结构类型，再声明结构变量

一般形式：

存储类型 **struct** **结构类型名** **结构变量列表**;

其中，

- ◆ **存储类型**：extern、static、auto、register 等；
- ◆ **struct 结构类型名**：已声明过的结构类型，**struct 不能少**；
- ◆ **结构变量列表**：合法标识符，有多个变量时用逗号分隔。
- ◆ **以分号结束**。

如，在前面定义的 **struct plane** 结构类型的基础上声明该结构类型的两个结构变量 **inner** 和 **outer**：

```
struct plane inner, outer;
```

◆ 可用**存储类型**修饰结构变量的声明

(1) **auto struct plane a, b;**

声明两个struct plane类型的**自动（局部）结构变量a和b**。

(2) **register struct plane c;**

声明一个struct plane类型的**寄存器结构变量c**。

(3) **static struct plane x;**

声明一个struct plane类型的**静态结构变量x，其成员都具有静态变量的性质**。

(4) **extern struct plane w;**

对一个struct plane类型的外部结构变量w做**外部声明**。

方式2：在定义结构类型的同时定义结构变量

如： **struct plane {** /* 声明结构类型 */

char name[16]; /* 行星名称数据成员*/

double diameter; /* 行星直径数据成员 (km) */

int moons; /* 行星卫星数数据成员*/

}inner, outer; /* 同时定义两个结构变量 inner 和 outer */



在右花括号和分号之间写
结构变量，有多个时用逗号
号隔开

- **无名结构类型**：声明时只有 **struct** 关键字，**没有给出结构类型名称的结构类型称为无名结构类型**。

例如：

```
struct {  
    int x;  
    float y;  
} u,v;
```

在声明该无名结构类型的同时
声明了两个结构变量u和v。

- ◆ 由于省略了结构类型名，所以在后续程序中不能再声明无名结构类型的结构变量。 如 **struct w;** ❌
- ◆ 如果一个结构类型只使用1次，以后不再用其声明其它结构变量，则可以定义为无名结构类型，否则就应该给出结构类型名（另外，可以用 **typedef** 另为无名结构类型命名，见后）。

□ 用 **typedef** 为结构类型命名

例如: **typedef** struct plane {
 char name[16];
 double diameter;
 int moons;
 } **Plane**;

- ◆ 用 **typedef** 为 **struct plane** 结构类型重命名为 **Plane**。

注：重命名不是定义新类型，而是给原类型起一个**别名**，**目的仅是方便使用**。如上，以后就可以用类型名 **Plane** 来代替 **struct plane** 声明结构变量，二者等价。例如：

Plane star; 或 **struct plane** star;

- 可以用 **typedef** 为一个无名结构类型命名，从而可使无名结构类型也有了名字，然后就可以用这个类型名来声明相应的结构变量。

例如: **typedef struct {**

int x;

float y;

} Location; 为无名结构类型命名

之后用 **Location** 单独声明结构变量: **Location p,q;**✓

2、结构变量的值

结构变量代表一个“**整体**”，是一组数据的集合，是一种抽象的存在。它的具体数据是各个数据成员所具有的相应数据。

为结构变量赋值本质上是给结构变量的各个数据成员赋值，而每个数据成员的类型可能不一样，所以要依据各数据成员的具体情况来完成赋值操作。

9.2.3 结构变量的初始化

结构变量初始化是在声明结构变量的时候为结构变量赋初值。

由**初值表**给出初值。

- **初值表**：由一对**花括号**界定，其中列出**各个数据项的初值**，
这些数据项初值的类型和顺序要与结构类型定义中
各数据成员的类型和顺序一致。

如：

```
struct plane {  
    char name[16];  
    double diameter;  
    int moons;  
} x={"Jupiter", 142987.0, 79};
```

或：

```
Plane y={"Jupiter", 142987.0, 79};
```

赋初值的方式：

- ◆ 按照定义的顺序——对应地将
各初值赋给相应的数据成员。

初值表中的初值可以缺省，但只能省略后面数据成员的初值，不能省略前面或中间的。

如：

```
struct plane {  
    char name[16];  
    double diameter;  
    int moons;  
} x={"Jupiter", 142987.0 }; ✓
```

```
Plane y={"Jupiter", 142987.0, }; ✓
```

但：

```
struct plane {  
    char name[16];  
    double diameter;  
    int moons;  
} x={ , 142987.0, 79 }; ✗
```

```
Plane y={"Jupiter", , 79 }; ✗
```


9.2.4 结构类型的大小和结构体的存储

1、结构类型的大小

结构中包含一系列成员，存储结构变量时，其数据成员依据各自的类型和大小**在内存中顺序、连续存放**。

结构体所占空间的总量 = 所有数据成员所占空间的和

—— 结构类型的大小等于其数据成员的大小之和。

如：

```
struct plane {  
    char name[16];  
    double diameter;  
    int moons;  
};
```



$$16 + 8 + 4 = 28$$

理论上

sizeof(struct plane) == 28

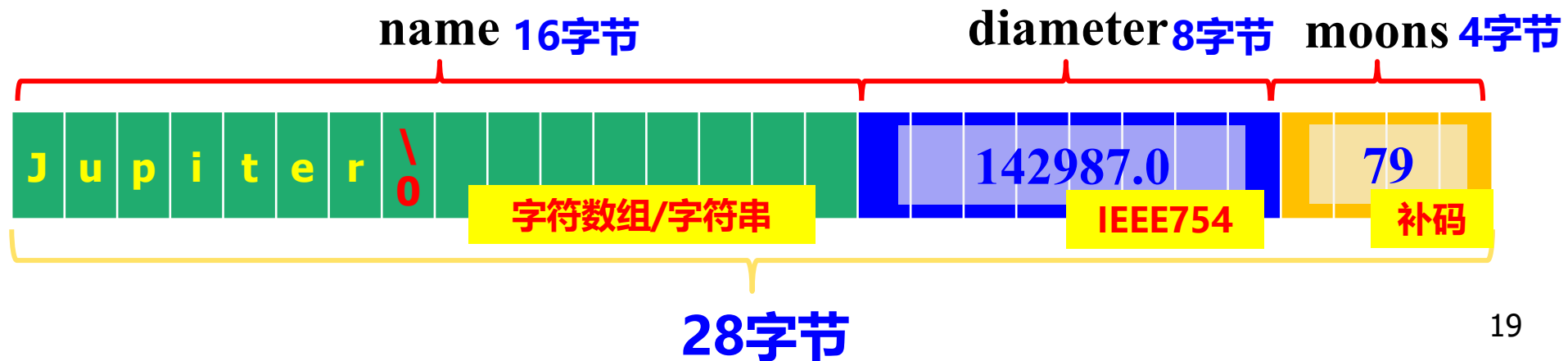
2、结构体（结构变量）的存储

结构体存储时，首先分配**总量大小的连续字节空间**，然后按照类型定义中数据成员的顺序和大小分成若干段，各段再**按照对应成员的类型、以相应的格式存放成员数据**。

如：

```
struct plane {  
    char name[16];  
    double diameter;  
    int moons;  
} x={ "Jupiter" , 142987.0, 79};
```

每个数据成员在其自己的空间里按其类型规定的格式存储。



3、内存对齐

考虑以下定义的两个结构类型，它们的成员除了**顺序不同**以外，其他都一样，但如果求其**大小却不同**，为什么？

```
struct st1 {  
    char a;  
    int b;  
    short c;  
    char d;  
};  
sizeof(struct st1)==12
```

```
struct st2 {  
    char a;  
    char d;  
    short c;  
    int b;  
};  
sizeof(struct st2)==8
```

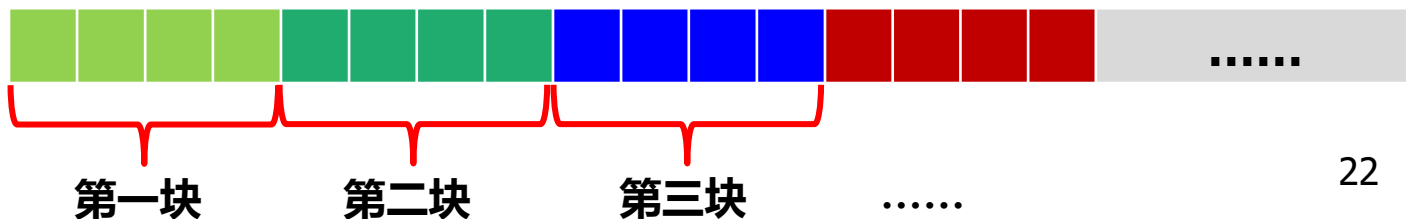
与物理内存的访问规则有关 —— 内存对齐的问题

内存对齐的问题

(1) **CPU的访存规则**：CPU在访问内存数据时，不是直接定位到一个数据的起始地址（即数据第一个字节所在的位置）、然后把该数据的几个字节读出来。而是以一种**以“块”为单位**的方式读取包含该数据的**字节块**，然后从中提取出该数据的字节单元。

对系统而言，**块的大小是固定的**，一般**等于机器字长**（即数据通路的宽度，在32位系统里就是**4 字节**，以下仅以32位系统为例），且分块的方法是从内存的第一个字节（地址为0）开始**每 4 个字节一块**，连续划分，由系统（计算机内部电路）控制，不受应用程序影响。

访存时按块读，一次读一块。



(3) 内存对齐的要求

(由以上讨论可知) 为提高**内存物理上的存取效率**，数据在内存里是“**不能随便放**”的，而是要放到**特定地址开始的一些字节单元**中，一般要求如下：

- ◆ 1 字节数据可以随便放，对起始地址没有特别的要求；
- ◆ 2 字节数据要放在从**偶数**地址开始的 2 个字节单元中；
- ◆ 4 字节数据要放在从 **4 的倍数**地址开始的 4 个字节单元中；
- ◆ 8 字节数据要放在从 **4 或 8 的倍数**地址开始的 8 个字节单元中。

—— 这种数据放置规则就是**内存对齐规则**。

基本规则是：数据在内存中的起始地址要是某个**整数 k** 的倍数，
这个**整数 k** 称为该数据类型的**对齐模数**。

对齐模数大小的设置会因系统和编译器而异，具体由编译器决定：

◆ **Windows 32位系统下**，大多编译器默认采用的规则是：

基本数据类型 T 的对齐模数等于 T 的大小，即 sizeof(T)

按此规则，char 型数据可以放在任何地址处，short 型数据要放在**偶数**地址开始处，int、long、float 型数据要放在 **4 的倍数**地址开始处，double 型数据要放在 **8 的倍数**地址开始的地方。

◆ **Linux系统下**，GCC的规则是：

char 型数据的对齐模数是 1，2 字节大小的基本类型数据对齐模数是 2，超过 2 字节的基本类型数据（int、long、double等）的对齐模数都是 4。

下面以Windows 32位系统为例：

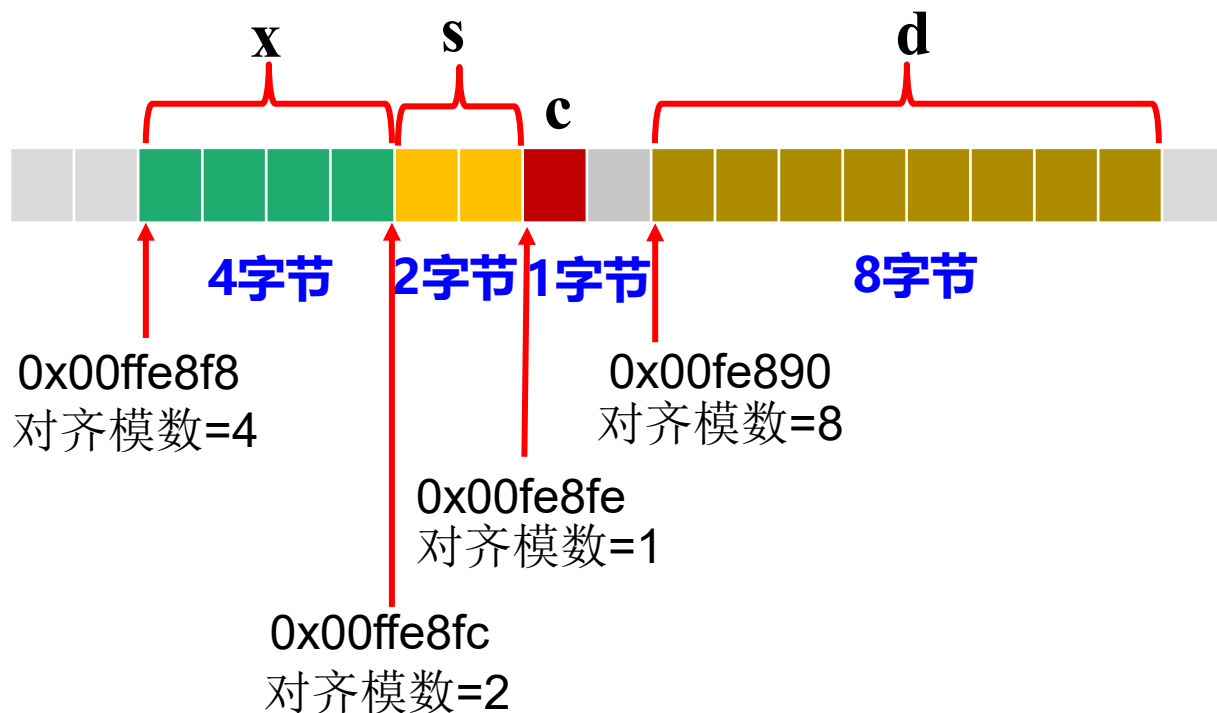
- ◆ **内存对齐**对**简单变量**来说比较“简单”，编译器仅需按照上述规则的要求为变量分配恰当的存储空间即可。

如： **int x;**

short s;

char c;

double d;



◆ 但对结构变量有点麻烦。

对结构变量，内存对齐还有另外 3 条规则：

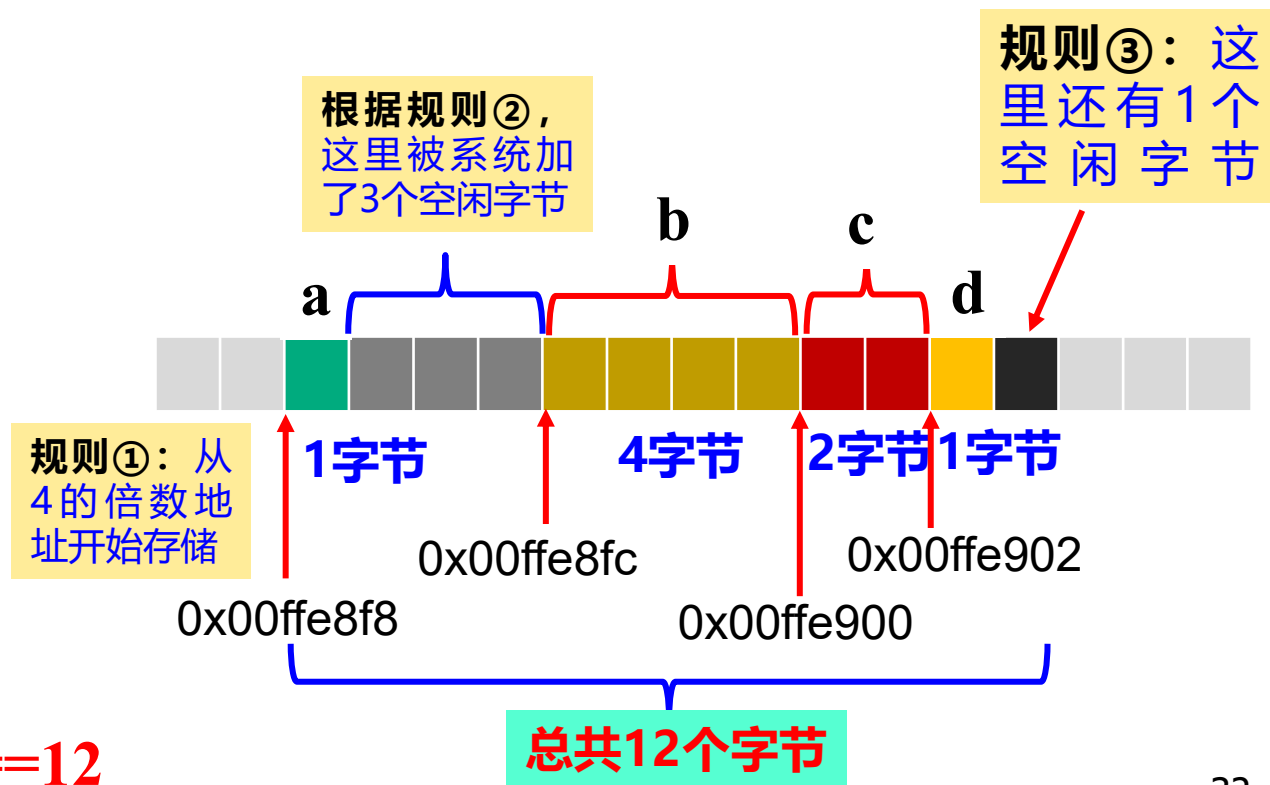
- ① 如果结构体**第一个成员**的基本数据类型长度**不大于 4**，则**结构变量的起始地址**（第一个字节的地址）要是**4 的倍数**；
否则结构变量的起始地址要是**8 的倍数**；
- ② 第一个成员从第一个字节开始（满足对齐要求）；**其后的每个成员都要符合各自类型要求的对齐规则**；
- ③ 整个结构的总大小（**实际占用的总字节数**）要等于**成员中最大数据类型长度的整数倍**。

这就使得结构体实际存储时，**结构体里的数据项在内存里并不一定是“紧密挨着”存储的**，而是中间可能会有些**“空闲”字节**，以使得每个数据项都能够从自己对齐模数的倍数开始的地址处进行存储，**另外还要考虑规则③**。**这就造成结构体实际存储时所需的总字节数不一定简单等于它的数据成员长度之和。**

如：

```
struct st1 {  
    char a;  
    int b;  
    short c;  
    char d;  
};
```

sizeof(struct st1)==12



再如: `struct st2 {`

`char a;`

`char d;`

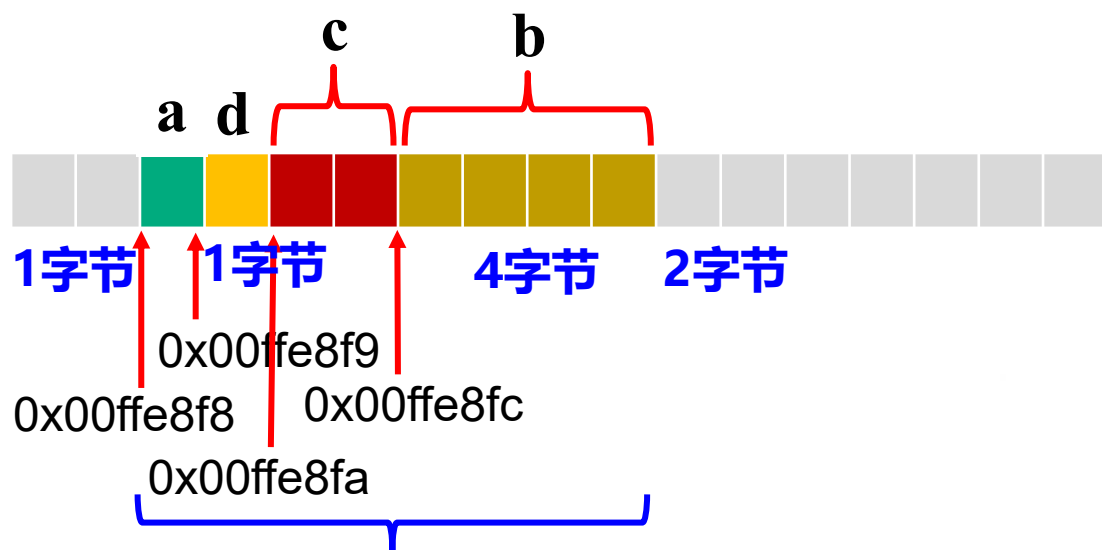
`short c;`

`int b;`

`};`

`sizeof(struct st2)==8`

调整一下成员的顺序



只需 8 个字节, 没有多余的空闲字节

再如: `struct st3 {`

`char a;`

`short c;`

`int b;`

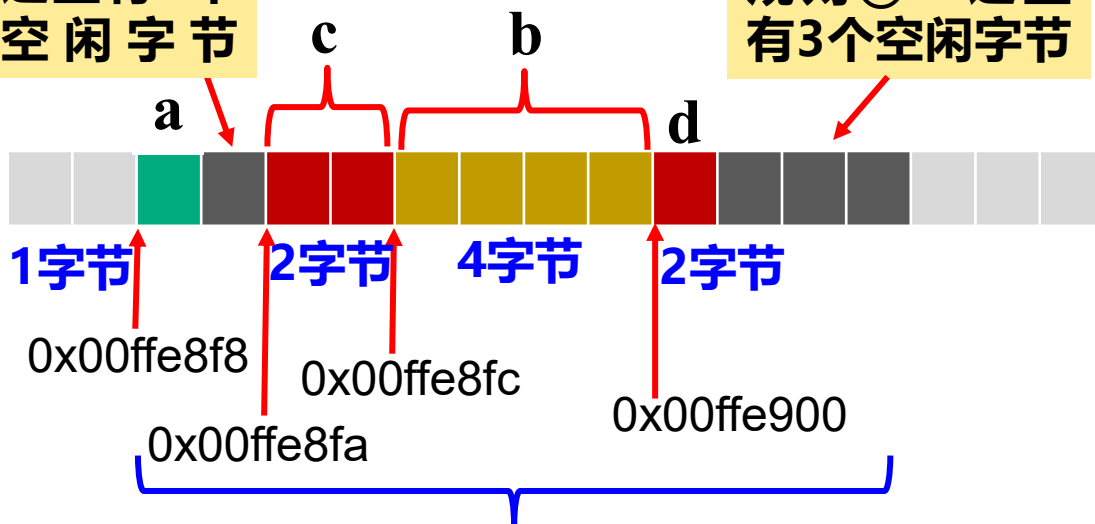
`char d;`

`};`

`sizeof(struct st3)==12`

这里有1个空闲字节

规则③: 这里有3个空闲字节



12个字节, 有4个多余的空闲字节

所以，**设计结构类型时要仔细设计结构体的数据成员：**

不仅是它们的**数据类型**、**数据长度**（如数组大小），还要注意它们的**编排顺序**。**成员的顺序不同，结构体实际占用的字节数量可能不同**。要仔细编排，使得结构体实际占用的空间尽量少。

但内存对齐也只是**系统的一种建议**而不是强制。事实上，任何数据可以存在任意地址处（系统可设置对齐方式），并且都可以完全正确地访问，只是效率可能会慢一点。所以结构体的空间事实上是可以绝对等于理论长度的。

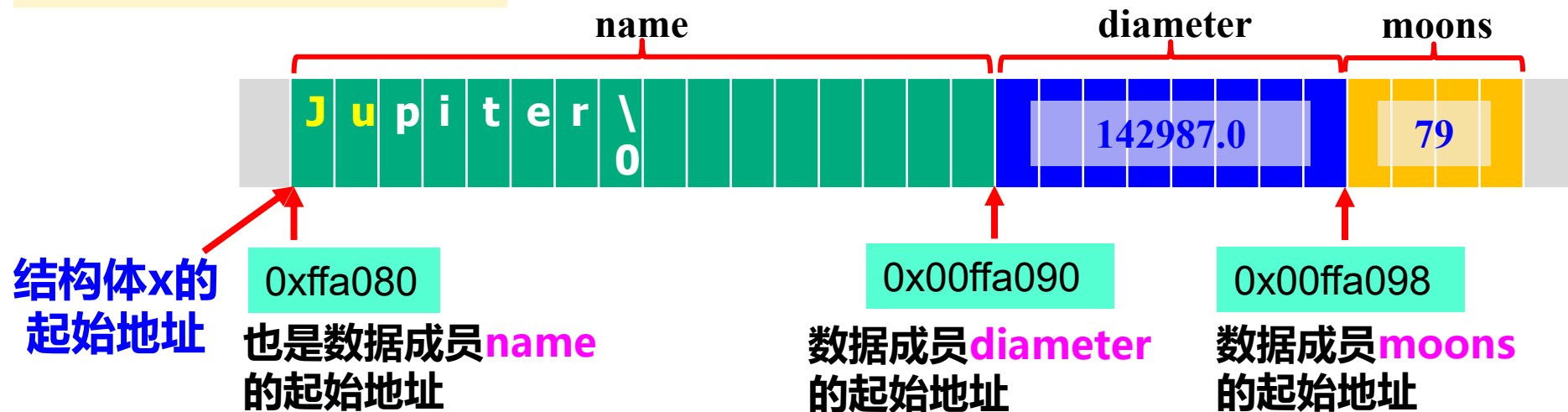
关于内存对齐的其它知识请自行阅读教材**282~284**页并查阅其它相关资料学习。为了描述简单，**以下均不考虑内存对齐的问题**，一般假设结构的所有数据成员连续、顺序“紧密”排列。

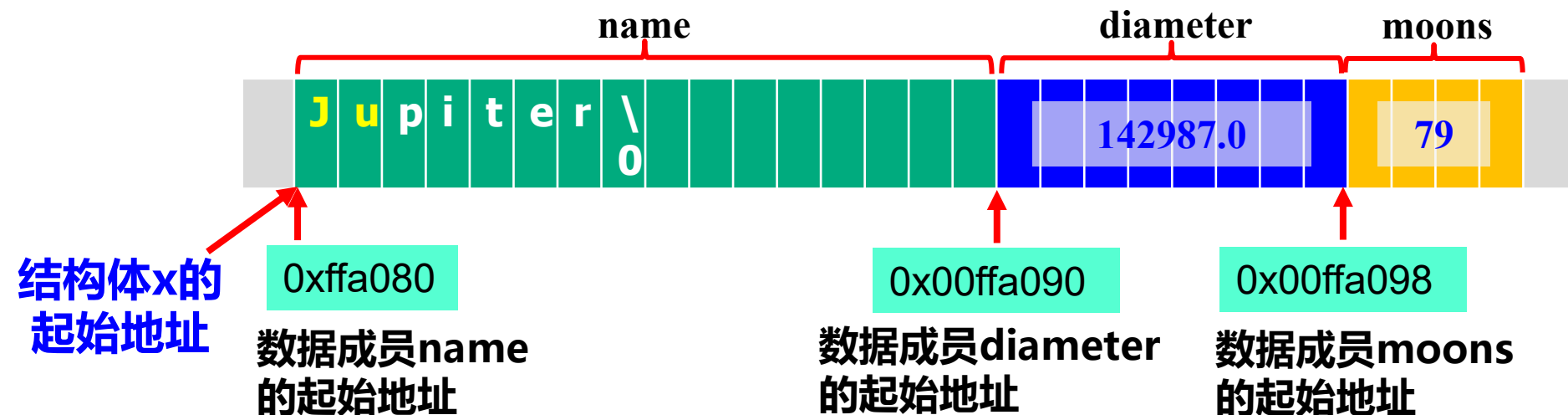
4、结构体的地址

结构体在内存中占有连续的若干字节空间，和其它类型的量一样，其**第一个字节的地址**称为**结构体的地址**。同时，每个成员也有自己的字节空间，**每个成员所占字节空间的第一个字节的地址**称为该**数据成员的地址**。

```
如：struct plane {  
    char name[16];  
    double diameter;  
    int moons;  
} x={"Jupiter", 142987.0, 79};
```

以下均不考虑内存对齐





(1) 可以用 **& 运算**取结构变量的地址，如 **&x**。&x==0xffa080

- ◆ 结构变量的地址是结构体整个空间第一个字节的地址；
- ◆ 结构变量的地址的类型是：**结构类型 ***。

(2) 每个数据成员也有自己的地址

- ◆ 一个成员的地址是这个成员**所占内存单元第一个字节的地址**；
- ◆ **成员地址的类型由成员的数据类型决定。**
- ◆ 可以用 “**&结构变量名.成员名**” 取成员的地址。

(注：若是数组成员，不能用 & 运算，成员数组名就代表其地址)

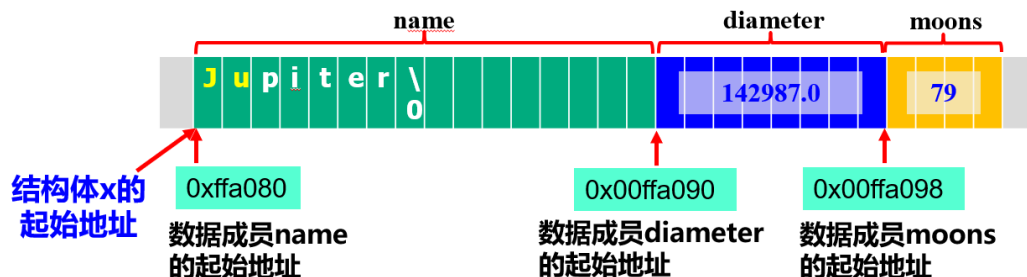
如：&x.diameter、

&x.moons

x.name

name 成员是字符数组，数组名就代表地址

```
struct plane {  
    char name[16];  
    double diameter;  
    int moons;  
} x={"Jupiter", 142987.0, 79};
```



注：结构体整个空间的第一个字节既是整个结构体的第一个字节，也是结构体第一个数据成员的第一个字节，因此**结构体的地址和第一个结构成员**的地址，**值一样，但二者的类型不一样**：

- ◆ 结构体地址的类型是：**结构类型 ***，
- ◆ 第一个成员的地址类型：**由成员的数据类型决定**。

如：

```
struct plane {  
    char name[16];  
    double diameter;  
    int moons;  
} x={"Jupiter",  
    142987.0,  
    79};
```

结构变量 **x** 的地址 (**&x**) 的类型是：**struct plane ***

其第一个数据成员 **x.name** 的地址的类型是：**char ***

另：第二个成员 **x.diameter** 的地址类型是 **double ***
第三个成员 **x.moons** 的地址类型是 **int ***。

9.3 结构变量的引用

对结构变量（结构体）的引用包括两个方面：

- ◆ **对结构变量整体的引用，包括：**

- 为结构变量赋值
- 取结构变量的地址
- 间访结构指针访问结构体
- 结构体作为函数的参数或返回值

- ◆ **对结构变量中数据成员的引用**

- 通过**成员选择运算符**访问结构体的成员。

9.3.1 对结构变量整体的操作

1. 赋值运算：两个同类型的结构变量可以相互赋值。

如：

```
Plane star1, star2={"Jupiter", 142987.0, 79};  
star1=star2;
```

```
typedef struct plane {  
    char name[16];  
    double diameter;  
    int moons;  
} Plane;
```

含义：将赋值号右边结构变量中各个成员的值——**对应**地赋给赋值号左边的结构变量的各个成员。

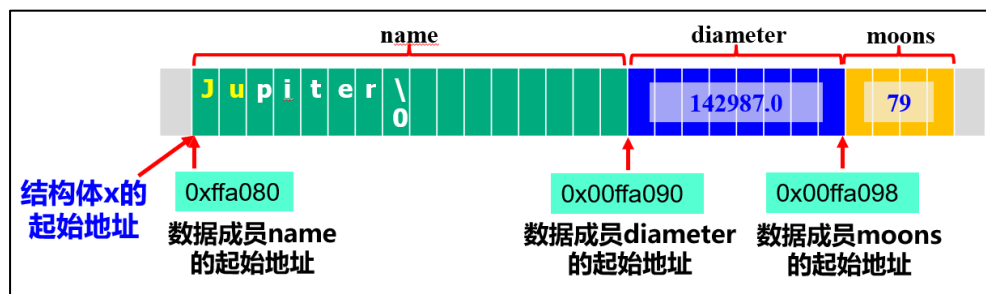
赋值后，结构变量 star1 的成员 name、diameter、moons 的值分别等于 **"Jupiter", 142987.0, 79**，与 star2 成员的值相同。

注：结构体里面的**数组成员**可以直接通过“赋值”实现**数组的自动复制**。这一点与单纯的数组不同。

2. 取结构变量的地址：取地址运算 &

例如：struct planet **star**; $\longrightarrow$ **&star**

结构变量的地址是结构变量内存存储的**第一个字节**的地址，
类型是：**结构类型 ***。



(1) 可以定义**结构类型的指针变量**，令其“**指向**”一个结构体：

struct planet ***p** = **&star**;

(2) 可以通过**间访结构指针**访问指针所指的结构体：***p** $\longrightarrow$ **star**

(关于结构类型的指针问题见9.4节)

9.3.2 对结构成员的引用

通过**成员选择运算符**可以引用结构的成员。有两种具体方式。

(1) 基于**结构变量**引用结构成员

基于**结构变量**引用结构成员使用**成员选择运算符** "."。

此时**成员选择表达式**的一般形式为：

结构变量名. 成员名

其中，

.：成员选择运算符，双目操作符，左操作数是结构变量名，右操作数是结构成员名。**第一优先级，左结合。**

- ◆ 成员选择表达式的值就是**所引用成员的值**；
- ◆ 成员选择表达式的类型就是**所引用成员的数据类型**。

由于**成员选择运算符**具有最高的优先级，所以由它构成的**成员选择表达式**在任何地方都是一个**整体**，表示对结构变量中成员实体的引用，**可以参与同类型数据能参与的所有运算**。

(如同数组中用下标运算符[]对数组元素的引用)

如：struct planet **star**;

strcpy(**star.name**, "Mars");

printf("%lf", **star.diameter**);

int summoons=0;

summoons += **star.moons**;

(2) 基于结构指针引用结构成员

基于**结构指针**引用结构成员需要使用**成员选择运算符 “->”**。

相关内容见9.4节 “结构指针”。

- ◆ 对结构变量**整体**而言，除以上**赋值、取地址、间访、成员选择**运算外，**其余的普通运算均不能施加在结构变量整体上**。
- ◆ **其它对结构的操作都只能作用在结构变量的成员上**，而不能以结构体整体的方式参与。
- ◆ **结构成员能够参与的运算**：只要通过“**成员选择**”引用到该成员，就可以以“**成员选择表达式**”的形式参与该成员类型所允许的所有运算中。

```
如： struct planet star;  
      strcpy(star.name, "Mars");  
      printf("%lf", star.diameter);  
      int summoons=0;  
      summoons += star.moons;
```

*9.3.3 嵌套结构的声明和成员引用

如果一个结构的成员又是另一种结构类型的结构体，则称该结构是**嵌套的结构**。

1、声明嵌套结构的结构类型（简称**嵌套结构类型**）

先声明被嵌套的结构类型，再声明本结构类型。

如 先声明一种日期结构类型

```
struct date {  
    char month[10];  
    short day;  
    int year;  
};
```

再声明一种行星信息结构类型

```
struct plane2 {  
    char name[16];  
    double diameter;  
    int moons;  
    struct date finddate;  
};
```

含有结构型成员

或:

```
typedef struct date {  
    char month[10];  
    short day;  
    int year;  
} Date;  
  
struct plane2 {  
    char name[16];  
    double diameter;  
    int moons;  
    Date finddate;  
};
```

注：嵌套的结构成员的类型必须是其它结构类型，不能是自己的类型。

```
struct plane {  
    char name[16];  
    double diameter;  
    int moons;  
    struct plane star; ×  
};
```


还可以：在嵌套结构类型内部直接声明被嵌套的结构类型，然后再声明其结构成员。

如：

```
struct plane3 {  
    char name[16];  
    double diameter;  
    int moons;  
    struct date2 {  
        char month[10];  
        short day;  
        int year;  
    } finddate;  
};
```

和前面声明方式的区别：

`struct date2` 的“**类型作用域**”
仅在其所在的 `struct plane3` 内部，
仅能在 `struct plane3` **内部使用** 该类型名声明成员，在 `struct plane3` 类型定义的外部不能用 `struct date2` 做其它用途。

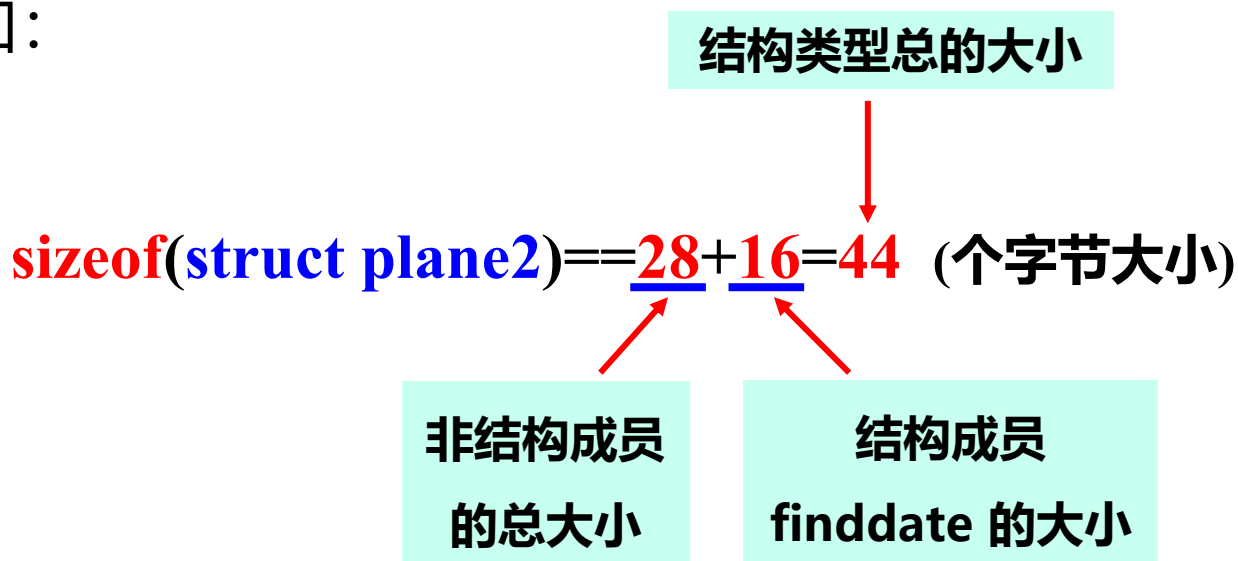
如：单独定义 `struct date2 d;` ❌

2、嵌套结构的大小

嵌套结构的大小也等于其内部包含的所有成员的大小之和。

如果有嵌套的结构成员，则逐层逐一核算每个结构成员的大小，然后累加在一起即可。

如：



```
typedef struct date {  
    char month[10];  
    short day;  
    int year;  
} Date;
```

```
struct plane2 {  
    char name[16];  
    double diameter;  
    int moons;  
    Date finddate;  
};
```

3、嵌套结构的结构变量

用嵌套结构类型声明结构变量的方式完全等同于非嵌套结构：

结构类型 **结构变量名**;

如： **struct plane2 x**;

```
typedef struct date {  
    char month[10];  
    short day;  
    int year;  
}Date;
```

```
struct plane2 {  
    char name[16];  
    double diameter;  
    int moons;  
    Date finddate;  
};
```

4、嵌套结构成员的引用

- ◆ **非结构成员**：“常规方式”：**结构名.成员名**
- ◆ **结构成员**：根据**嵌套的层数**，利用**多个成员选择运算符“.”**
连接构成的**多级成员选择表达式**引用结构成员
的成员。

如：引用两层嵌套结构中的结构成员的成员，**两级成员选择表达式**的基本形式为：

结构变量名. 结构成员名. 成员名



第一层的结构成员名



第二层结构成员的成员名

注：成员选择运算符“.”为**左结合性**，上述两级成员选择表达式等价于：**(结构变量名.结构成员名).成员名**

对更多层嵌套的结构，引用成员的形式以此类推。

如:

```
struct date {  
    char month[10];  
    short day;  
    int year;  
};  
  
struct plane2 {  
    char name[16];  
    double diameter;  
    int moons;  
    struct date finddate;  
};
```

struct planet2 star;

用**两级成员选择表达式**引用 **star**
中结构成员 **finddate** 的成员:

star.finddate.year

star.finddate.month

star.finddate.day

多级成员选择表达式语法上同样被视为一个整体，代表被引用的成员，可以参与该成员类型允许参与的所有运算。

如： `struct plane2 st;`
`strcpy(st. finddate. month, "Oct.");`
`st. finddate.day = 30;`
`st. finddate.year = 2023;`
`printf("%s%hd, %d", st. finddate. month, st. finddate.day,`
`st. finddate.year);`

输出： **Oct.30, 2023**

或： `scanf("%s%hd%d", st.finddate. month, &st.finddate.day,`
`&st.finddate.year);`

先引用成员，再取成员的地址

4、嵌套结构体的存储

尽管嵌套结构体类型的内部有嵌套所带来的**层次性**组织关系，但存储时，一个结构体的所有成员（包括内部嵌套的结构成员）按照基本数据成员（非嵌套形式）**展开成一个线性排列的一层结构**，然后按照它们的**自然组织顺序**依次排列。如：

```
typedef struct date {  
    char month[10];  
    short day;  
    int year;  
}Date;
```

```
struct plane5 {  
    char name[16];  
    double diameter;  
    Date finddate;  
    int moons;  
};
```

按基本数据成员展开，所有基本数据成员成**一层结构**顺序排列

```
{  
    char name[16];  
    double diameter;  
    char month[10];  
    short day;  
    int year;  
    int moons;  
};
```

结构成员的基本成员也依次排列

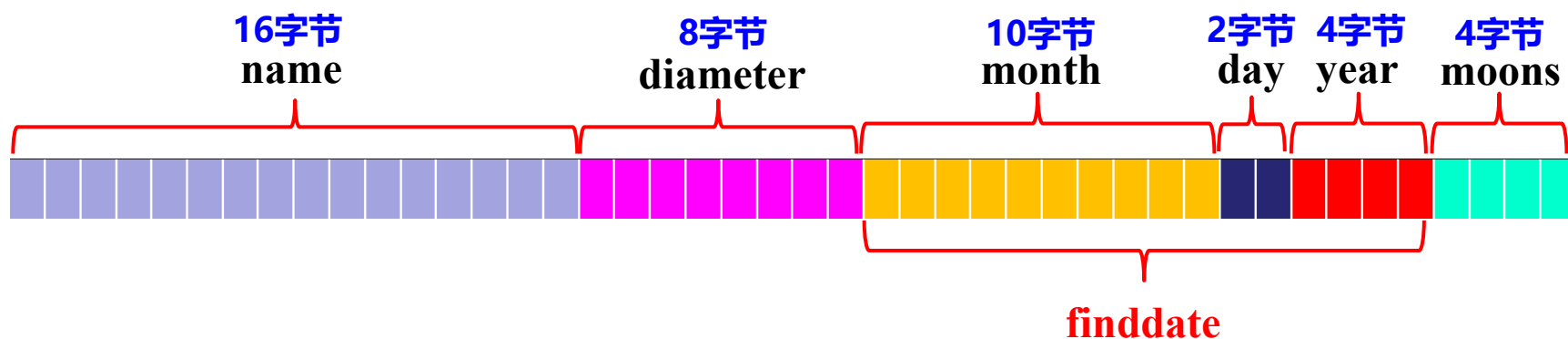
依据上述基本数据成员的排列形式：

- ◆ 首先为结构体分配总量等于该嵌套结构类型大小的**连续字节空间**；
- ◆ 然后**分成若干段**，各段大小依次等于顺序排列下各对应数据成员的大小；
- ◆ 最后各段依次存储相应数据成员的数据。

```
{
    char name[16];
    double diameter;
    char month[10];
    short day;
    int year;
    int moons;
};
```

```
typedef struct date {
    char month[10];
    short day;
    int year;
}Date;

struct plane5 {
    char name[16];
    double diameter;
    Date finddate;
    int moons;
};
```



5、嵌套结构体及其成员的地址

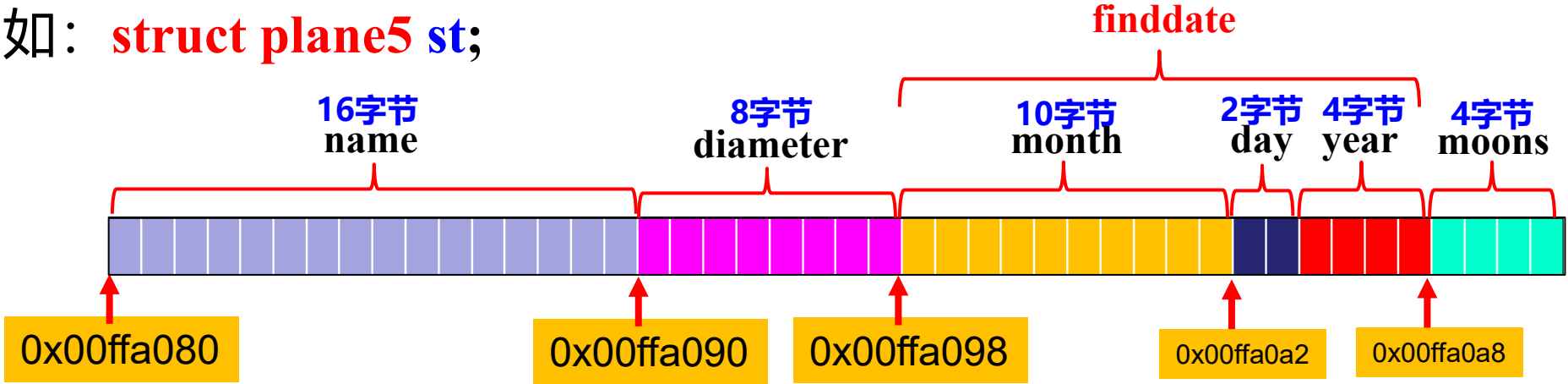
- ◆ **嵌套结构变量的地址**：整个结构体空间的第一个字节的地址；
结构变量地址的类型是：**结构类型 ***。
- ◆ **每个数据成员的地址**：是成员所占内存单元第一个字节的地址；
成员地址的类型由成员的数据类型决定。
- ◆ **用 & 取地址**：结构体、结构体的非结构成员、结构成员、以及嵌套结构成员的成员都可以直接用 **& 取地址**（注：数组成员除外）。

如：struct plane5 st; 见后。

```
typedef struct date
{
    char month[10];
    short day;
    int year;
}Date;
```

```
struct plane5 {
    char name[16];
    double diameter;
    Date finddate;
    int moons;
};
```

如: **struct plane5** st;



表达式	值	类型
&st	0x00ffa080	struct plane5 *
st.name	0x00ffa080	char *
&st.diameter	0x00ffa090	double *
&st.moons	0x00ffa0a8	int *
&st. finddate	0x00ffa098	struct date *
st. finddate. month	0x00ffa098	char *
&st. finddate.day	0x00ffa0a2	int *
&st. finddate.year	0x00ffa0a4	int *

9.4 结构指针和结构指针变量

9.4.1 结构的指针

结构体存储空间中的**第一个字节的地址**称为**结构体的地址**，简称为**结构的地址** 或 **结构体的指针**、**结构的指针**。

不失一般性，设结构类型为 **struct T**，则：

```
struct T
{
    ...
};
```

- ◆ 结构的地址（指针）类型是：**struct T ***
- ◆ 若有该类型的结构体 **struct T x;**，则取 x 的地址为：**&x**

注：结构的指针代表结构体在内存中的**整体存在**，而不是其内部的哪个数据成员。

和所有地址型数据一样：

- ◆ **结构的地址也是一个长度等于机器字长的特殊无符号整数；**

【在32位系统中地址的长度等于 4 字节、32位】

- ◆ **符合所有地址型数据（指针）的基本运算规则：**

- **结构指针可以+或-一个整数：** 表示计算结构元素地址偏移；
- **两个结构指针可以比较大小：** 表示判断它们所指内存位置的前后或是否相等；
- **两个结构指针可以相减：** 表示计算两个指针之间的元素数。

以上运算一般要基于结构数组进行，对独立的结构变量没有实际意义。

9.4.2 结构指针变量

基类型为结构类型的指针变量就是**结构指针变量**，在上下文不混淆的情况下，一般简称为**结构指针**。

1、结构指针变量的声明

结构指针声明的一般方式：**结构类型名 * 结构指针变量名;**

如：**struct T * p;**

(1) 可以用结构体（结构变量）的地址为结构指针（变量）赋值，赋值后称**结构指针“指向”该结构体**。

如：**struct T x;**

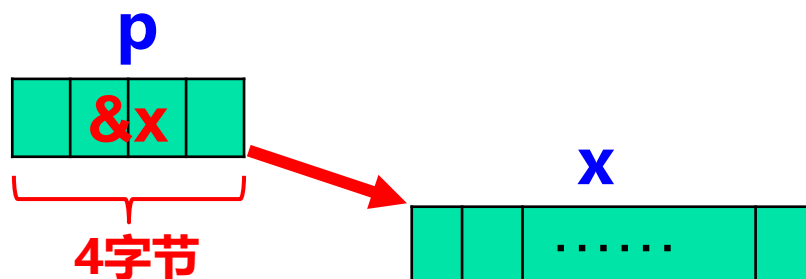
p=&x;

```
struct T
{
    ...
};
```

- (2) 和其它类型的指针变量一样，**结构指针变量只能指向基类型一致的结构体**，类型要匹配；
- (3) 和其它类型的指针变量一样，**结构指针变量的长度是 4 字节 (32位系统)**，**其值等于所指结构体的地址**：

如，若有：

```
struct T x, *p=&x;
```



(4) 和其它类型的指针变量一样，**结构指针变量的运算符合所有指针变量运算的一般规则：**

- **结构指针可以+或-一个整数n**：表示计算地址偏移而指向元素偏移为 n 的下一个元素；
- **两个结构指针可以比较大小**：表示判断它们所指对象在内存中存储位置的前后或是否相等；
- **两个结构指针可以相减**：表示计算它们所指对象的位置之间有几个元素。

同理，**以上运算都要基于结构数组进行**，对指向独立存在的结构体的结构指针变量，施加上述运算没有实际意义。

(5) 可以在声明结构类型的同时声明结构指针变量。

如: `struct tstru`

{

...

} `x, *p=&x;`

在定义结构指针的时候为其赋初值
称为（结构指针）**初始化**。

- 上述声明中，`x` 和 `p` 的先后顺序不可颠倒。**任何变量都要先声明后使用。**
- **结构指针要通过赋初值或单独赋值，使其指向特定对象后再使用，杜绝使用“悬挂指针”。**

9.4.2 间访结构指针

间访结构指针表达式的一般形式：

*** 结构指针**

- ◆ **间访结构指针**表示引用指针**所指对象的实体**（结构变量）
 - 间访结构指针表达式的值：就是**其所指的结构对象的"值"**；
 - 间访结构指针表达式的类型：就是**其所指结构对象的类型**。

如，设有

```
struct T
{
    ...
} x, *p=&x;
```

则：***p 等于 x**

9.4.3 通过结构指针引用结构成员

方式1：先 **间访结构指针** 得到所指对象（结构变量），然后再用成员选择符 “.” 引用结构成员：

(*结构指针).成员名

该表达式表示引用结构成员的实体：

- ◆ 该表达式的**值**即**成员值**
- ◆ 该表达式的**类型**即**成员的数据类型**。

注：根据运算符优先级顺序，上述表达式的**括号不能少**，
否则含义不对： *** 结构指针.成员名** **×**

```
struct plane {  
    char name[16];  
    double diameter;  
    int moons;  
} star, *ps=&star;
```



(*ps).name	等价于 star.name
(*ps).diameter	等价于 star.diameter
(*ps).moons	等价于 star.moons

- ◆ 表达式 **(*结构指针).成员名** 作为**成员实体**，可以参与成员可以参与的所有运算，如：

```
strcpy((*ps).name, "Mars");  
printf("%lf", (*ps).diameter);  
int summoons=0;  
summoons += (*ps).moons;
```

方式2：通过 **成员选择运算符 “->”** 访问结构成员

基于结构指针、使用成员选择运算符 “->” 引用结构成员的表达式的一般形式：

结构指针->结构成员名

其中，

- ◆ **->：成员选择运算符，双目运算符，第一优先级、左结合。**
- ◆ **表达式含义：**访问结构指针所指对象（结构变量）中指定的结构成员。
- ◆ 该表达式的**值**为所访问的**结构成员的值**。
- ◆ 该表达式的**类型**为所访问的**结构成员的数据类型**。

如果一个结构指针指向了一个结构体，如 **ps=&star**；则：

“结构指针->成员名” 与 **“结构变量.成员名”** 等价；但：

- ◆ **“->”** 只能用于结构指针，不能用于结构变量；
- ◆ **“.”** 只能用于结构变量，不能用于结构指针。

如：

```
struct plane {  
    char name[16];  
    double diameter;  
    int moons;  
} star, *ps=&star;
```



```
ps->name ✓  
ps.name ✗  
star.name ✓  
star->name ✗  
ps->diameter ✓  
ps.diameter ✗  
star.diameter ✓  
star->diameter ✗
```

注：“结构指针->成员名”与“(*结构指针).成员名”的细微区别：

- ◆ 通常认为“结构指针->成员名”的访问效率更高：

“(*结构指针).成员名”方式需要进行**二次操作**才能访问到结构成员：

- 先通过“*”间接访问到指针所指的结构对象；
- 然后再通过“.”完成对结构成员的访问。

- ◆ 书写形式麻烦。

“结构指针->成员名”的方式更加简洁、高效。

利用结构指针访问嵌套结构中结构成员的成员

如：

```
struct date {  
    char month[10];  
    int day;  
    int year;  
};
```

```
struct plane2 {  
    char name[16];  
    double diameter;  
    int moons;  
    struct date finddate;  
};
```

设有： `struct planet2 s, *ps = &s;`

则： `ps->finddate.year` 等价于 `x.finddate.year`

`ps->finddate. day` 等价于 `x.finddate. day`

`ps->finddate. year` 等价于 `x.finddate. year`

即先通过 `->` 引用到结构成员 (实体)，再引用结构成员的成员。

设: **struct T** {

int n;

char ***pm**;

} **s**={**10**,**"abcdef"**}, ***p**=**&s**;

注: 结构体 s 中的成员 pm
指向一个字符串常量

正确理解下表中的表达式

表达式	值与类型	表达式执行的操作	等价的表达式
++p->n	11 , int	访问 n 并使其增1	++s.n
p->n++	10 , int	访问 n, 取其原值参与运算, 再使 n 增 1	s.n++
*p->pm	'a' , char	访问 pm, 得所指字符 'a'	*s.pm
*p->pm++	'a' , char	访问 pm, 得所指字符 'a', 然后使 pm 增 1 指向 'b'	*s.pm++
*++p->pm	'b' , char	先引用 pm, 使其增 1, 指向 'b', 再访问 pm, 得所指字 符 'b'	*++s.pm

分析：表达式 **(*p->pm)++** 和 **++*p->pm** 正确吗？

◆ **编译没问题，但是运行时出错。**

原因：**p->pm**所指字符串 **“abcdef”**
是常量字符串，内容不得修改。

```
struct T{  
    int n;  
    char *pm;  
}s={10,"abcdef"}, *p=&s;
```

◆ **(*p->pm)++**或**++*p->pm**在访问到pm所指字符对象 'a' 之后，都企图对其进行自增操作。但字符串常量不允许更改，因此是非法的。

当表达式中同时出现 **->**、**.**、*****、**++**、**--**时，要严格**按照运算符的优先级和结合性**，以及**前缀++/--**、**后缀++/--**的操作语义，正确解析并计算表达式的值。

9.5 结构数组

元素为**结构类型**的数组称为**结构数组**。

1、结构数组的声明方式：

- ◆ 一维结构数组的声明：

结构类型名 结构数组名[常量表达式];

- ◆ 多维结构数组以此类推

2、一维结构数组的应用：构造一个多行多列的**二维表格**。

- ◆ **每行是一个元素**，称为**记录**；
- ◆ **每列是元素的一个数据项**（或称为对象的**属性**）。

如学生登记表、通信录等。

例：设有**学生登记表**二维表格如下：

列是学生记录的各项属性

表头

30行，每行是一个学生的信息记录

num	name	sex	score
U20230001	Li	m	89.5
.....			
U20230030	Zhang	f	92.0

可以定义**结构类型及数组**描述该二维表格：

```
struct stud {
```

结构类型给出了二维表格的表头结构

```
    char num[12];    /* 学号 */
```

```
    char name[9];    /* 姓名 */
```

```
    char sex;        /* 性别 */
```

```
    float score;     /* 成绩 */
```

数据项对应二维表格的各列

```
} students[30];
```

定义结构数组，保存表格中30个学生的记录

再设，有**复杂表头**的**二维表格**如下：

num	name	sex	birthday			score
			month	day	year	
U20030001	Li	m	10	31	2005	89.5
.....						

可以定义**嵌套结构类型**描述该二维表格：

```
struct stud {
```

```
    char num[12];           /* 学号 */
```

```
    char name[9];          /* 姓名 */
```

```
    char sex;               /* 性别 */
```

```
    struct date birthday;   /* 生日 */
```

```
    float score;            /* 成绩 */
```

```
} students[30];
```

数据项对应表格
的各列；
嵌套的结构成员
表示复杂表项。

```
struct date {  
    char month[10];  
    short day;  
    int year;  
};
```

3、结构数组的初始化

结构数组的初始化由**初值表**给出，基本形式如下：

例， **struct stud** **students**[3]={
 { "U20230001", "Li", 'm', {"Oct",31,2023}, 89.5},
 { "U20230002", "Zhang", 'f', {"June",12,2024}, 90.0},
 { "U20230003", "Yang", 'm', {"May",1,2023}, 75.3}
};

- ◆ **外层花括号**界定整个初值表；
- ◆ **内层花括号**界定每个元素的初值（如果是嵌套结构类型，可以再嵌套多层的花括号）。
- ◆ **赋初值的方式**：用每个元素的初值为数组元素**依次顺序赋值**。

- ◆ 如果没给出数组大小，编译器可以根据**初值的个数**定义数组的大小，如：

```
struct stud students[ ]={  
    { "U20230001", "Li", 'm', {"Oct",31,2023}, 89.5},  
    { "U20230002", "Zhang", 'f', {"June",12,2024}, 90.0},  
    { "U20230003", "Yang", 'm', {"May",1,2023}, 75.3}  
};
```

- ◆ 初值表如果给出了 **“完整” 的初值**，内层嵌套的花括号也可以省略，如：

```
struct stud students[ ]={  
    "U20230001", "Li", 'm', "Oct",31,2023, 89.5,  
    "U20230002", "Zhang", 'f', "June",12,2024, 90.0,  
    "U20230003", "Yang", 'm', "May",1,2023, 75.3  
};
```

- ◆ 初值表可以不用给出所有元素的初值，但**只能缺少后面元素的初值**，不能缺少前面或中间的，如：

```
struct stud students[10]={  
    "U20230001", "Li", 'm', "Oct",31,2023, 89.5,  
    "U20230002", "Zhang", 'f', "June",12,2024, 90.0,  
    "U20230003", "Yang", 'm', "May",1,2023, 75.3  
};
```



```
struct stud students[10]={  
    "20230001", "Li", 'm', "Oct",31,2023, 89.5,  
    , , ,  
    "20230002", "Zhang", 'f', "June",12,2024, 90.0,  
    "20230003", "Yang", 'm', "May",1,2023, 75.3  
};
```



- ◆ 在有内层元素的花括号时，每个元素的初值也可以有缺省，但也只能**缺省后面数据项的初值**，不能缺省前面或中间数据项的初值。如：

```
struct stud students[ ]={  
    { "U20230001", "Li", 'm', {"Oct",31}, 89.5},  
    { "U20230002", "Zhang", 'f', {"June",12,2024}},  
    { "U20230003", "Yang", 'm'}  
};
```



```
struct stud students[ ]={  
    { "20230001", , 'm', {"Oct",31, 2023}, 89.5},  
    { "20230002", "Zhang", 'f', , 90.0},  
    { "20230003", "Yang", 'm', '男', { , 1, 2023}, 75.3}  
};
```



4、结构数组元素的引用

和其它类型的数组一样，对结构数组元素引用的基本方式也是“**数组名+下标**”：

- ◆ 引用一维结构数组的元素：**数组名[下标]**

如：**students[0]**、**students[1]**、**students[2]**

注：一维结构数组的元素是基本元素，是普通的结构变量

- ◆ 多维结构数组元素的引用：以此类推。

如二维结构数组：**数组名[下标1][下标2]**

另，多维结构数组也有**基本元素**、“**超数组元素**”等相关性质，同于普通类型的多维数组，自行思考。

5、结构数组元素的成员的引用

先通过 “**数组名+下标**” 引用到结构数组的元素（指**基本元素**），
然后基于数组元素再用 **成员选择符 “.”** 引用元素的数据成员。

引用一维结构数组元素的成员：**数组名[下标].成员名**

如：**students[0].num** **students[1].name**

- ◆ 根据'**[]**'和'**.**'的左结合性，**数组名[下标].成员名** 等价于
(数组名[下标]).成员名
- ◆ 运算表达式中，**数组名[下标].成员名** 视为一个**整体参与运算**。
- ◆ 该性质同理适用于多维数组中元素成员的引用。

例 结构数组 `struct stud students[3]` 的使用举例。

```
int main(void)
```

```
{  int i;
```

```
    for(i=0;i<3;i++){
```

```
        printf("input num: ");  scanf("%s", students[i].num);
```

```
        printf("input name: ");  scanf("%s", students[i].name);
```

```
        printf("input birthday: ");  scanf("%s%d%d", students[i].birthday.month,  
                                            &students[i].birthday.day, &students[i].birthday.year);
```

```
        printf("input score: ");  scanf("%f", &students[i].score);
```

```
    }
```

```
    for(i=0;i<3;i++){
```

```
        printf("num:  %s\n", students[i].num);
```

```
        printf("name: %s\n", students[i].name);
```

```
        printf("birthday: %s %d, %d\n", students.birthday.month,  
                                         students.birthday.day, students.birthday.year);
```

```
        printf("score: %8.2f\n", students[i].score);
```

```
    }
```

```
    return 0;
```

```
}
```

```
struct stud {  
    char num[12];  
    char name[9];  
    char sex;  
    struct date birthday;  
    float score;  
};
```

6、结构数组的存储

首先：系统为结构数组分配一个**连续字节空间**：

字节总数等于 = $N * \text{sizeof}(\text{结构类型})$

(N为结构数组的大小 (元素数))

然后：将整个字节空间被分成**N段**，每段存储一个数组元素；

所有的元素按照下标顺序依次、连续存放。

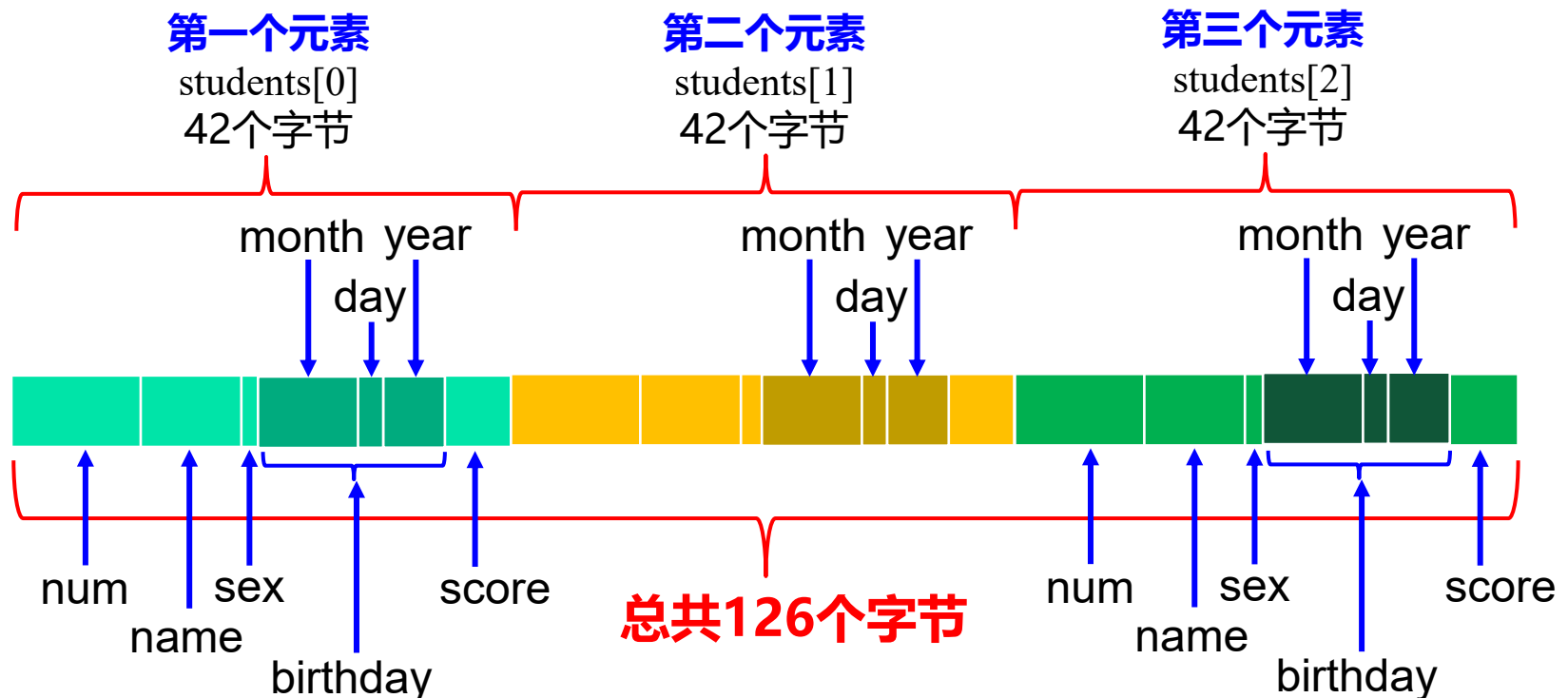
再者：每段内按照普通结构变量的存储方式存储一个元素内部各数据成员的数据。

以行序为主序

如: **struct stud** {
 char num[12];
 char name[9];
 char sex;
 struct date birthday;
 float score;
} **students**[3];

```
struct date {  
    char month[10];  
    short day;  
    int year;  
};
```

sizeof(struct stud)==42
(考虑内存对齐是44)



7、结构数组的地址

(1) 结构数组的地址

内存中，结构数组所占字节空间的第一个字节的地址称为**结构数组的地址**。

- ◆ **结构数组名**代表结构数组的（起始）地址，其值就等于数组的地址值，注：**不能用 ‘&’ 取数组名的地址**。
- ◆ **数组名是地址常量**，其值不能被修改。
- ◆ 对一维结构数组而言，若数组类型是 **struct T**，则其的地址就是 **struct T *** 类型的；
- ◆ **一维结构数组的数组名是指向其第一个元素的指针**。

多维数组的相关性质同于其它数据类型的多维数组，自行思考

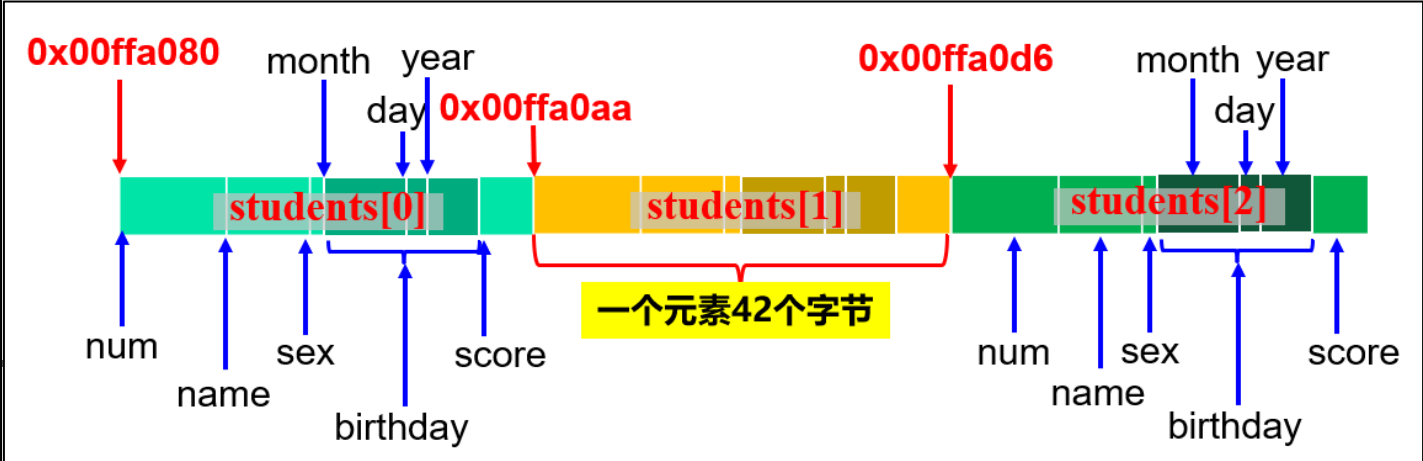
(2) 结构数组中元素的地址

数组元素所占字节空间的第一个字节的地址称为**元素的地址**。

- ◆ 若结构数组元素的类型是 **struct T**，则其元素的地址类型是 **struct T ***。可以用 '**&**' 取每个元素的地址。
- ◆ 由于第一个元素的第一个字节也是整个结构数组的第一个字节，所以**第一个元素的地址的值和结构数组的地址的值相等，但类型不一定一致**：
 - 对于一维结构数组，结构数组的地址的值和类型与其第一个元素的地址的值和类型都相同。
 - 对于二维及以上的高维数组，结构数组地址的类型将不同于其第一个基本元素的地址的类型。

```
struct stud {  
    char num[12];  
    char name[9];  
    char sex;  
    struct date birthday;  
    float score;  
} students[3];
```

```
struct date {  
    char month[10];  
    short day;  
    int year;  
};
```



表达式	值	类型
<code>students</code>	<code>0x00ffa080</code>	<code>struct stud *</code>
<code>&students[0]</code>	<code>0x00ffa080</code>	<code>struct stud *</code>
<code>students[0].num</code>	<code>0x00ffa080</code>	<code>char *</code>
<code>students[0].name</code>	<code>0x00ffa08c</code>	<code>char *</code>
<code>&students[0].sex</code>	<code>0x00ffa095</code>	<code>char *</code>
<code>&students[0].birthday</code>	<code>0x00ffa096</code>	<code>struct date *</code>
<code>students[0].birthday.month</code>	<code>0x00ffa096</code>	<code>char *</code>
<code>&students[0].birthday.day</code>	<code>0x00ffa0a7</code>	<code>short int *</code>
<code>&students[0].birthday.year</code>	<code>0x00ffa0a2</code>	<code>int *</code>
<code>&students[0].score</code>	<code>0x00ffa0a6</code>	<code>float *</code>
<code>&students[1]</code>	<code>0x00ffa0aa</code>	<code>struct stud *</code>

8、用结构指针变量指向结构数组的元素

- (1) 声明**结构指针变量**，然后用结构数组中**元素的地址**为结构指针变量赋值，使其指向元素。

例如， `struct stud *p;`

◆ `p=students;` `/* 使p指向结构数组中的第一元素 */`

或 `p=&students[0];`

◆ `p=&students[1];` `/* 使p指向结构数组中的第二元素 */`

- (2) **间访结构指针** 得到结构指针所指对象

如 `*p` 等价于 `p` 当前所指的元素。

- (3) **通过 “->” 引用元素的数据成员**

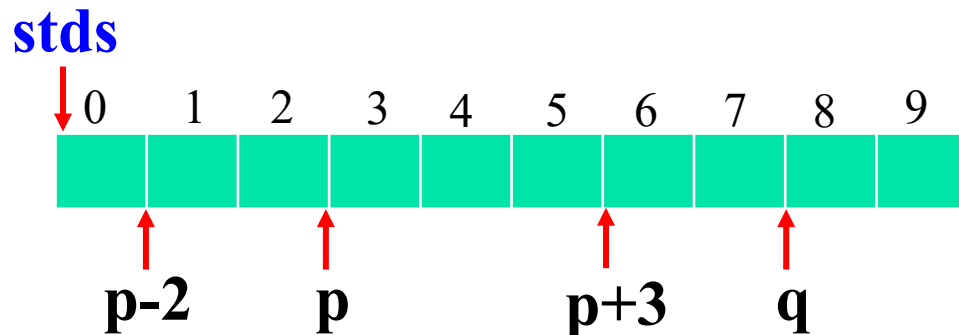
如 `p->num`，引用 `p` 当前所指元素的 `num` 成员

(4) 指向结构数组元素的指针变量的算术运算

设 `struct stud stds[10];`

`struct stud *p, *q;`

若 `p = stds[3], q = stds[8];`



则 (设以下表达式相互独立, 均基于 `p` 和 `q` 初始值计算)

`*(p+3)`: 引用数组元素 `stds[6]`

`++p`: 使 `p` 指向 `stds[4]`

`*(p-2)`: 引用数组元素 `stds[1]`

`--p`: 使 `p` 指向 `stds[2]`

`p < q`: 为真, `p` 指示的位置在 `q` 指示的位置之前

`q-p`: 等于 5, 即从 `p` 到 `q` 之间的元素数

继续设 `struct stud stds[10];`

`struct stud *p=stds[3];`

```
struct stud {  
    char num[12];  
    char name[9];  
    char sex;  
    struct date birthday;  
    float score;  
};
```

分析以下表达式

- ◆ **`(++p)->num`**：先使p自增1而指向stds[4]，然后访问stds[4]成员num。
- ◆ **`++p->num`**：先引用p当前所指元素stds[3]的num成员，然后试图使num自增1，但因为num是**字符数组名**，所以该操作为**非法运算**。
- ◆ **`++p->score`**：引用p当前所指元素stds[3]的score成员，使其自增1，合法。
- ◆ **`(p++)->num`**：先引用p当前所指元素stds[3]的num成员，然后p自增1而指向下一个元素stds[4]，stds[3].num没有变化，合法。
- ◆ **`p++->num`**：根据++和->的结合性，该式等同于上式。

继续设 `struct stud stds[10];`

`struct stud *p=stds[3];`

```
struct stud {  
    char num[12];  
    char name[9];  
    char sex;  
    struct date birthday;  
    float score;  
};
```

分析以下表达式

- ◆ **`*(p++)->num`**: 先引用p当前所指元素stds[3]的num成员，然后间访num（字符数组名）得到stds[3]->num[0]，再然后p自增1而指向stds[4]。
- ◆ **`*p++->num`**: 根据优先级、结合性分析，该式等价于上式。
- ◆ **`*(++p)->name`**: p自增1后指向stds[4]，然后引用stds[4]的成员name，再间访该成员名，得到stds[4]->name[0]。
- ◆ **`*++p->score`**: 先引用p当前所指元素stds[3]的score成员，然后使score的值自增1，再后“间访”score，但**score不是指针**，所以是**非法操作**。

当表达式含有 “.” 、 “->” 、 “*” 、 “++” 、 “--” **混合运算**时，要严格按照运算符的**优先级和结合性**来进行分析。

9.6 结构和函数

- (1) 函数的参数可以是**结构类型**、**结构指针类型**或**结构数组类型**;
- (2) 函数的返回值可以是**结构体**、**结构指针**。

以下以结构类型 **struct stud** 为例进行说明

```
struct stud {  
    char num[12];  
    char name[9];  
    char sex;  
    struct date birthday;  
    float score;  
};  
  
struct date {  
    char month[10];  
    short day;  
    int year;  
};
```

1、结构体作为函数的参数

- ◆ **结构体**作为函数的参数，参数传递采用“**值传递**”的方式。

例：设有输出学生信息的函数**ShowStud**。

```
void ShowStud(struct stud s)
```

```
{
```

```
    printf("num: %s\n", s.num);
```

```
    printf("name: %s\n", s.name);
```

```
    printf("sex: %c\n", s.sex);
```

```
    printf("birthday: %s %d, %d\n",
```

```
        s.birthday.month, s.birthday.day,
```

```
        s.birthday.year);
```

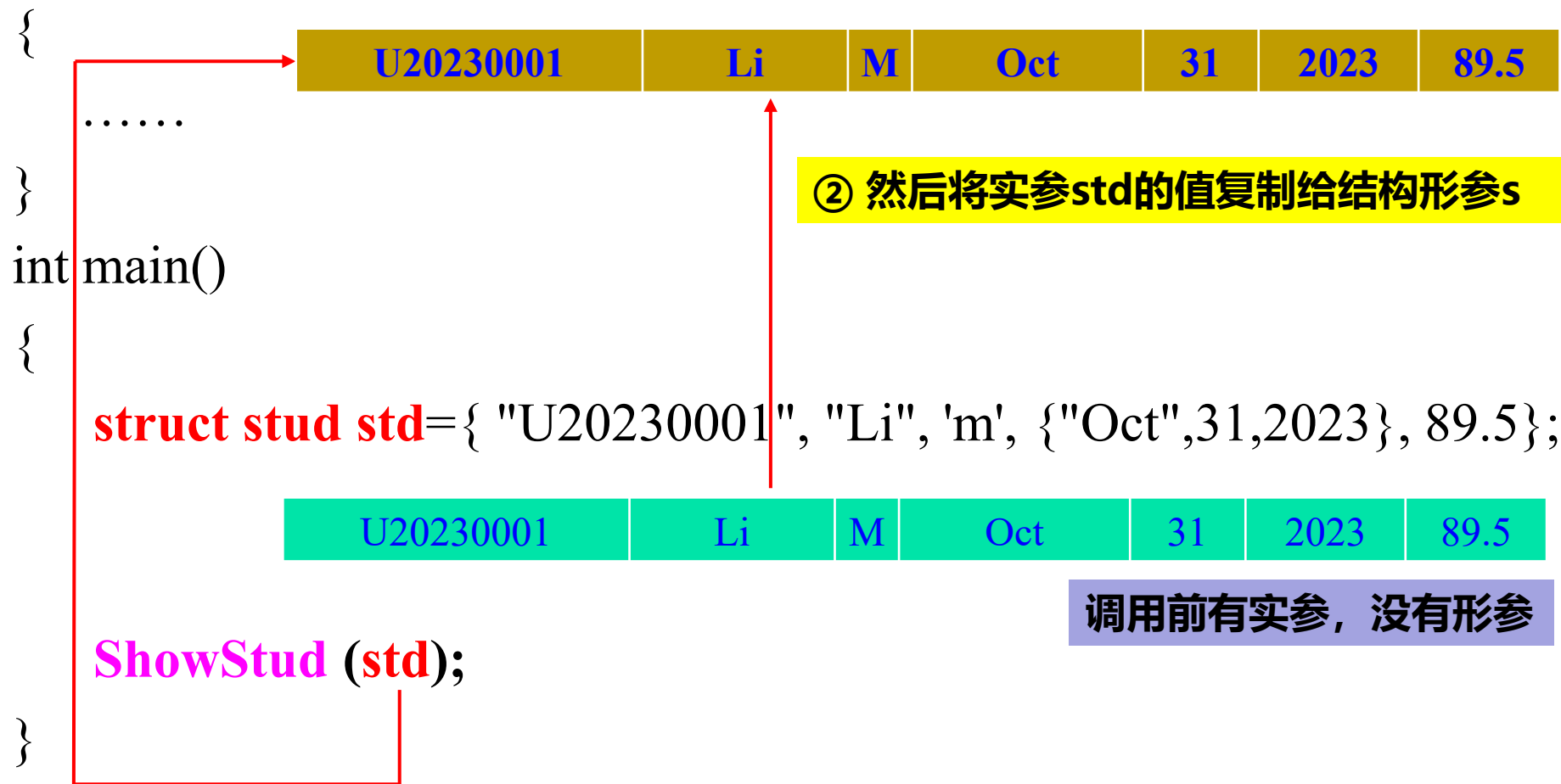
```
    printf("score: %f\n", s.score);
```

```
}
```

结构体形参就是普通的**结构变量**，在函数内部按照结构变量的一般规则使用即可。

◆ 结构体做参数，函数调用时采用“**值传递**”，将**实参复制**给**形参**

```
void ShowStud(struct stud s)
```



① 在调用之时，首先为结构形参s分配存储单元

2、结构指针作为函数的参数

- ◆ 函数的参数可以是**结构指针**，此时的参数传递采用“**址传递**”。

例：设有输出学生信息的函数**ShowStudPtr**

```
void ShowStudPtr(struct stud * sp)
```

```
{  
    printf(“num: %s\n”, sp->num);  
    printf(“name: %s\n”, sp->name);  
    printf(“sex: %c\n”, sp->sex);  
    printf(“birthday: %s %d, %d\n”,  
           sp->birthday.month, sp->birthday.day,  
           sp->birthday.year);  
    printf(“score: %f\n”, sp->score);  
}
```

结构指针形参也是普通的**结构指针变量**，在函数内按照结构指针的一般规则使用即可。

- ◆ 结构指针做参数，函数调用时采用“**址传递**”，将结构体实参的地址复制给结构指针形参。

```
void ShowStudPtr(struct stud *sp)
```

```
{
```

```
.....
```

```
}
```

```
int main()
```

```
{
```

```
struct stud std={ "U20230001", "Li", 'm', {"Oct",31,2023}, 89.5};
```

U20230001	Li	M	Oct	31	2023	89.5
-----------	----	---	-----	----	------	------

```
ShowStudPtr (&std);
```

```
}
```

&std

4个字节

② 然后将“实参” std的地址复制给指针形参sp

取结构体“实参”的地址

调用前有“实参”，没有形参

① 在调用之时，首先为结构指针形参sp分配存储单元

3、结构数组作为函数的参数

- ◆ 函数的参数可以是**结构数组**，此时的参数传递采用“**址传递**”。

例：设有输出学生信息的函数**ShowStudAry**

```
void ShowStudAry(struct stud sa[ ], int n)
```

```
{  
    int i;  
    for(i=0;i<n;i++) {  
        printf("num: %s\n", sa[i].num);  
        printf("name: %s\n", sa[i]. name);  
        printf("sex: %c\n", sa[i]. sex);  
        printf("birthday: %s %d, %d\n",  
                sa[i]. birthday.month, sa[i]. birthday.day, sa[i]. birthday.year);  
        printf("score: %f\n", sa[i]. score);  
    }  
}
```

结构数组形参可以作为普通的**结构指针**，在函数内按照结构数组（结构指针）的一般规则使用即可。

◆ 结构数组参数 “址传递” 过程

```
void ShowStudAry(struct stud sa[ ], int n)
```

```
{
```

```
.....
```

```
}
```

```
int main()
```

```
{
```

```
struct stud stdary[3]={{"U20230001", "Li", 'm', {"Oct",31,2023}, 89.5},  
                        {"U20230002", "Zhang", 'f', {"June",12,2024}, 90.0},  
                        {"U20230003", "Yang", 'm', {"May",1,2023}, 75.3}  
                        };
```

```
ShowStudAry (stdary);
```

```
}
```

stdary

4个字节

② 然后将实参数组名的值（实参数组的地址）复制给数组形参名sa

调用前有实参数组，没有形参

用实参数组名做实参

① 在调用之时，首先为结构数组形参sa分配存储单元

◆ **结构数组名和结构指针作为函数的参数可以“混用”**

例：设有输出学生信息的函数**ShowStudAry2**

```
void ShowStudAry2(struct stud *sa, int n)  
{  
    int i;  
    for(i=0;i<n;i++) {  
        printf("num: %s\n", sa[i].num);  
        printf("name: %s\n", sa[i]. name);  
        printf("sex: %c\n", sa[i]. sex);  
        printf("birthday: %s %d, %d\n",  
            sa[i]. birthday.month, sa[i]. birthday.day, sa[i]. birthday.year);  
        printf("score: %f\n", sa[i]. score);  
    }  
}
```

此时的函数调用依然是“址传递”

```
void ShowStudAry2(struct stud *sa, int n)
```

```
{
```

```
.....
```

```
}
```

```
int main()
```

```
{
```

```
struct stud stdary[3]={{"U20230001", "Li", 'm', {"Oct",31,2023}, 89.5},  
                        {"U20230002", "Zhang", 'f', {"June",12,2024}, 90.0},  
                        {"U20230003", "Yang", 'm', {"May",1,2023}, 75.3}  
                        };
```

```
ShowStudAry2 (stdary);
```

```
}
```

stdary

4个字节

② 然后将实参数组名的值（实参数组的地址）复制给指针形参名sa

调用前有实参数组，没有形参

用实参数组名做实参

① 在调用之时，首先为结构指针形参sa分配存储单元

或:

```
void ShowStudAry2(struct stud *sa, int n)
```

```
{
```

```
.....
```

```
}
```

```
int main()
```

```
{
```

```
struct stud stdary[3]={{"U20230001", "Li", 'm', {"Oct",31,2023}, 89.5},  
                        {"U20230002", "Zhang", 'f', {"June",12,2024}, 90.0},  
                        {"U20230003", "Yang", 'm', {"May",1,2023}, 75.3}  
};
```

```
ShowStudAry2 (&stdary[0]);
```

```
}
```

&stdary[0]

4个字节

② 然后将“实参地址”复制给形参指针

调用前有实参数组，没有形参

用实参数组的第一个元素的地址做实参

① 在调用之时，首先为结构指针形参sa分配存储单元

或者:

例: 设有输出学生信息的函数**ShowStudAry3**

```
void ShowStudAry3(struct stud sa[ ], int n)
```

```
{
```

```
    int i;
```

```
    for(i=0;i<n;i++) {
```

```
        printf("num: %s\n", sa->num);
```

```
        printf("name: %s\n", sa->name);
```

```
        printf("sex: %c\n", sa[0].sex);
```

```
        printf("birthday: %s %d, %d\n",
```

```
                sa[0].birthday.month, sa->birthday.day,
```

```
                sa[0].birthday.year);
```

```
        printf("score: %f\n", sa->score);
```

```
        sa++;
```

```
    }
```

```
}
```

形参数组名本质上是一个指针变量，不是常规意义下的数组名，所以可以当作指针变量使用。

可改变形参数组名的值，从而使其“指向”不同的元素。

4、结构体作为函数的返回值

函数的返回值可以是**结构体**。此时，函数的类型要声明为**结构类型**，返回时直接返回一个**结构体的值**。

例：设有输入学生信息的函数**GetStud**。

```
struct stud GetStud( )
```

```
{
```

将函数的类型声明为结构类型

struct stud s; 如同声明普通局部变量一样声明一个局部结构变量

```
printf("input num:"); scanf("%s", s.num);  
printf("input name:"); scanf("%s", s.name); getchar();  
printf("input sex:"); scanf("%c", &s.sex);  
printf("input birthday month day year:");  
scanf("%s%hd%d", s.birthday.month, &s.birthday.day, &s.birthday.year);  
printf("input score:"); scanf("%f", &s.score);
```

return s; 最后如同返回普通变量的**值**一样返回结构变量的值。

```
}
```

进行常规操作

◆ 结构体作为返回值，直接返回一个结构体的“值”

```
struct stud GetStud()
```

```
{
```

```
    struct stud s;
```

③在GetStud函数中生成s，输入s的所有数据成员的值。

U20230001

Li

M

Oct

31

2023

89.5

```
    return s;
```

```
}
```

```
int main()
```

```
{
```

```
    struct stud st;
```

④ 返回时将s的值带回到主调函数中

① 在主调函数中声明一个结构变量st

U20230001

Li

M

Oct

31

2023

89.5

②调用GetStud

```
    st = GetStud();
```

调用前没值，调用后得到返回结果

⑤将GetStud的返回结果“赋给” st

```
}
```

5、结构指针作为函数的返回值

函数的返回值可以是结构指针。此时，函数的类型要声明为**结构指针类型**，返回时返回值应是一个“**结构类型 ***”的地址值。

例：设有输入学生信息的函数**GetStud2**。

```
struct stud * GetStud2( )
```

```
{
```

将函数的类型声明为**结构类型 ***

```
    static struct stud s;    声明静态局部结构变量，长生存期，“有地址”
```

进
行
常
规
操
作

```
    printf("input num:"); scanf("%s", s.num);  
    printf("input name:"); scanf("%s", s.name); getchar();  
    printf("input sex:"); scanf("%c", &s.sex);  
    printf("input birthday month day year:");  
    scanf("%s%hd%d", s.birthday.month, &s.birthday.day, &s.birthday.year);  
    printf("input score:"); scanf("%f", &s.score);
```

```
    return &s;    将s的地址返回
```

```
}
```

◆ 结构指针作为返回值，要返回一个 **结构类型 *** 的地址值

```
struct stud * GetStud2()
```

```
{
```

```
    static struct stud s;
```

③在GetStud函数中输入s各个数据项的数据

.....

U20230001

Li

M

Oct

31

2023

89.5

```
    return &s;
```

0x00ffa080

```
}
```

④ 返回s的地址

```
int main()
```

```
{
```

```
    struct stud * sp;
```

① 在主调函数中声明一个结构指针sp

②调用GetStud2

0x00ffa080

```
    sp = GetStud2();
```

调用前没值，调用后得到返回结果

```
    ShowStudPtr(sp);
```

调用ShowStudPtr显示sp所指对象的内容

```
}
```

D:\CTest\555.exe

```
input num:U20230001
input name:Li
input sex:m
input birthday month day year:May 31 2023
input score:89
num: U20230001
name: Li
sex: m
birthday: May 31, 2023
score: 89.000000
```

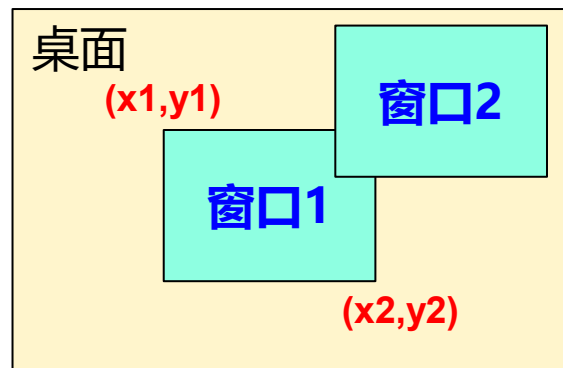
一个应用举例：

设图形操作系统（如Windows）的桌面区域打开有多个**窗口**。每个窗口是一个矩形区域，用其左上角和右下角的一对坐标描述其位置。每个窗口有一个编号。同时可以打开多个窗口，每个窗口都有自己的位置记录。

现在模拟鼠标**点击窗口**、**移动窗口**、**关闭窗口**的操作。

(1) 定义窗口结构类型描述窗口在桌面上的编号和坐标信息

```
typedef struct _win {  
    int num;    /* 窗口编号 */  
    int x1,y1;  /* 左上角坐标 */  
    int x2,y2;  /* 右下角坐标 */  
} WIN;
```



(2) 定义窗口结构类型数组

```
WIN wins[10];    /* 设系统中最多同时可以打开10个窗口 */  
  
int n=0;          /* n记录系统中当前已打开的窗口数，开始的时候等于0 */
```

要求: **wins中的窗口按照叠加的层次排列**: 最上层窗口在下标0处, 最下层窗口在下标n-1处; 一个窗口被点击了, 要将它挪到数组下标0处, 表示把它显示到最上层。

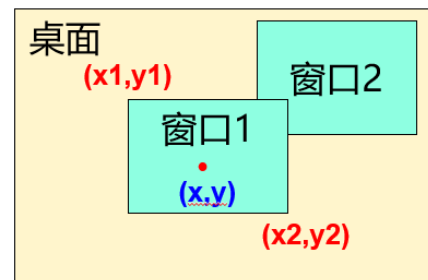
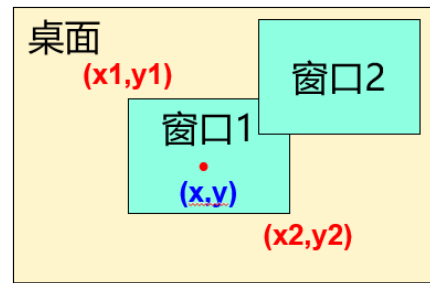
(3) 定义一组窗口操作的函数

int ClickWin(WIN w[], int n, int x, int y);

函数功能： 参数(x, y)是鼠标的点击坐标，函数判断是否点中了w中的一个窗口。若点中了窗口，则将其挪到数组下标为0的位置并返回该窗口的num；如果没有点中任何窗口，返回-1。

判断的顺序： 由于窗口有重叠，所以按照叠加层次从上到下（即w中下标从0到n-1）依次判断(x,y)是落在哪个窗口内。

挪到顶层： 设点中的窗口下标是i，若i==0，可以不做任何操作；否则，将前面下标为0~i-1的窗口在w数组中后移一个位置，然后把当前被点中的窗口（元素）放在w[0]处。



```
int ClickWin( WIN w[ ], int n, int x, int y)
```

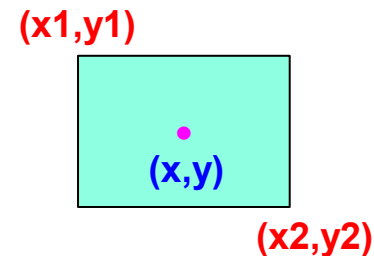
```
{  
    w是结构数组, n是w中的元素数
```

```
    int i, j;
```

```
    WIN tw;
```

```
    for(i=0;i<n;i++) {
```

判断(x,y)是否落在下标
为i的窗口矩形区域内



```
        if(w[i].x1<=x&& w[i].y1<=x && x<=w[i].x2&&y<=w[i].y2) {
```

```
            tw=w[i];
```

```
            for(j=i; j>0;j--)
```

```
                w[j]=w[j-1];
```

```
            w[0]=w[i];
```

如果找到了第一个被点
中的窗口，则将其移到
数组的开始位置。

```
            return w[0].num; /*返回点中的窗口号，被点中的窗口已经挪到了下标0处*/
```

```
        }
```

```
    }
```

```
    return -1;
```

```
}
```

```
int CloseWin(WIN *w, int n, int num)
```

```
{
```

w是结构指针，n是w所指地址处开始的元素数

```
int i;
```

```
for(i=0; i<n; i++) /*查找窗口号是num的窗口*/
```

```
if(w[i].num == num) {
```

← **对w的引用依然可以用数组形式**

```
for(j=i; j<n-1; j++)
```

```
w[j]=w[j+1];
```

```
break;
```

```
}
```

```
return -1;
```

```
}
```

功能：在w中找到窗口号是num的元素，若干找到则将其在w数组中删除，表示“关闭”了窗口。

如果找到了窗口号是num的窗口，则将w中该窗口后面的所有元素都依次前移一个位置，覆盖掉该窗口，表示将其在w中删除了


```
int MovWin(WIN w[ ], int n, int num, int deltax, int deltax)
```

w是结构数组，n是w中的元素数

```
{
```

```
int i;
```

```
for(i=0; i<n; i++, w++)
```

```
if(w->num == num) {
```

```
w->x1 += deltax;
```

```
w->y1 += deltax;
```

```
w->x2 += deltax;
```

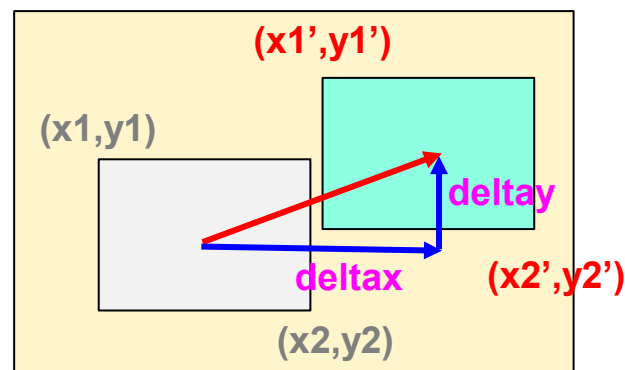
```
w->y2 += deltax;
```

```
break;
```

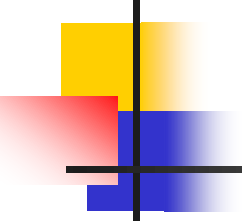
```
}
```

```
}
```

功能：首先在w中找到窗口号是num的元素，然后将其在x方向平移deltax，y方向平移deltay，表示窗口在桌面中移动了(deltax, deltax)距离。



结构数组形式的形参本质是指针，
可以直接当作指针变量使用。



■ 2024.12.5 补课，讲完结构的基本部分

9.8 链表

问题的提出:

一方面，数组可以用下标索引元素，位置关系很明确，使用很方便。但另一方面C语言规定，数组一经创建，大小就固定了，之后只能用，而不能改变其大小，这种 **“只能用而不能改变大小”的数据结构**称为 **“静态” 数据结构**。

静态数据结构最多只能存指定的“最大数量”的数据，超出了就会溢出。而如果应用环境数据量的差异很大，使用静态数据结构时，为了照顾到**最坏情况**，只能按照 **“最多”** 设置大小，这势必造成数据量不是那么大的时候的**空间浪费**。**有没有办法能够根据实际数据量的大小，需要多少就申请多少数据单元来用呢？**

解决方案（动态数据结构）：

(1) **动态数组**：C99后提供了动态数组机制，可以用，但一经申请也不能改变大小，且属于内部实现机制，这里不讨论。

(2) 用 **malloc** 动态申请一块可以放多个数据的连续字节空间：

`malloc(n*sizeof(T))`

可行，当作数组使用。但**要求分配的空间必须是连续的**。如果 **`n*sizeof(T)`** 很大，内存中就未必有这么大的一块连续空间能分配出来。而如果没有这么大的连续空间，**malloc** 就因分配不到空间而**失败**！

分析：事实上，单个数据元素“并不大”，只是 n 个元素需要的总空间会很大。内存即使没有一大块的连续空间，但通常会有很多小块、但不一定连续的空间，总量不少，甚至比 **`n*sizeof(T)`** 大得多。

能不能利用“小空间”实现“大需求”呢？

(3) **链表**：就是可以实现上述“**大需求**”的一种数据结构。

链表不要求物理存储空间连续，“**一小块**”空间只要能放下一个元素即可，然后**申请很多“小块”空间保存所有的元素**，再通过“**指针**”把所有元素“**关联**”起来而形成一个很大的数据集。

并且，不要求一下子把所有元素都申请出来，而是根据需要，用的时候“**用多少就申请多少**”，克服了静态数据结构存在的数据数量受限制的问题。

—— 以上根据实际情况“用多少就申请多少”的数据结构称为**动态数据结构**。

链表就是一种动态数据结构。

9.8.1 自引用结构类型

1、自引用结构类型的定义

如果一个结构类型中包含有一个或多个是**自身结构类型指针**的数据成员，则该结构称为**自引用结构类型**。

例， `struct lnode{`

`int data;` ← 数据域

`struct lnode * next;` ← 指针域

`};`

`next` 成员是 `struct node *` 类型的指针

通常把除指针以外的其他数据项统称为**数据域**，把指针部分统称为**指针域**

自引用结构类型的结构变量称为**自引用结构变量**，或**自引用结构体**，如： `struct lnode node;`

理解自引用结构类型：

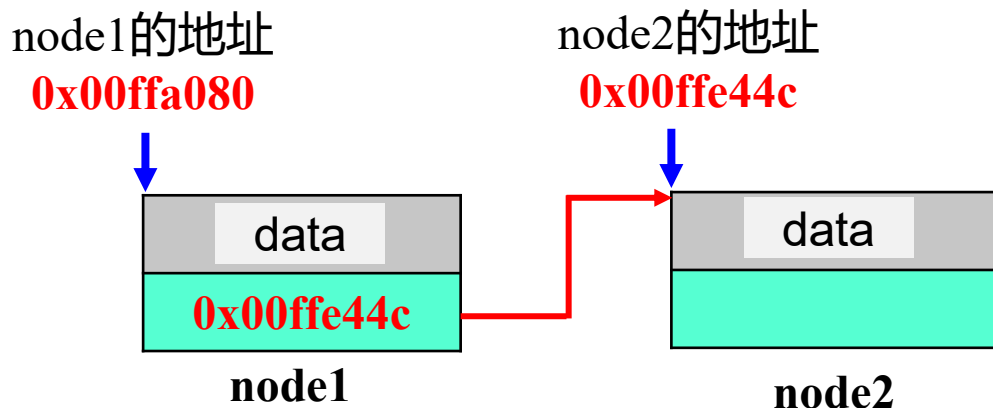
设，声明有如下自引用结构类型**struct lnode**，及该类型的两个结构变量**node1**和**node2**：

```
struct lnode{  
    int data;  
    struct lnode * next;  
} node1, node2;
```

问：data 域可以存整数，next 域存什么数据呢？

答：next 域可以存 struct lnode * 类型的地址数据。

若有：**node1.next = &node2**，则 node1 和 node2 可以形成如下图所示的“关联”关系：



```
struct lnode{  
    int data;  
    struct lnode * next;  
} node1, node2;
```

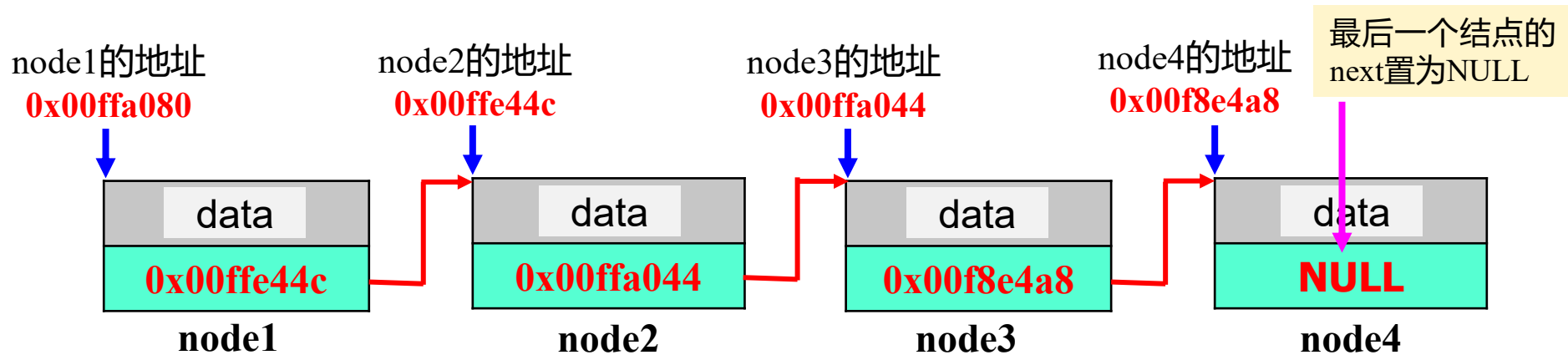
即 **node1** 的指针 **next** 指向了 **node2**，这样 **node1** 和 **node2** 通过 **node1** 的指针成员 **next** “串在了一起”，而形成一条链——这就是**链表**。

链表：就是**自引用结构变量**通过**自引用指针** “**串在一起**”而形成的一种**链式结构**。

如果再有 `struct lnode node3;` 接在 `node2` 的后面：

`node2.next=&node3;`

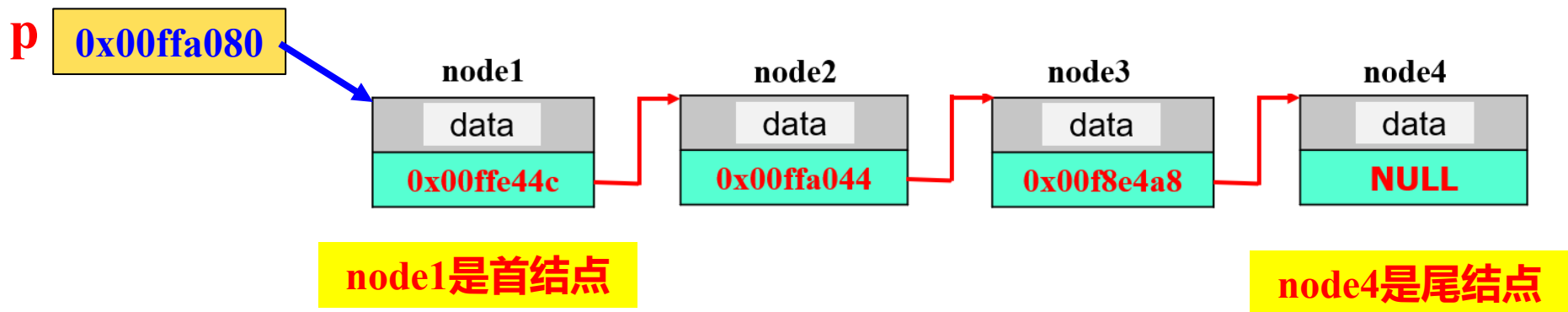
以及 `node4`、`node5`、... **继续链在后面**就会形成更长的链表：



可见，**链表**就是一些**自引用结构体**通过内部的**自引用指针**关
联起来的**一条链**：第一个结点(next指针)指向第二个结点，第二个结
点(next指针)指向第三个结点，....，**最后一个结点的 next 通常等于**
NULL (置空)，**作为标记**，指示后面不再有其它结点，链表结束。

通常，把链表里的结构体（结构变量）称为**结点**。并把第一个结点称为**首结点**，最后一个结点称为**尾结点**。

再设一个**结构指针变量** `struct lnode *p;`，令 `p=&node1`，则 `p` 指向链表的首结点，此时简称**结构指针 `p` 指向了链表**。

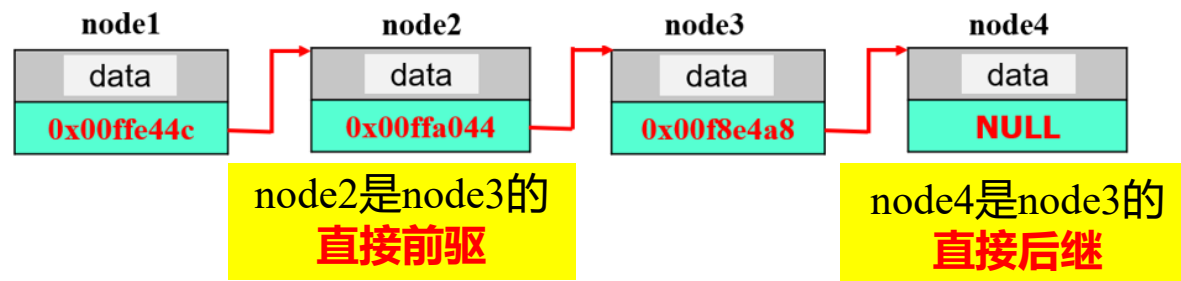


注： `p` 是一个单独的指针变量，4字节，里面存了 `node1` 的地址而指向 `node1`。

当一个结点 `A` 的 `next` 存有另一个结点 `B` 的地址时，称**结点 `A` 指向了结点 `B`**。如 `node1` 指向 `node2`。

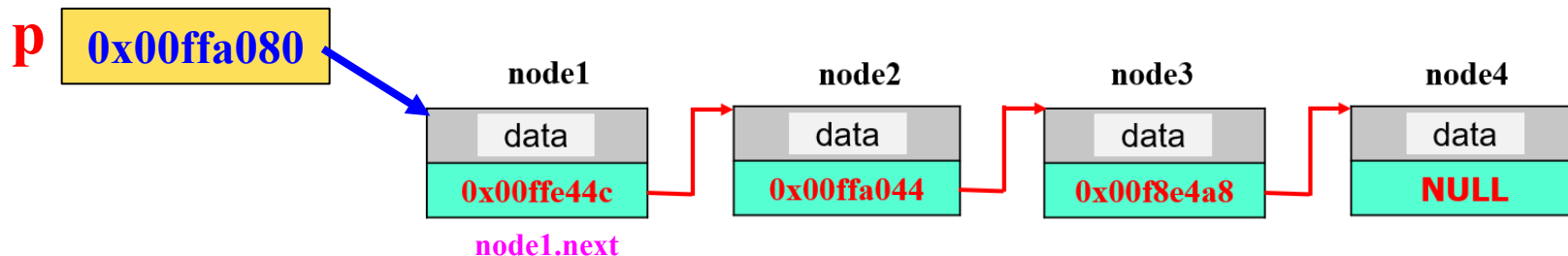
几个概念：

- ◆ **前驱**：链表中，一个结点**前面**的所有结点都是该结点的**前驱结点**，其中紧挨着的前面一个结点称为它的**直接前驱结点**；
- ◆ **后继**：链表中，一个结点**后面**的所有结点都是该结点的**后继结点**，其中紧挨着的后面一个结点称为它的**直接后继结点**。



- ◆ **单向链表**：如果结点的指针域只**包含一个自引用指针**，指向其直接后继，这样的链表称为**单向链表**，简称**单链表**。
单链表中每个结点的指针指向其直接后继，最后一个结点的指针为空（NULL）。

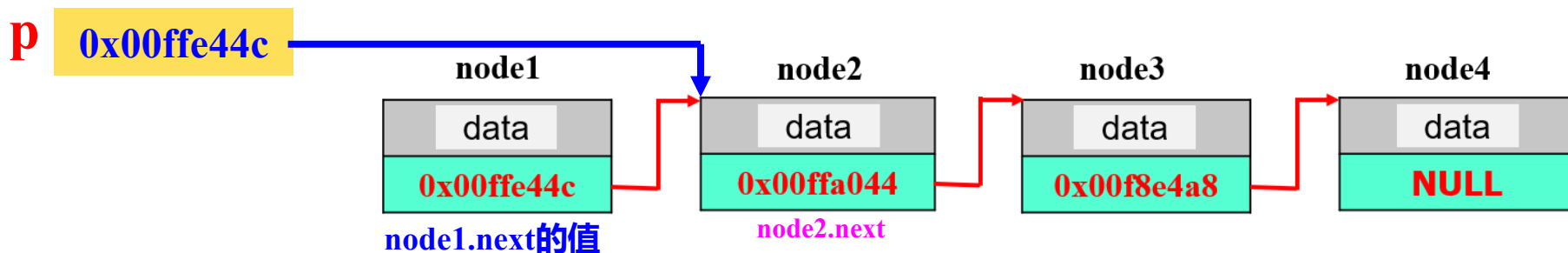
2、扫描链表：设 **p** 是指向链表的指针 `struct lsnod *p=&node1;`

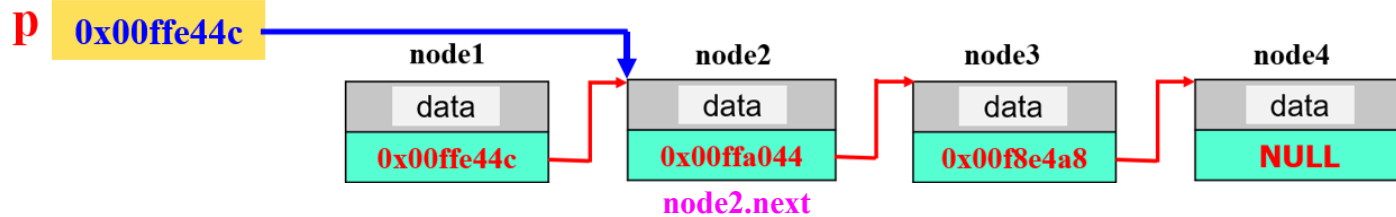


◆ 可通过 **p** 引用当前所指结点的成员：**p->data**、**p->next**

思考：执行 **p = p->next** 是什么效果？

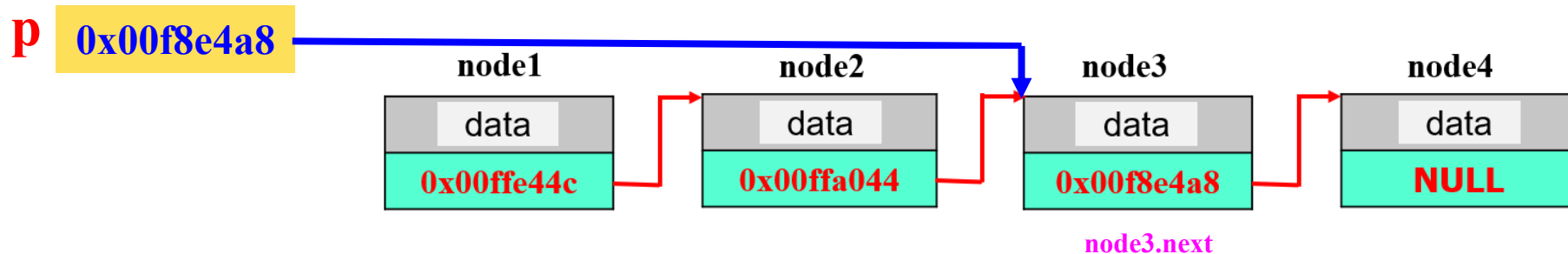
p=p->next：就是把 **node1.next** 的值赋给 **p**，从而使得 **p** 的值是下一个结点 **node2** 的地址，进而使得 **p** 指向 **node2**。



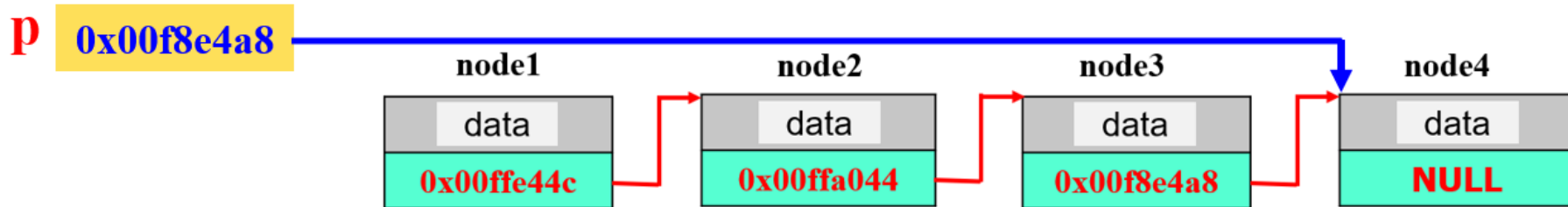


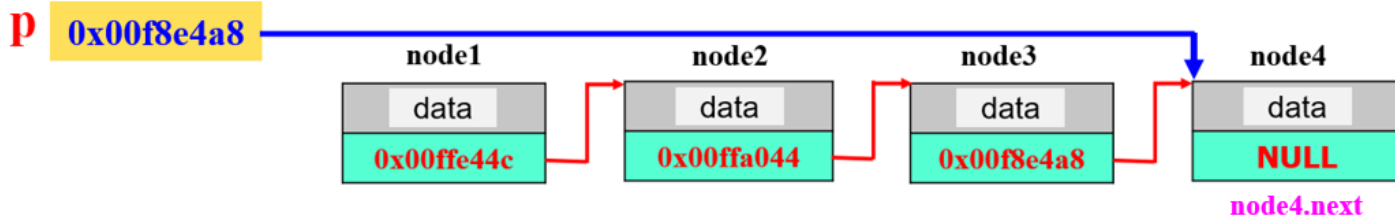
- 此时，如果再执行 **p = p->next** 又是什么结果？

使 p 指向 node3



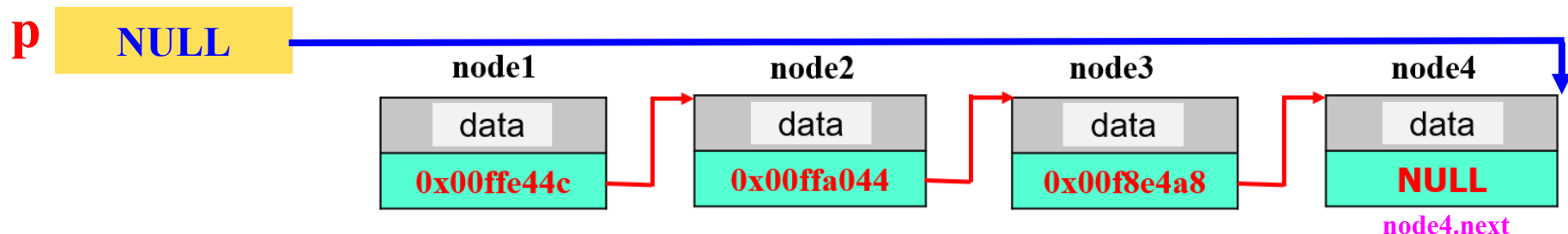
- 如果再执行 **p = p->next** 就可以使 p 指向 **node4**。





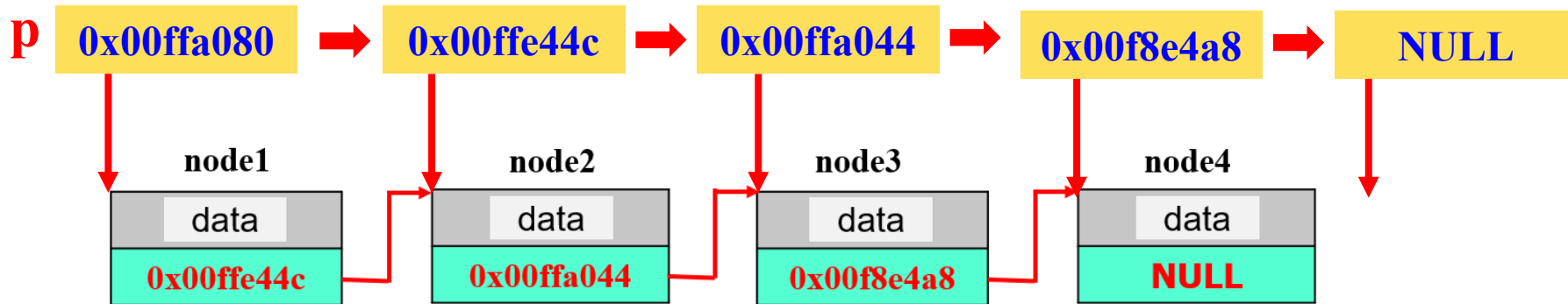
- ◆ 在 **p** 指向 **node4** 的时候，如果再执行 **p = p->next**，**p** 的值什么？

p == NULL （即 **p** 等于空）



上述过程，实际上就是用指针变量 **p** 扫描一个链表的过程：

- ◆ 这样的指针变量被称为“**扫描指针**”或“**遍历指针**”。
- ◆ 扫描链表通常从首结点开始，扫描指针沿结点指针域的指向**顺序遍历**每个结点，直至扫描指针为 **NULL** 时结束。



扫描链表的过程:

通常从 **p** 指向第一个结点开始, 通过 **p=p->next** 操作, 遍历链表的所有或部分结点, 直到 **p==NULL** 或 “找到某个想找的结点” 为止。

程序基本框架:

```
struct lsnod *p = &node1;
while(p!=NULL) {
    .....
    if(p->data==5)
        break;
    else p=p->next;
}
if(p==NULL)
    printf("失败!");
else { ..... } /*成功*/
```

9.8.2 创建链表

静态创建：前面创建链表的过程是**事先声明好几个结构变量**，然后用这些结构变量做结点构建链表，所以**结点的数量是确定的**，**不能动态增加**（释放这些结构变量也不行）。

如何实现“**需要多少结点就建立多少结点，而不用的时候就释放掉结点、甚至整个链表**”呢？—— **动态创建链表**。

动态创建链表的一般过程：

首先，声明一个称为**头指针**的指针变量 **head**，其**基类型和链表的结点类型相同**。

- ◆ 在链表的生存期内，**链表都将由head指针指示**。
- ◆ 开始的时候链表中没有一个结点，故置 **head=NULL**；而其后 **head 将指向链表的第一个结点**。

然后，从第一个结点起，通过**动态申请 (malloc)** 生成结点，并将其 **“插入”** 到链表中；

最后，不用的结点或链表使用完后，要**逐一释放 (free) 结点**，**将其存储空间还给操作系统**。

1、头指针

动态创建的链表一般由一个**指向第一个结点的指针变量** **head** “牵引”，**head** 称为**头指针**。

头指针就是一个**基类型为结点类型**的结构指针变量：

- ◆ 设结构类型为 **struct T**，则头指针可声明为：

struct T * head;

指针变量的名字可以任意起，
只要是合法标识符即可

- ◆ 开始的时候链表中没有结点，此时置 **head==NULL**，表示链表是 “**空**” 的状态。

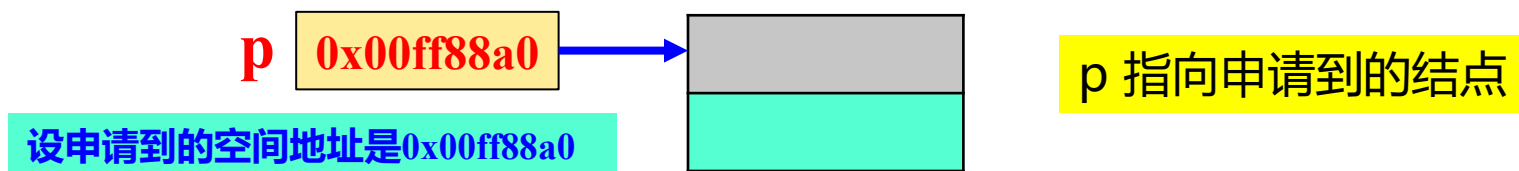
2、动态申请结点

动态链表中的结点一般是“**使用时才生成**”。生成结点的基本方式是**动态申请**。设结构类型为**struct T**，操作一般如下：

(1) 先声明一个结构指针变量：**struct T * p;**

(2) 然后**调用 malloc**（或 **calloc**等）函数申请**一个大小为 一个结点大小**（**sizeof(struct T)**）的空间，并将返回地址类型强制转化为 **struct T *** 类型，然后赋给结构指针变量 **p**，如：

p = (struct T *)malloc(sizeof(struct T));



理解动态申请的结点的存储结构：

(1) **从系统的角度**：`malloc`申请到的若干字节空间**仅是字节序列**，
没有内部逻辑结构；

如：`struct stud` {

`char num[12];`

`char name[9];`

`char sex;`

`struct date birthday;`

`float score;`

}; ***p;**

`p = (struct stud *)malloc(sizeof(struct stud));`

`struct date` {

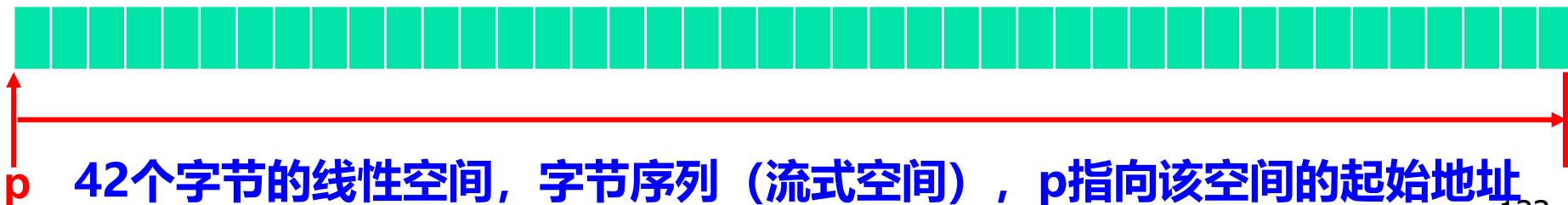
`char month[10];`

`short day;`

`int year;`

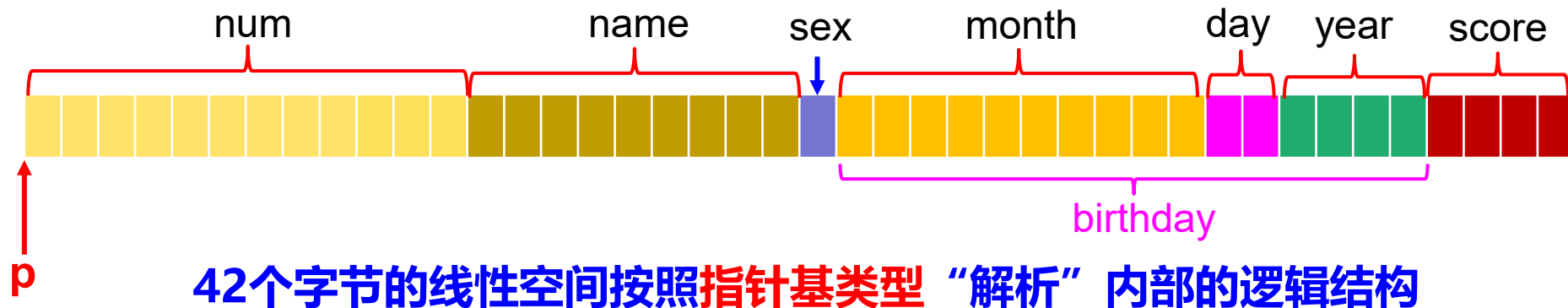
};

`sizeof(struct stud)==42`



(2) 从结构指针的角度

当把 `malloc` 申请到的字节空间（的地址）赋给一个结构指针时，访问该结构指针**将按照指针的基类型来“逻辑解释”指针所指的字节空间**。从而就按照指针的基类型从逻辑上将字节空间分成若干“段”，每段对应结构类型的一个成员，之后就如同对待普通结构变量空间一样，引用结构体及其数据成员了。



引用结构体：**`*p`**：

引用数据成员：**`p->num`** 或 **`(*p).num`**

引用嵌套的结构成员的成员：**`p->birthday.month`** 或 **`(*p). birthday.month`**

3、为空链表插入第一个结点

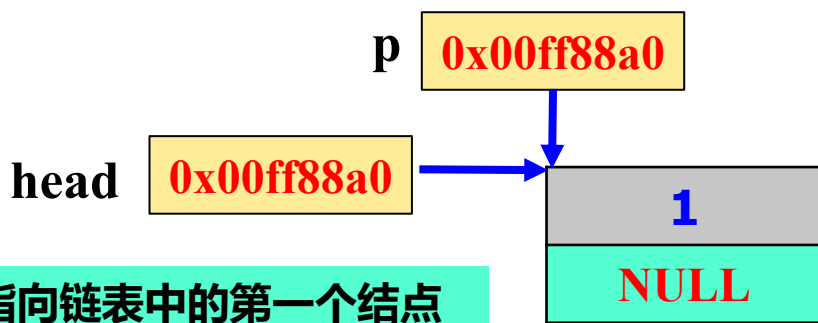
把一个结点“链入”链表的操作称为“**插入**”。

开始的时候，链表为“**空**”，**head==NULL**。

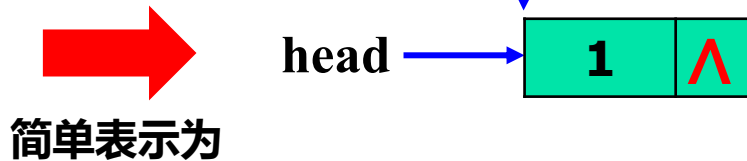
在空链表中插入第一个结点，仅需执行以下操作（设新结点已经 **malloc** 出来，并由指针 **p** 指示，**p->data=1**）：

head=p;

p->next=NULL; 或 **head->next=NULL;**



head指向链表中的第一个结点



简单表示为

分析以下函数的功能

```
struct lnode * GetNode( )
```

```
{
```

```
    struct lnode *s;
```

```
    s = (struct lnode *)malloc (sizeof(struct lnode));
```

```
    return s;
```

```
}
```



调用 **GetNode** 函数申请到一个结点，返回申请到的结点的指针，然后在main函数中使用。



```
int main ( )
```

```
{
```

```
    struct lnode * p, *head;
```

```
    p = GetNode();
```

```
    scanf("%d", &p->data);
```

```
    p->next=NULL;
```

```
    head=p;
```

```
    .....
```

```
    return 0;
```

```
}
```

另外一种实现：

```
GetNode2(struct lnode ** h)
```

```
{  
    *h = (struct stud *)malloc (sizeof(struct lnode));  
}
```



调用**GetNode2**函数申请到一个结点，**将地址赋给间接的二级指针**带回main函数中使用。



```
int main ( )
```

```
{  
    struct lnode * head;  
    GetNode2(&head);  
    scanf("%d", &head->data);  
    head->next=NULL;  
    .....  
    return 0;  
}
```


4、将其它结点插入到非空链表中

在只有一个结点的基础上把其它结点插入链表，有三种典型情况：插在链表的**头部**、**尾部**或的**中间**。

(1) 将新结点插在链表的头部，称为“**头插法**”。

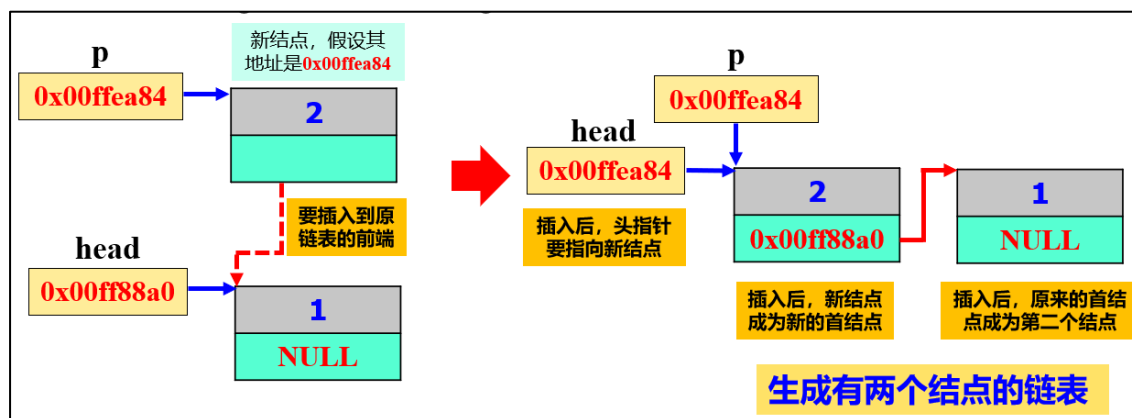
设新结点由指针 **p** 指示（设 **p** 已经生成，且 **p->data=2**）。

头插法将新结点插在链表的**第一个结点的前面**，成为新的首结点，插入后再修改 **head** 使之指向新结点。执行以下操作：

◆ **p->next=head;**

◆ **head=p;**

两句的顺序不能颠倒



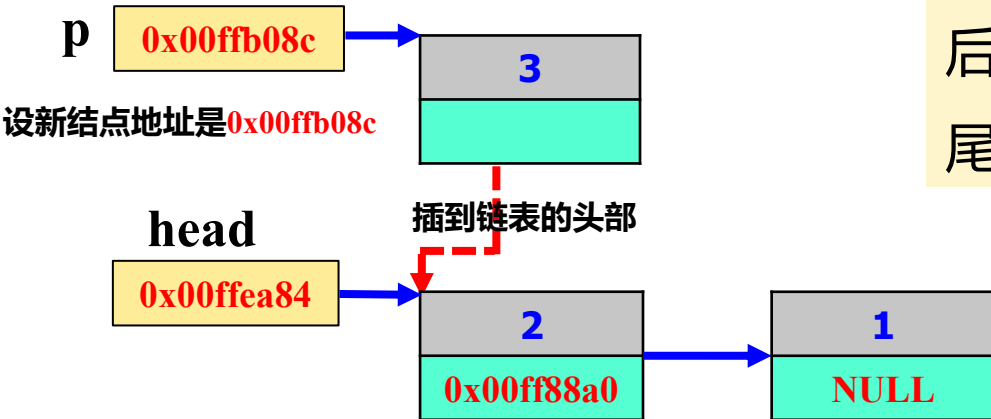
如果再 “**头插**” 第三个结点，同样的操作：

(I) 申请结点：`p = (struct lnode *)malloc(sizeof(struct lnode));`

(II) 执行操作：`p->next=head;` 设数据域：`p->data=3;`

`head=p;`

“**头插法**” 的特点：链表中结点从左向右的排列顺序与结点插入链表的先后顺序相反，最先插入的结点排在表尾，最新插入的结点排在表首。



(2) 将新结点插在链表的尾部，称为“**尾插法**”。

设新结点由指针 **p** 指示（设 **p** 已经生成，且 **p->data=2**）。

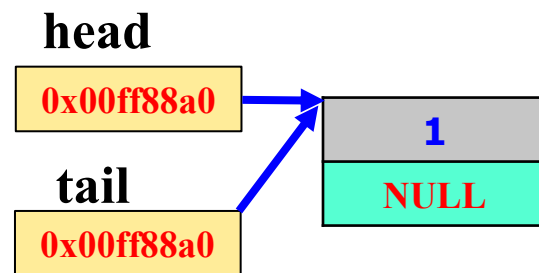
尾插法将新结点插在链表**最后一个结点的后面**，成为新的尾结点，并令其 **next** 为 **NULL**，表示链表新的结束。

尾插入时，一般首先设置一个称为**尾指针**的结构指针变量 **tail**，令其指向当前链表的最后一个结点。

可以用**tail**扫描链表找到尾结点

```
struct lnode *tail=head;  
while(tail->next!=NULL)  
{  
    tail=tail->next;  
}
```

最后一个结点的 **next==NULL**



在链表只有一个结点的时候，**tail** 也指向第一个结点

然后执行以下操作

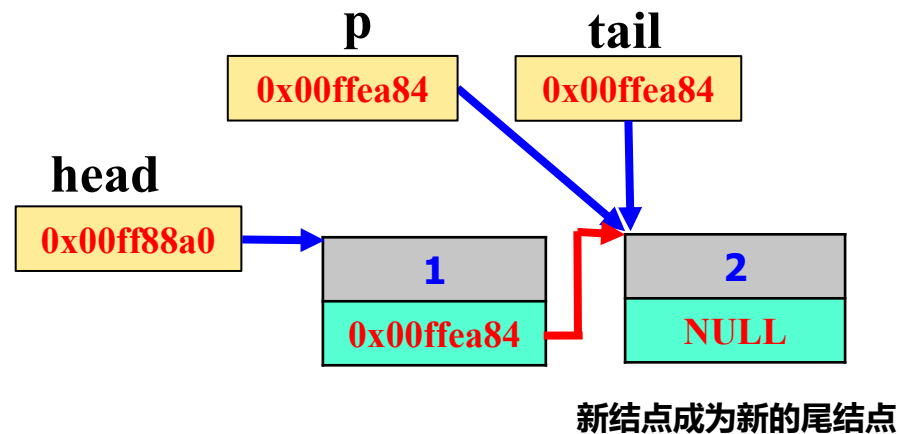
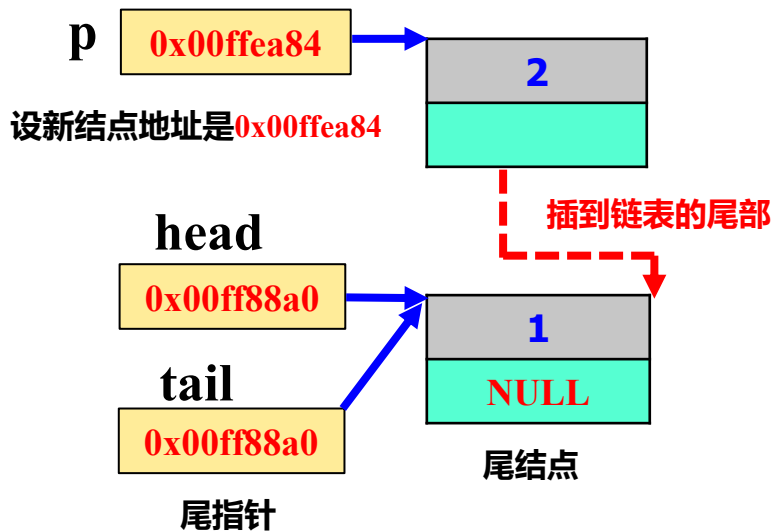
◆ **tail->next=p;**

◆ **tail=tail->next;**

◆ **tail->next=NULL;**

令 tail 指向新结点，为下一次插入做好准备

新尾结点的 next 域置空



如果再“**尾插**”第三个结点：

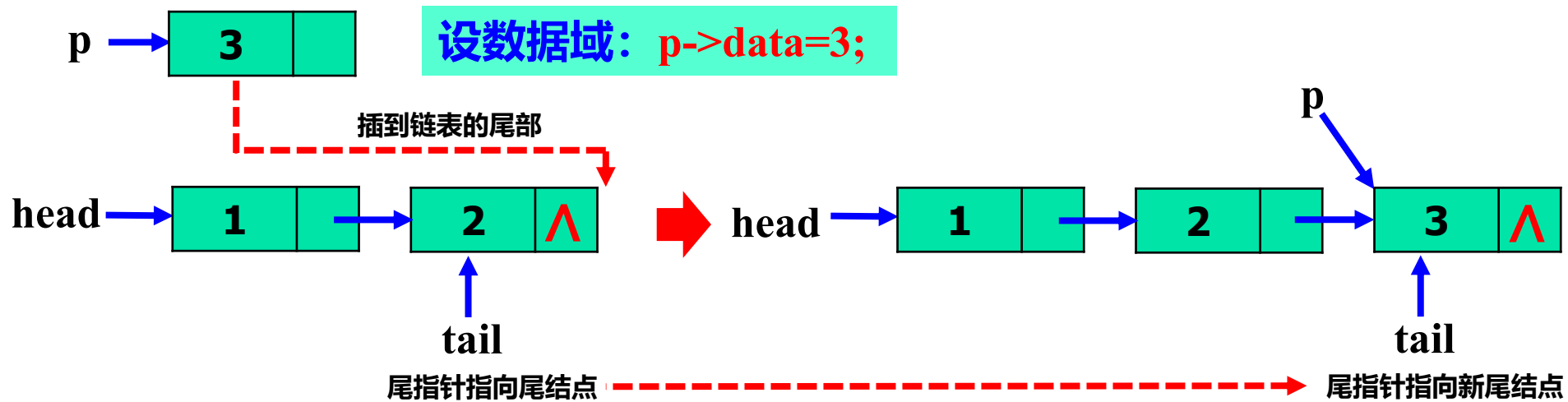
- (I) 申请新结点：**p = (struct lsnod *) malloc (sizeof (struct lsnod));**
- (II) 在第二次尾插后的基础上 (**tail已指向尾结点**，不用重新扫描链表找尾结点)，直接执行以下操作即可：

tail->next=p;

tail=tail->next;

tail->next=NULL;

“尾插法”的特点：链表中结点从左向右的排列顺序与结点插入链表的先后顺序相同，最先插入的结点排在表首，最新插入的结点排在表尾。



例：创建链表的函数 CreatList()：输入一批整数，以 0 结束，用尾插法将它们建成链表，返回表头指针。

```
struct lnode * CreateList()
```

```
{
```

```
    struct lnode * head, *tail, *p;
```

```
    int x;
```

```
    head=tail=NULL;
```

```
    scanf("%d", &x);
```

```
    while(x){
```

```
        p=(struct lnode *)malloc(sizeof(struct lnode));
```

```
        p->data=x;
```

```
        if(head==NULL) head=p; /*第一个结点 “入链” */
```

```
            else tail->next=p; /*其后的结点 “尾插入链” */
```

```
        tail = p; /* 新结点是新的尾结点，令tail指向新尾结点*/
```

```
        scanf("%d", &x);
```

```
    }
```

```
    if(tail!=NULL) tail->next=NULL;
```

```
    return head;
```

```
}
```

注：由于是“连续”入链，所以每次插入一个新结点后没有马上令 tail->next=NULL；而是在所有结点都输入后，最后令 tail->next=NULL；

tail==NULL表示一个结点都没有建立，最后还是个“空”链表

```
#include <stdio.h>
```

```
#include <malloc.h>
```

```
struct lnode * CreateList( );
```

```
int main( )
```

```
{  
    struct lnode * hd, *p;
```

```
    hd = CreateList( );
```

```
    p = hd;
```

```
    while(p) {
```

```
        printf("%d ", p->data);
```

```
        p = p->next;
```

```
    }
```

```
}
```

**CreateList返回所建链表的
“头指针”，赋给hd。**

如输入：

2

4

5

1

3

0

则输出：

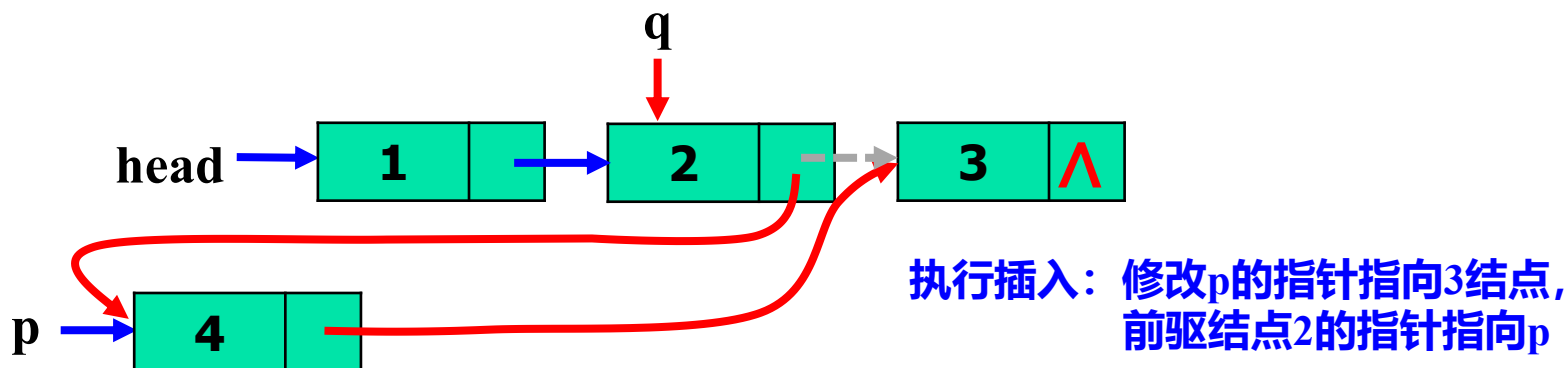
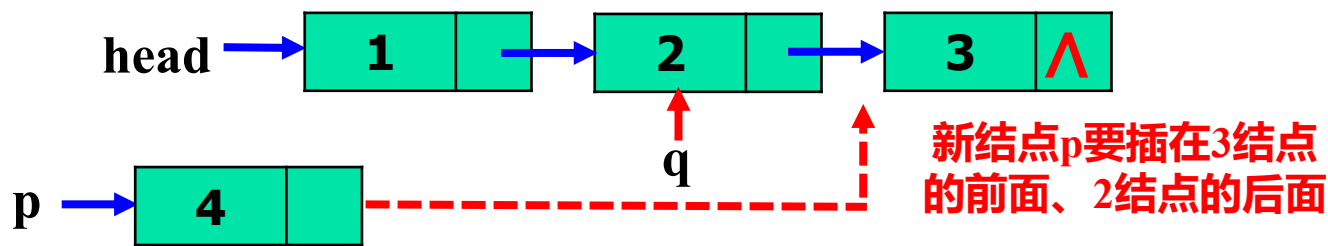
2 4 5 1 3

无序

(3) 将新结点插在链表的中间。

此时，首先根据插入要求，**找到插入点的（直接）前驱结点**，**然后将新结点插在前驱的后面**。

如在下面的三个结点的链表中，将新结点 p 插在 data 域等于 3 的结点的前面。**q 指针**指向了 3 的结点的前驱（即data域等于2的结点）：



◆ 查找一个结点的前驱：

如，将新结点插在 data 域等于 3 的结点前，查找插入位置（即结点 3 的前驱）：

```
struct lsnod *q = head;
```

```
if(head && head->data!=3) {
```

```
    while(q->next && q->next->data!=3)
```

```
    {
```

```
        q=q->next;
```

```
    }
```

```
}
```

```
else .....
```

要考虑链表为空或首结点的 data 域就等于 3 的情况

判断 `p->next->data!=3` 是为了找 3 结点的前驱，因为 next 的指向关系，新结点要插在前驱的后面。同时还要考虑链表中没有 data 域等于 3 的结点的情况

设新结点由 p 指针指示，已经生成，并且数据域 $p \rightarrow data = 4$ 。

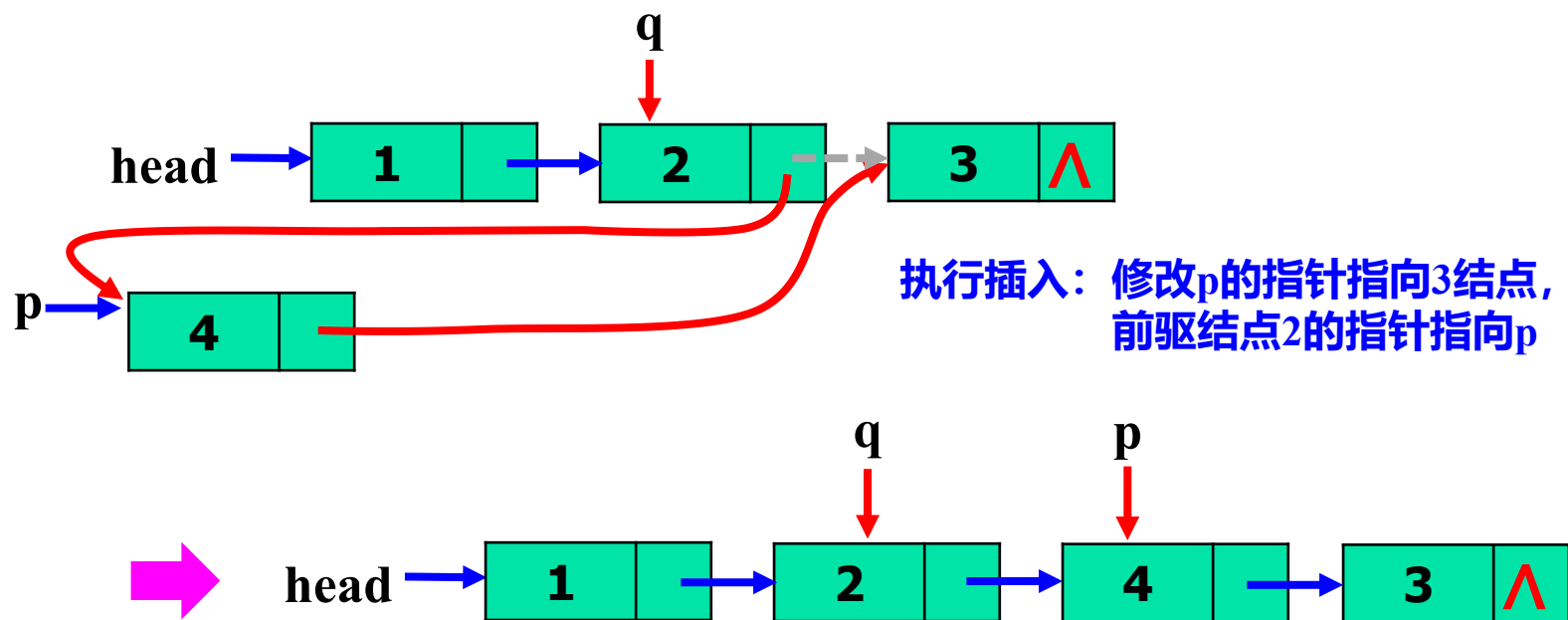
执行以下操作即可将 p 插在结点 2 的后面、结点 3 的前面，

这里 q 指针指向了结点 3 的前驱（结点 2）：

◆ $p \rightarrow next = q \rightarrow next;$

两句的顺序不能颠倒

◆ $q \rightarrow next = p;$



注：特殊情况可以做以下讨论

- ◆ **如果一开始链表为空**：可以不插入，或将新结点作为第一个结点插入链表；
- ◆ **如果首结点的 data 域就等于 3**：把新结点插入到当前首结点前面而成为新的首结点（相当于**头插**）；
- ◆ **如果链表中没有 data 域等于 3 的结点**：不插入，或者将新结点插入到尾结点的后面而成为新的尾结点（相当于**尾插**）。

例：下述函数在head指示的链表中**按元素值的大小**插入一个值为x的结点，
即插入的位置，x 要比前面结点的值大，比后面结点的值小。

```
void InsertNode(struct lsnod ** head, int x)
```

```
{
```

由于可能会改变头指针的指向，所以这里 head 是一个**二级指针**

```
    struct lsnod * q;
```

```
    q=(struct lsnod *)malloc(sizeof(struct lsnod)); q->data=x; 生成值为x的结点
```

```
    if(*head==NULL) { *head=q; q->next=NULL;}
```

如果为空表，作为第一个结点插入

```
    else if(x<(*head)->data) { q->next=*head; *head = q; }
```

x**比第一个结点还小**，新结点插在第一个结点前面而成为新的首结点。

```
    else {
```

```
        if(!(*head)->next) { (*head)->next = q; q->next=NULL;}
```

x比第一个结点的值大，但当前只有一个结点，x插在最后，成为尾结点。

```
        else {
```

```
            p=*head;
```

```
            while(p->next && p->next->data<x) p=p->next; 寻找插入位置
```

```
            q->next=p->next;
```

```
            p->next = q;
```

执行插入

```
        }
```

```
    }
```

```

#include <stdio.h>
#include <malloc.h>
void InsertNode(struct lsnod ** head, int x)
int main( )
{
    struct lsnod * head=NULL, *p;
    int x;
    scanf("%d", &x);
    while(x) {
        InsertNode(&head, x);
        scanf("%d", &x);
    }
    p=head;
    while(p) {
        printf("%d ", p->data);
        p = p->next;
    }
}

```

用&head做实参

输出链表中各结点的值

以上本质上是插入排序算法的实现

如输入：

2

4

5

1

3

0

则输出：

1 2 3 4 5

有序

用递归方式建立链表 (自学)

```
struct s_list *createlistR()
```

```
{
```

```
    int x;
```

```
    struct lstnode * lothead=NULL;
```

```
    scanf("%d", &x);
```

```
    if(x) { /* 对非0输入建立结点, 0表示结束 */
```

```
        lothead=(struct lstnode *)malloc(sizeof(struct lstnode ));
```

```
        lothead->data=x;
```

```
        lothead->next = createlistR(); /*递归建立下一结点并 “接在” 在lothead后面*/
```

```
    }
```

```
    return lothead; /*返回头指针*/
```

```
}
```

◆ 关于链表性质的说明

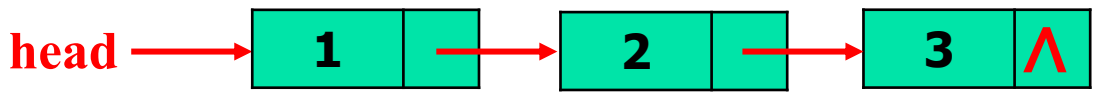
(1) 链表中，**结点间的存储空间不连续**，所以无法**有规则地计算**每个结点的“偏移地址”，也就不能象数组一样用 **head[下标]** 的方式访问其中的结点，而只能用**扫描指针**，从第一个结点开始、沿结点的 **next** 指针**顺序往后扫描**来找到想要的结点或遍历整个链表，所以链表只能**顺序访问**。

注：**数组可以随机访问**。即，访问元素时，不用每次都从第一个元素开始，而是直接用“**数组名[下标]**”就可直接访问到一个元素。

(2) 静态链表用完后不用释放空间，由系统自动回收。但**动态链表**由于结点都是 **malloc** 出来的，所以用完后一定要**释放**结点空间，即**删除结点**。

9.8.4 删除结点

设经过一系列操作，生成了如下三结点的链表：



删除链表中的结点分四种情况：

情况1：删除首结点

(I) 声明一个指针变量 p：

`struct lsnod * p;`

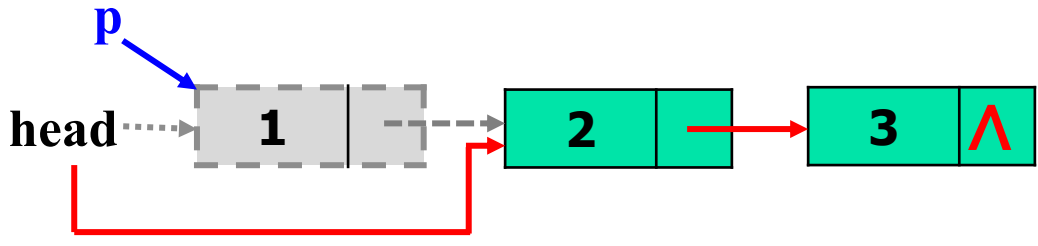
(II) 删除首结点的操作：

`p=head;` p 指向首结点

`head=head->next;`

`free(p);` 释放p所指结点

注：删除首结点不能：
`free(head);`
`head=head->next;` ❌



head指向第二个结点，当前首结点被删除后第二个结点成为新的首结点。

情况2：删除尾结点

(I) 声明两个结构指针变量 p 和 q: **struct lnode *p, *q;**

(II) 删除尾结点的操作:

if(!head) return; 如果链表为空, 则直接返回

if(!head->next) { **p=head; head=NULL;** }

如果链表中只有一个结点, 令p指向该结点, 并置head为空 (删除后链表就为空了)。

else { //至少有二个结点

p=head;

while(**p->next**) { next==NULL的结点就是最后一个结点

q=p; 令 q 是 p 的前驱

p=p->next;

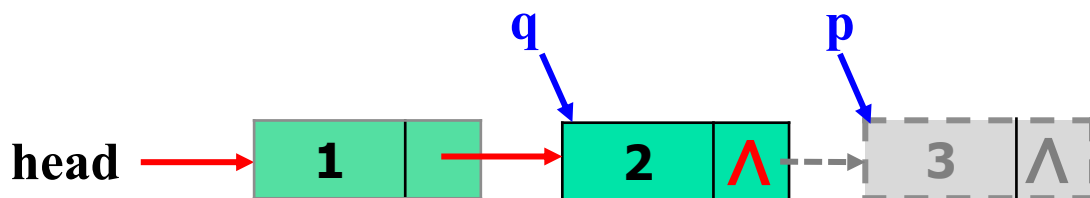
}

q->next=NULL;

}

q 所指的结点在当前尾结点被删除后将成为新的尾结点

free(p); 释放 p 指向的结点

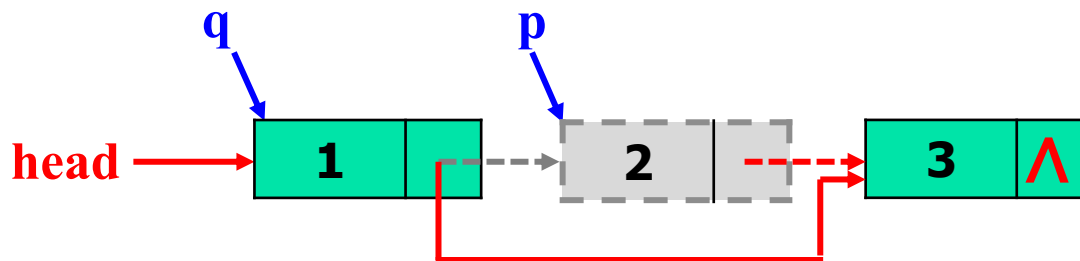


情况3：删除某个查找到的结点

声明二个临时指针变量 p 和 q: `struct lsnod *p, *q;`

首先用 p 扫描链表，找到待删除的结点，p 指向该结点；同时令 q 指向 p 的前驱结点，然后令 `q->next=p->next`，最后释放 p。

如删除下面链表中 data 域等于2的结点。



例：下述函数在参数head指示的链表中删除数据域data为x的结点。

```
void DeleteNode(struct lnode ** head, int x)
```

```
{
```

由于可能会改变头指针的指向，所以这里 head 是一个二级指针

```
    struct lnode *p, *q;
```

```
    if((*head) == NULL) return;    链表是空的，直接返回
```

```
    if((*head)->data == x) { p=*head; *head=(*head)->next; free(p); }
```

```
    else {
```

如果首结点的值就等于x，则令 p=*head，同时删除首结点后第二个结点就成为新的首结点，所以令头指针指向其后继。最后free(p)。

```
        q=*head;
```

```
        p=q->next;
```

```
        while(p!=NULL && p->data!=x) {q=p; p=p->next;}    p 扫描链表，查找值等于 x 的结点，并令 q 指向 p 的前驱
```

```
        if(p) { q->next=p->next; free(p); }
```

```
    }
```

◆ 如果 p 不为空，则代表找到了值为 x 的结点，然后令其前驱指向其后继，free 掉 p 即可。

◆ 如果 p 为空，则代表没有找到值为 x 的结点，此时什么都不做。

```
}
```

情况4：删除整个链表

链表使用完后，要把所有的结点都释放掉。链表不支持整体操作，必须一个结点一个结点释放。

下面的函数将一个链表的所有结点逐一释放掉。

```
void DeleteList(struct lsnod ** head)
```

```
{
```

head是二级指针

```
    struct lsnod * p = *head;
```

```
    while(p!=NULL) {
```

```
        *head = p->next;
```

```
        free(p);
```

```
        p = *head;
```

```
    }
```

退出循环时，*head==NULL，
表示链表“被清空”了。

```
}
```

p 是马上要删除的结点，令头指针后移，
以准备下次循环。

注：删除整个链表不能

free(head); ❌

```
#include <stdio.h>
#include <malloc.h>
```

```
void InsertNode(struct lsnod ** head, int x);
```

有序插入结点

```
void DeleteNode(struct lsnod ** head, int x);
```

```
void DeleteList(struct lsnod ** head);
```

```
int main( )
```

```
{
```

```
    struct lsnod * head=NULL, *p;
```

```
    int x, y;
```

```
    scanf("%d", &x);
```

```
    while(x) {
```

创建“有序”链表

```
        InsertNode(&head, x);
```

```
        y=x;
```

用&head做实参

```
        scanf("%d", &x);
```

```
    }
```

```
    p=head; while(p) { printf("%d ", p->data); p = p->next; }
```

输出

```
    DeleteNode(&head, y);
```

删除最后输入的那个元素

```
    p=head; while(p) { printf("%d ", p->data); p = p->next; }
```

再输出

```
    DeleteList(&head);
```

销毁链表

```
}
```

使用链表的流程

①创建链表



②使用链表



③销毁链表

如输入：

2

4

5

1

3

则输出：

1 2 3 4 5

1 2 4 5

9.8.5 链表的其它操作

1、遍历链表，输出一个链表中各个结点的信息（或其它应用）

```
void PrintList(struct lsnod * head)
```

```
{
```

```
    struct lsnod *p;
```

```
    p=head;
```

```
    while(p!=NULL) {
```

```
        printf(“%d\t”, p->data);
```

```
        p=p->next;
```

```
    }
```

```
}
```

设置一个扫描指针 p，用 p 遍历链表，顺序输出各结点的信息。

例：用**递归遍历**的方式遍历链表，并统计链表中结点的数目。

链表的结点数目 = $\begin{cases} 0, & \text{如果 head} == \text{NULL}; \\ 1 + \text{head后面子链表的结点数目}, & \text{否则。} \end{cases}$

```
int countnodes_recursive(struct lsnod * head)
{
    if(head)
        return (1+count_nodes_recursive(head->next));
    else
        return 0;
}
```

2、逆序输出一个链表中各个结点的信息

- ◆ **逆序输出**：从后往前输出，用递归过程实现。

```
void ReversePrintList(struct lsnod * head)
{
    if(head!=NULL) {
        ReversePrintList(head->next);    /*先递归输出后续结点*/
        printf("%d\t", head->data); /*再输出自己，实现逆序输出*/
    }
}
```

正序输出：从前往后输出，用递归过程实现。

```
void PrintList(struct lsnod * head)
{
    if(head!=NULL) {
        printf("%d\t", head->data); /*先输出自己*/
        PrintList(head->next);    /*再递归输出后续结点*/
    }
}
```


上次课主要内容 (2024.12.6)

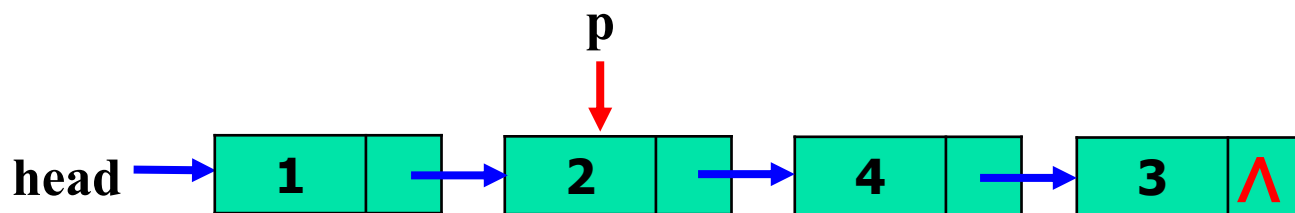
链表：

- 1、**自引用结构类型、自引用结构指针**
- 2、**链表的相关概念**：结点、首结点、尾结点、前驱结点、后继结点、单向链表、头指针
- 3、**链表扫描、结点查找**
- 4、**链表的插入**：动态申请结点、前驱结点的查找、头插法、尾插法、中间插入、有序插入/无序插入
- 5、**链表的删除**：删除首结点、尾结点、中间某个结点
删除整个链表
- 6、**交换链表的结点**：只交换data域、交换结点位置

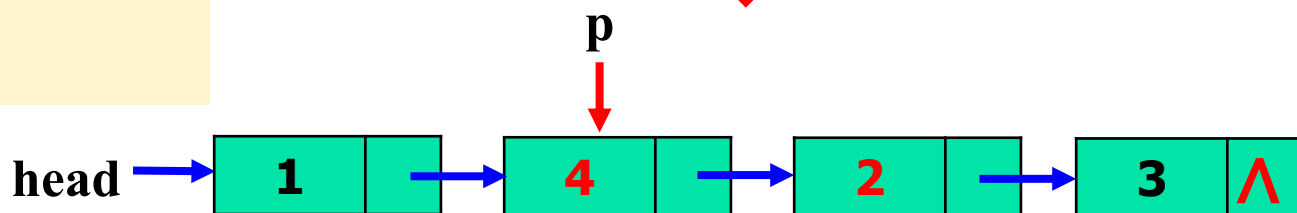
3、交换链表中两个相邻的结点

方式1：只交换结点的data，指针域不变

如，指针 **p** 指向链表中的一个结点，交换 **p** 和其**直接后继**的 data (即交换 **p->data** 和 **p->next->data**)，指针域不变。

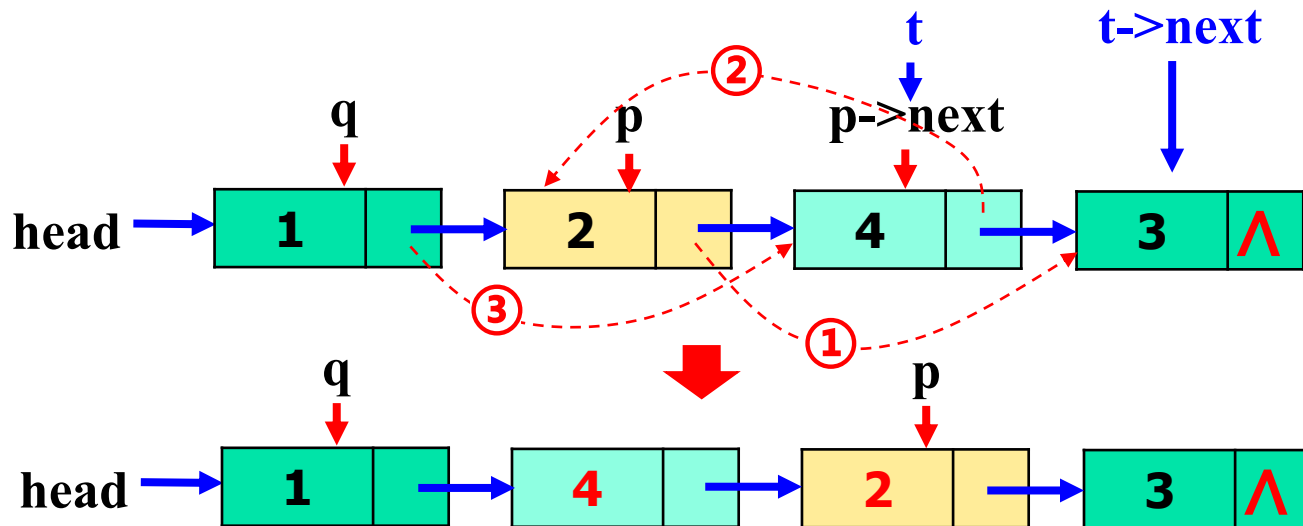


```
int t;  
t = p->data;  
p->data = p->next->data;  
p->next->data = t;
```



方式2：交换结点，此时它们在链表中的位置要互换

如，指针 **p** 指向链表中的一个结点，交换 **p** 和其**直接后继**。



此时要先找到 **p 结点的前驱**，用指针 **q** 指向 **p** 的前驱结点。

然后再执行以下操作：

`struct lsnod * t;`

`t = p->next;`

① `p->next=t->next;`

② `t->next=p;`

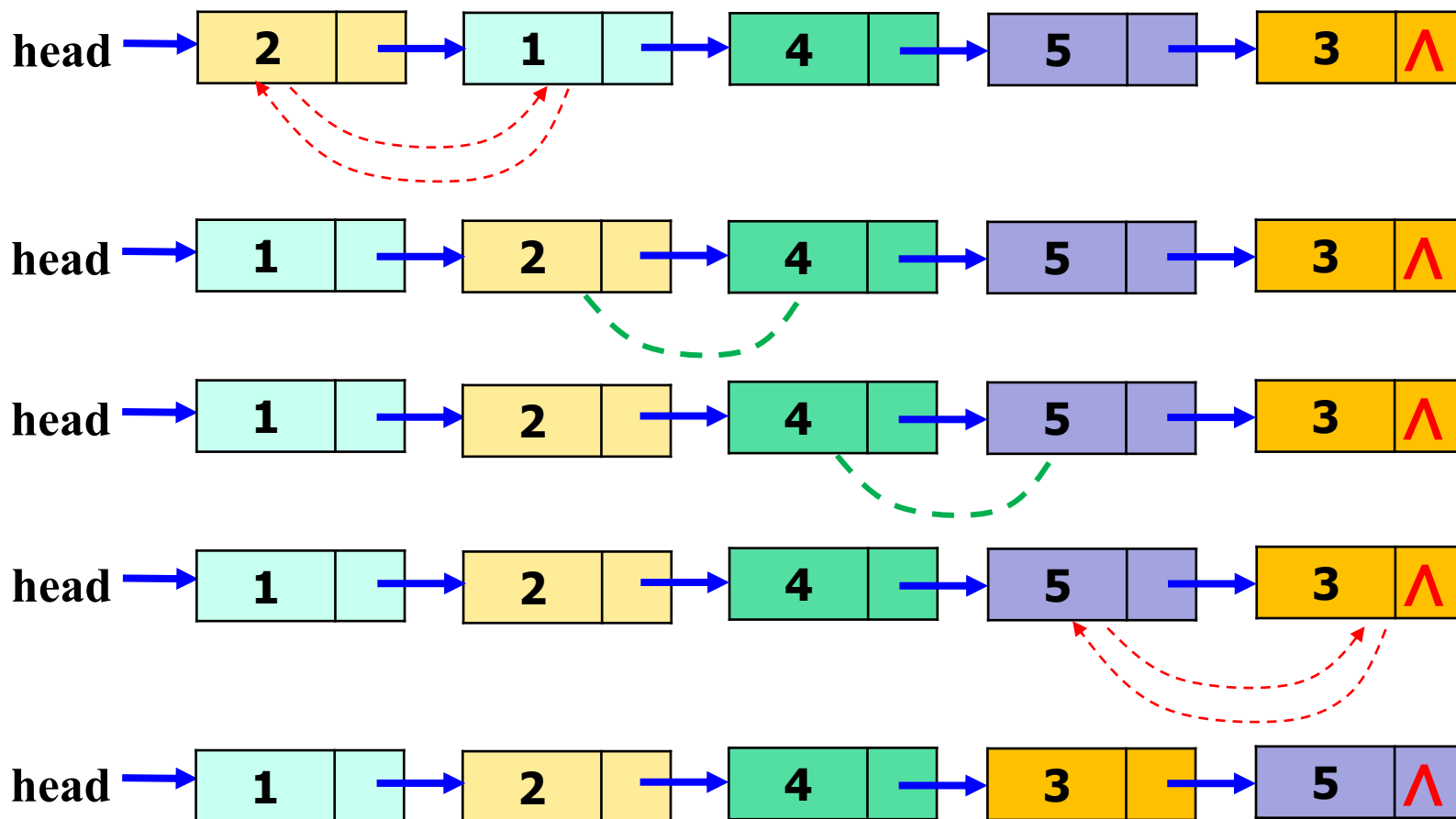
③ `q->next=t;`

这两句顺序不能颠倒

应用：基于链表实现findmax过程

findmax：通过**比较-交换**相邻的结点，**将链表中data最大的结点交换到尾结点**，采用**交换结点**的方法。

如：



```
void findmaxlist(struct lsnode ** head)
```

```
{  
    struct lsnode *p, *r, *t;
```

```
    if((*head)->data > (*head)->next->data) {
```

首结点比第二个结点数大，交换前两个结点

```
        p=(*head)->next; (*head)->next=p->next; p->next=*head; *head=p;
```

头指针发生变化

```
    }
```

```
    else {
```

```
        r=*head; p=*head->next;
```

从第二个结点开始向后两两比较，r 保持为 p 的前驱。

```
        while(p->next != NULL) {
```

```
            if(p->data > p->next->data) {
```

前大后小，需要交换

```
                t=p->next->next; p->next->next=p; r->next=p->next; p->next=t;
```

①

②

③

```
            }
```

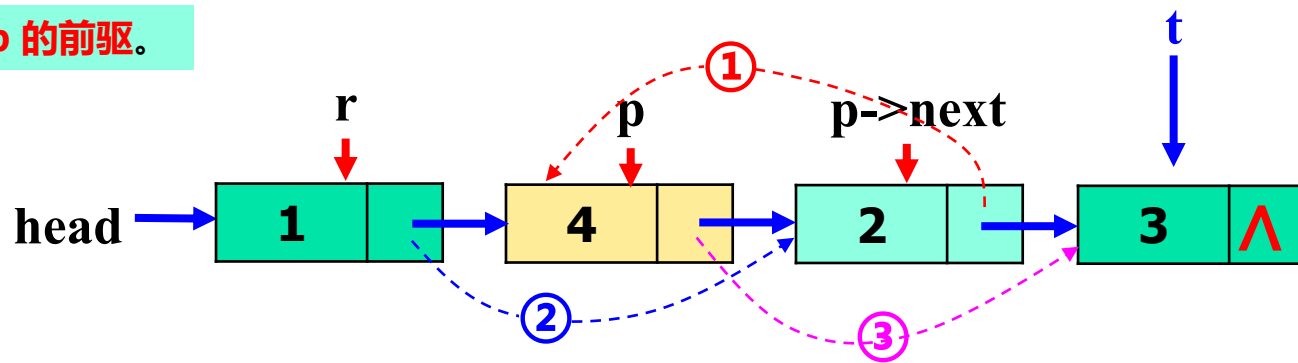
```
            r=p;
```

r 保持为 p 的前驱。

```
            p=p->next;
```

```
        }
```

```
    }
```



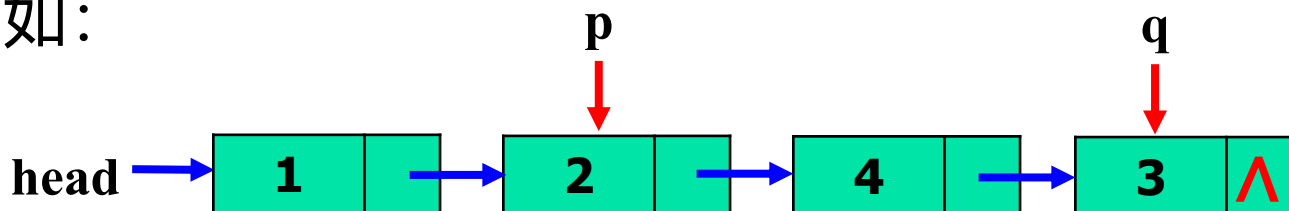
应用：基于findmax过程实现基于链表的冒泡排序
自己尝试实现。

注：书上引入了一种**带有头结点的单链表**形式。带有头结点的单链表见下面的选择排序。

4、交换链表中任意两个结点

声明二个指针变量 p 和 q : `struct lsnod *p, *q;`

扫描链表, 找到待交换的两个结点, 并用 p 和 q 分别指向这两个结点。如:



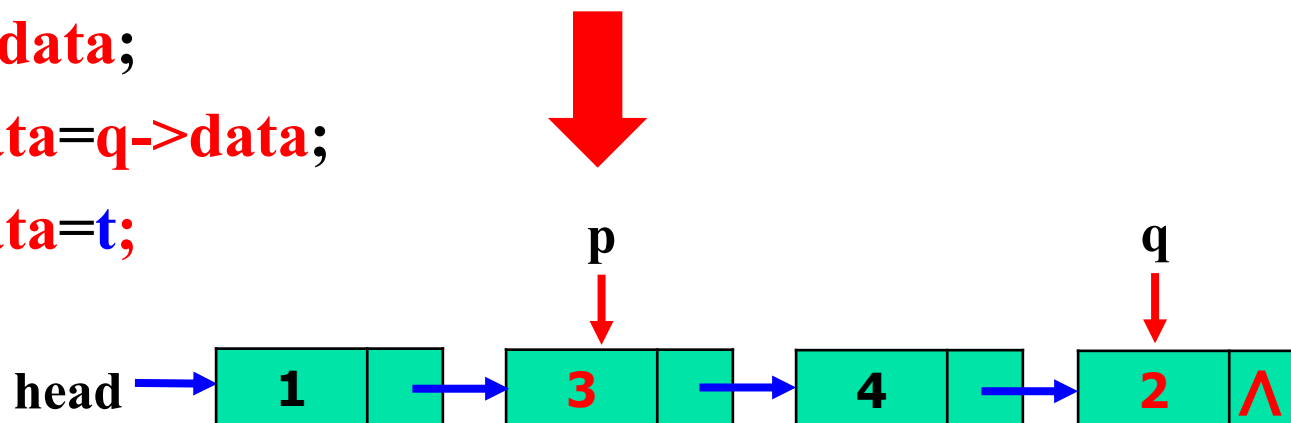
方式1: 只交换两个结点的 `data`, 不改变它们在链表中的位置

```
int t;
```

```
t=p->data;
```

```
p->data=q->data;
```

```
q->data=t;
```



方式2：交换结点，此时它们在链表中的位置要互换

首先扫描链表，找到待交换的两个结点，并用 **p** 和 **q** 分别指向这两个结点。然后分两种情况：

(1) p 或 q 其中之一刚好是首结点，不失一般性，设 **p==head**。

此时，**找到 q 的前驱结点**，并用 **r** 指针指向该前驱结点，如图，然后执行以下操作：

```
struct lsnod * t;
```

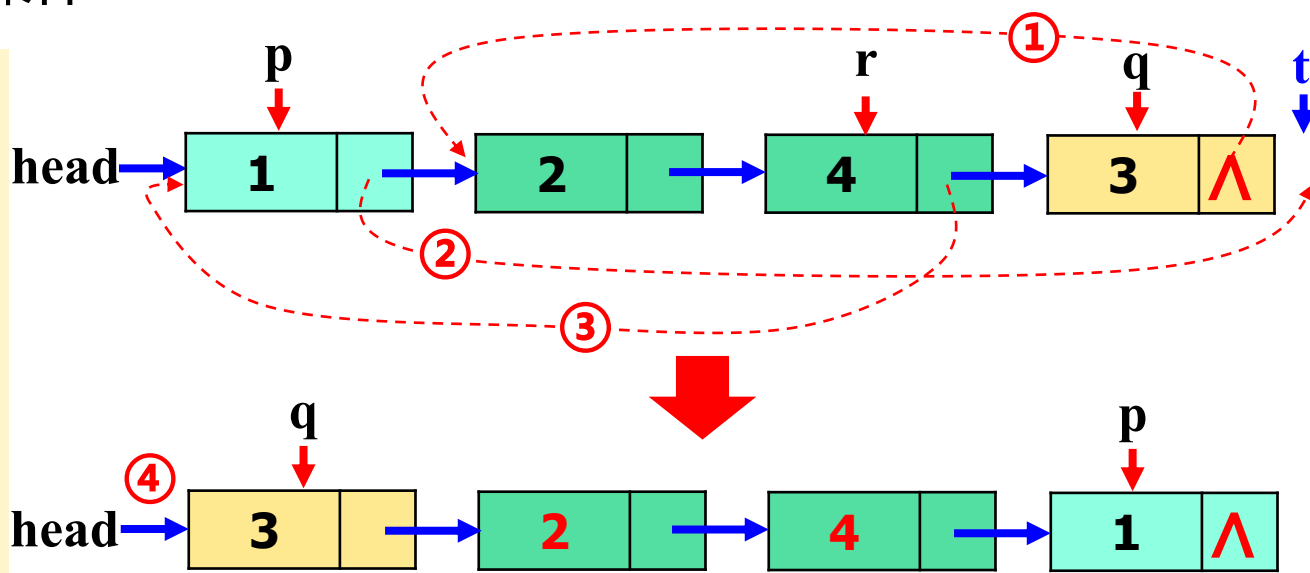
```
t = q->next;
```

```
① q->next=p->next;
```

```
② p->next=t;
```

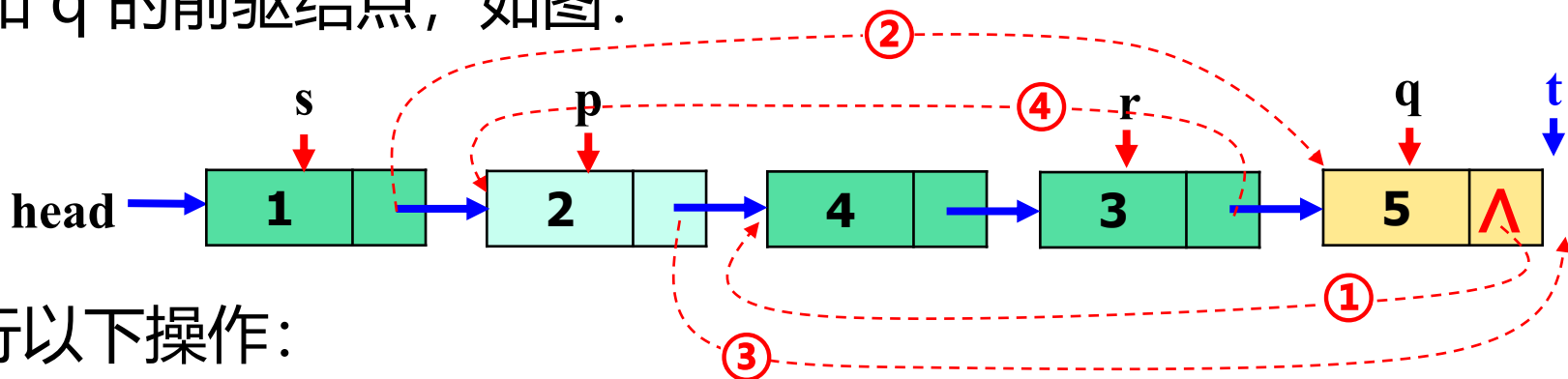
```
③ r->next=p;
```

```
④ head=q;
```



(2) p和q都不是首结点

此时，分别**找到 p 和 q 的前驱结点**，并用 **s 和 r** 指针分别指向 p 和 q 的前驱结点，如图：



然后执行以下操作：

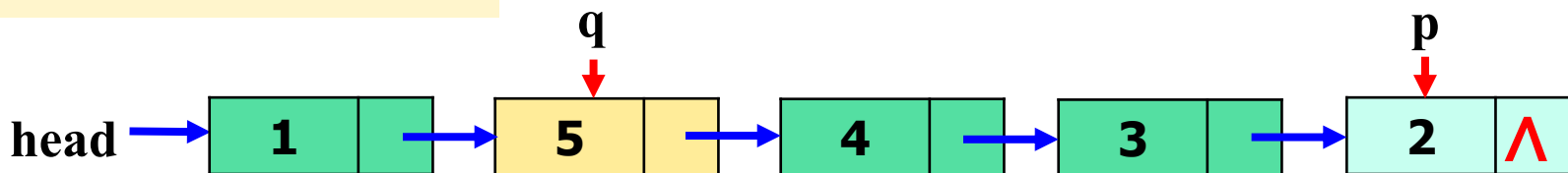
```
struct lnode * t = q->next;
```

① q->next = p->next;

② s->next = q;

③ p->next = t;

④ r->next = p;



例：交换链表中两个指定的两个不同结点。

```
void Exchange2Nodes(struct lsnod ** head, struct lsnod * p, struct lsnod * q)
```

```
{
```

```
    struct lsnod *s, *r, *t;
```

如果 q 指向首结点，交换 p 和 q 的值，使 p 指向首结点

```
    if(q==*head) { t=p; p=q; q=t; }
```

```
    r=*head; while(r->next!=q) r = r->next; 查找 q 的前驱结点。用 r 指向该前驱结点
```

```
    if(p==*head) { p 是首结点的情况
```

```
        t=q->next; q->next=p->next; p->next=t; r->next=p; *head=q;
```

①

②

③

④

头指针发生了变化

```
    }
```

```
    else { p 不是首结点的情况
```

```
        s=*head; while(s->next!=p) s = s->next; 查找 p 的前驱结点。用 s 指向该前驱结点
```

```
        t=q->next; q->next=p->next; s->next=q; p->next=t; r->next=p;
```

①

②

③

④

```
    }
```

```
}
```

5、基于链表的选择排序

选择排序算法也是一个**多趟循环**处理过程，基本思想如下：

设 **a** 是由 **n 个数据元素**组成的序列，开始的时候 a 中的元素无序，然后进入下面的 **n-1 次“趟”循环处理过程**，将其元素逐步调整有序（不失一般性，这里假设按增序顺序进行排序）：

第一趟循环： 在**整个序列的 n 个数据范围**内，从第一个元素开始至最后一个元素查找**最小元素所在的位置**，然后**将其和 a 中的第一个元素交换**，这样最小元素就排在了“第一小”的位置上了——称为元素“**就位**”了。

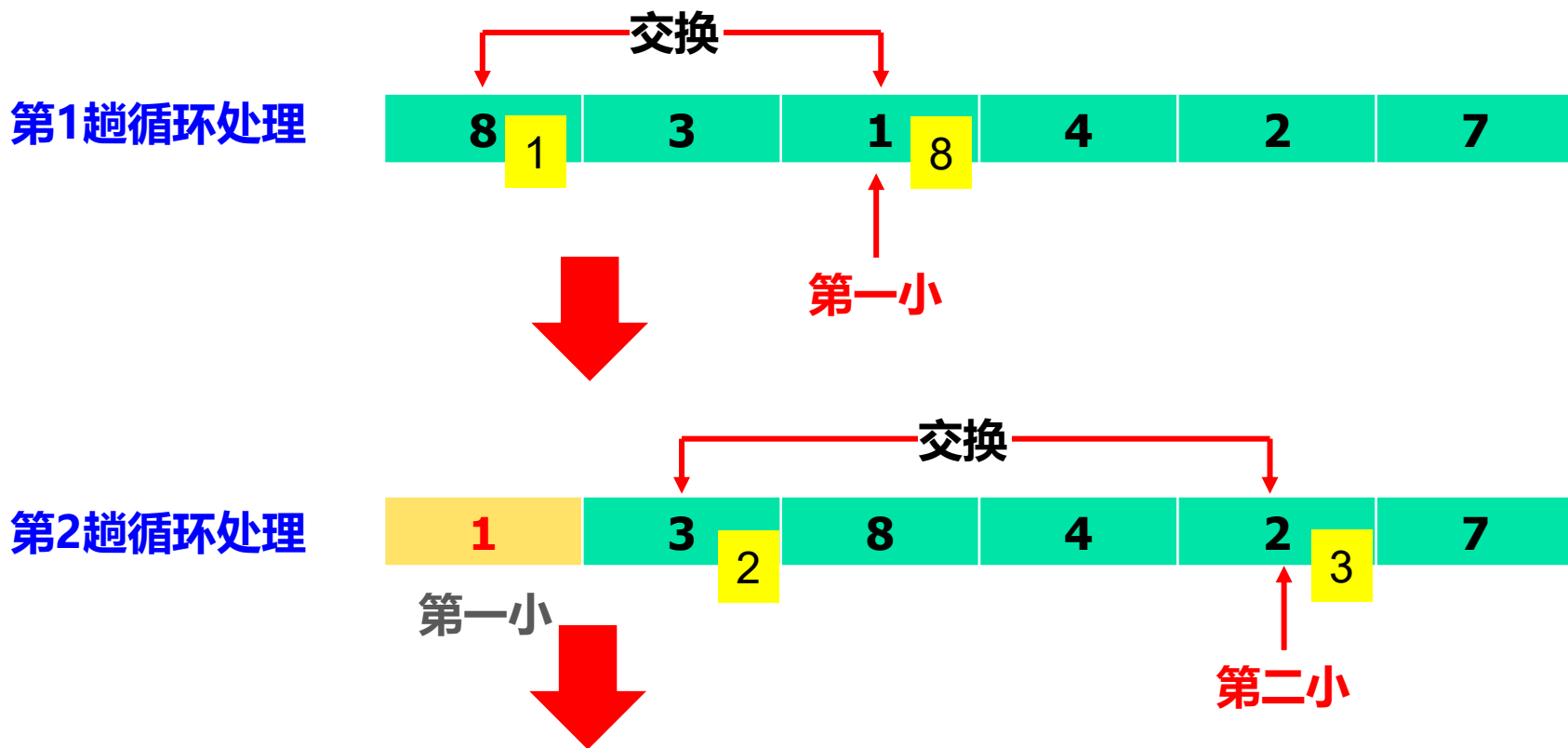
而剩下的从第二个元素至最后一个元素的 **n-1** 个元素还没有序，为此进入第二趟循环：

第二趟循环：在第一趟循环处理后剩下的 $n-1$ 个数据范围内（第二个元素至最后一个元素）再找到其中的最小元素，这个元素是所有 n 个元素里的**第二小元素**，同样记下它的位置，然后**把它和现在整个序列的第二个元素交换**，这样第二小元素也“**就位**”了——此时**整个序列的第一小、第二小就都“就位”**了。

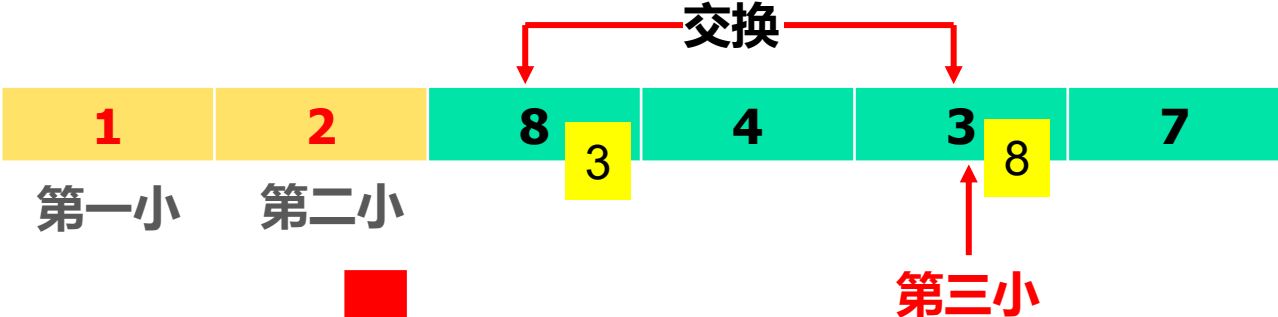
再剩下的就是从第三个至最后一个元素（共 $n-2$ 个元素）还没有排序，为此继续进行类似的循环处理：

第*i*趟循环处理：找到上一趟（第 $i-1$ 趟）处理后剩下的还没排序的 $n-i+1$ 个元素中最小的元素，该元素是整个序列的第 i 小元素，记下位置，然后把它和整个序列的第 i 个元素交换，这样整个序列的**第 i 小元素就“就位”**了。

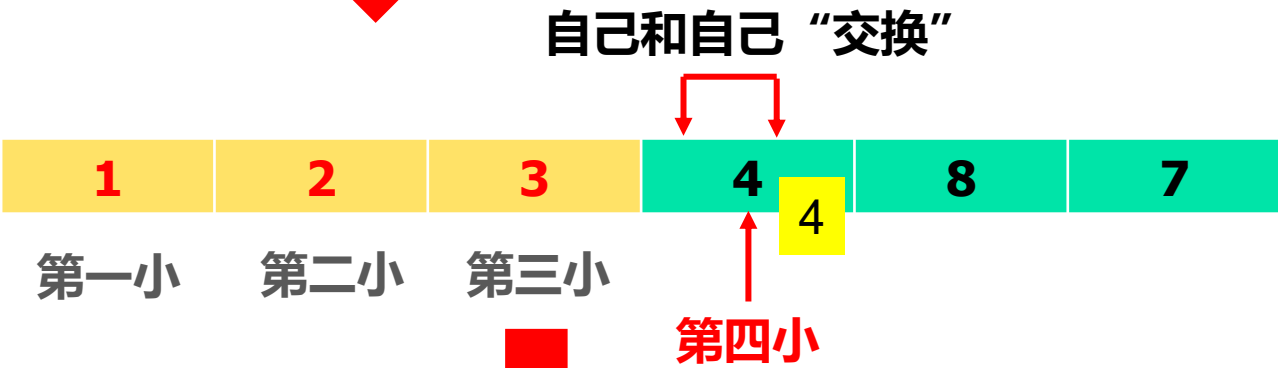
上述循环处理过程重复 $n-1$ 次，第 i 趟循环，第 i 小元素“就位”，所以 $n-1$ 趟循环处理后，第 1 小至第 $n-1$ 小元素依次“就位”，最后还剩下一个元素，就是整个序列的最大元素，而这个元素也已经在 a 的最后一个位置“就位”了。所以全局来看，所有的元素都排列有序了。这个时候循环终止，算法结束。



第3趟循环处理



第4趟循环处理



第5趟循环处理



完成

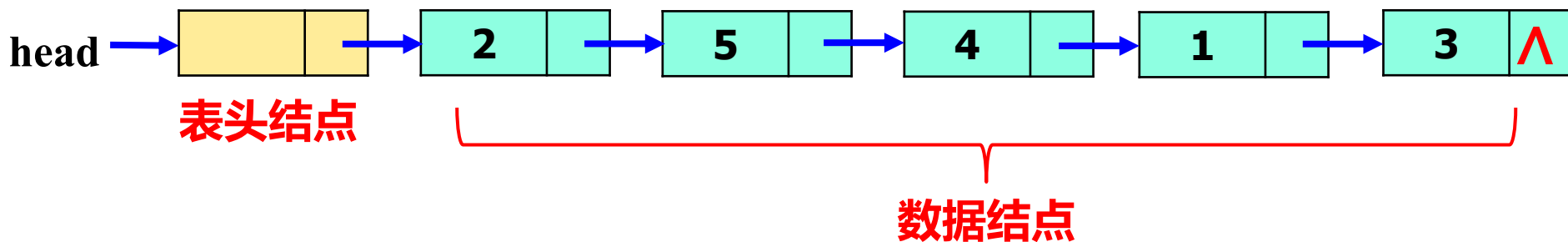


(1) 基于数组实现的选择排序算法（如上所示，自己实现）

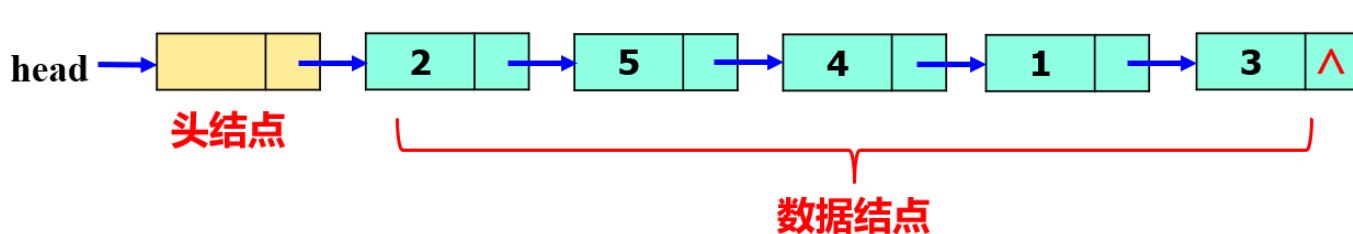
(2) 基于链表实现的选择排序算法

带表头结点的链表：在第一个**数据结点**之前**增加一个结点**，其类型一般和数据结点类型一样，但里面不存数据，只是指向第一个数据结点，这个结点称为**表头结点**（注意：表头结点不是链表的数据结点，和前面的首结点不同），这样的**增加了表头结点的链表**称为**带表头结点的链表**。

如下所示，是一个带有头结点的单链表。



在**带表头结点的链表**中，**头指针**指向**表头结点**，**表头结点**（的next）再指向**第一个数据结点**。



创建带表头结点的链表，先预先生成**表头结点**，并令**head**指向它、**head->next=NULL**，表示空表：

```
head = (struct *)malloc(sizeof(struct T));  
head->next=NULL;
```

然后再将后续的数据结点插在表头结点后面。

注：对带表头结点的链表，**空表**并不是“真空”，其中有且只有表头结点，只是没有数据结点，并且**表头结点的next域为空**。所以此时**表空的判断条件**是：

```
if( head->next==NULL ) printf("空表!"); else .....
```

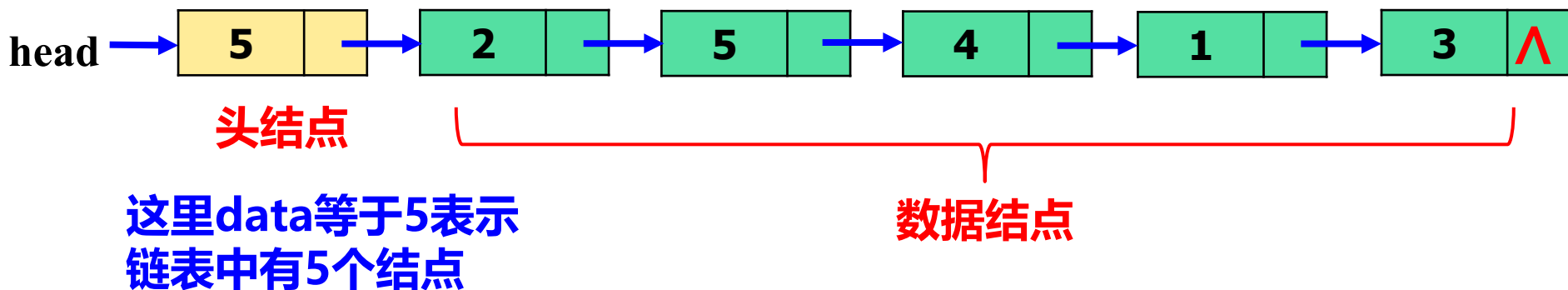

增加表头结点的作用：简化程序控制

不带表头结点的时候，如果操作过程中涉及改变首结点，就要相应修改头指针 **head**，使之指向新的首结点，所以处理的时候就总要对首结点进行单独判断和处理，控制起来比较麻烦；

引入表头结点后，头指针 **head** 总是指向表头结点，表头结点指向第一个数据结点。表头结点不会改变，**head** 指针指向表头结点的关系也不会变

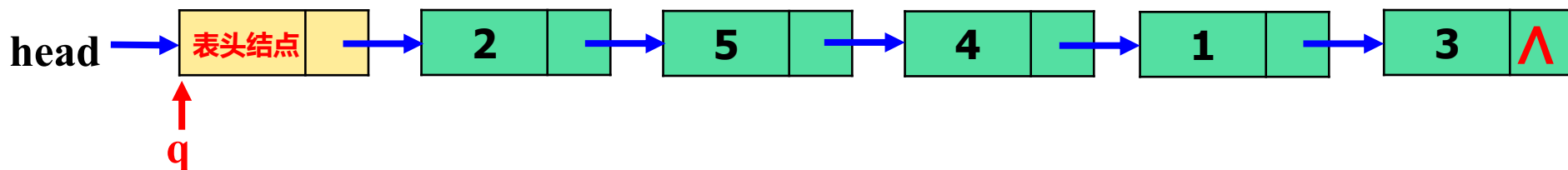
这样不管第一个数据结点怎么变，都不会影响 **head** 指针的指向，就会使得对第一个数据结点的处理与对其它数据结点的处理在控制逻辑上是一致的，而不需要对第一个数据结点及 **head** 做单独的判断和处理，程序流程会更加简洁、清晰。

注：事实上，**头结点也可以存数据**，只是不是结点数据。比如在数据域保存整个链表中的结点数目，这样可以在以后的链表循环处理中，用结点数来控制循环，比用指针控制“略显简单”。还可以记录其它一些信息，灵活使用。如



基于带表头结点的链表实现的选择排序算法

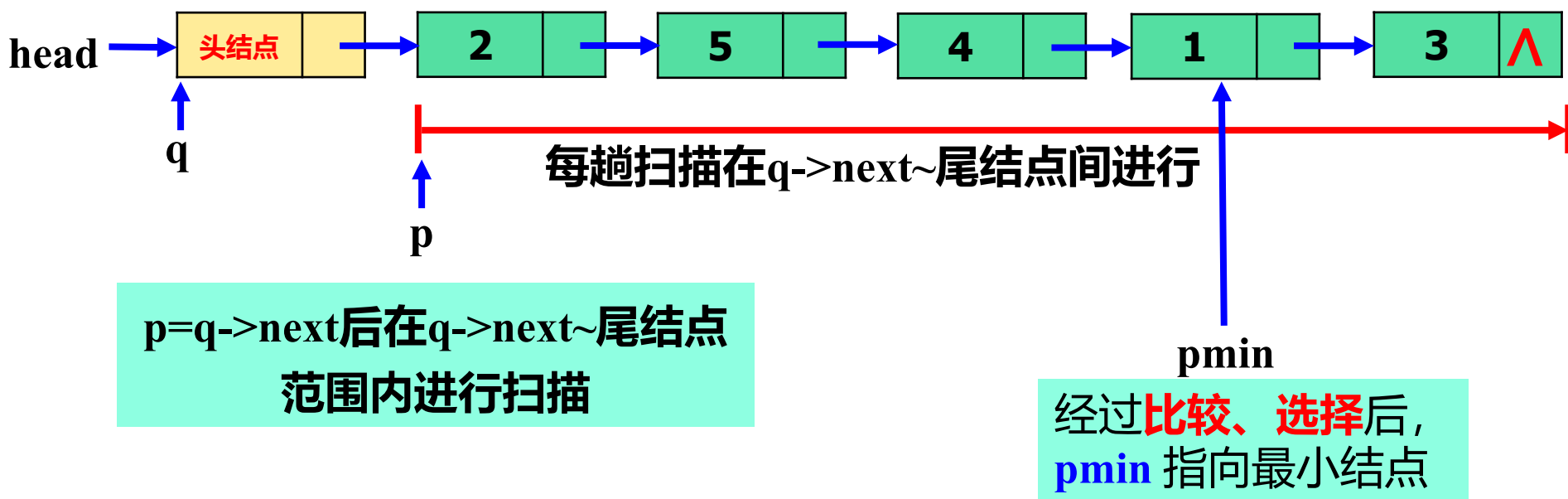
首先对序列建一个带表头结点的单链表：每个元素对应一个数据结点（data 域放元素的值），加上表头结点，链表中总共有 $n+1$ 个结点：head 指向表头结点，表头结点指向第一个数据结点、...。开始的时候元素间无序。



然后进入 $n-1$ 趟循环处理：

① 设置一个指针 q ，开始的时候也指向表头结点： $q=\text{head}$ ，表示从头开始第一趟扫描（如上所示）。

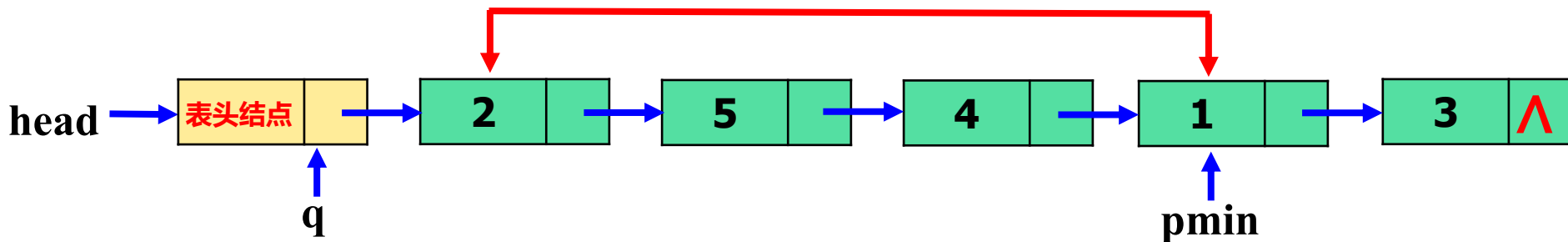
② 然后再设置一个**扫描指针 p**，令 $p = q \rightarrow next$ ，在 **q 的后继结点~尾结点范围内**进行扫描，找出其中**data**值最小的结点，并令**第三个指针 pmin**指向该最小结点。



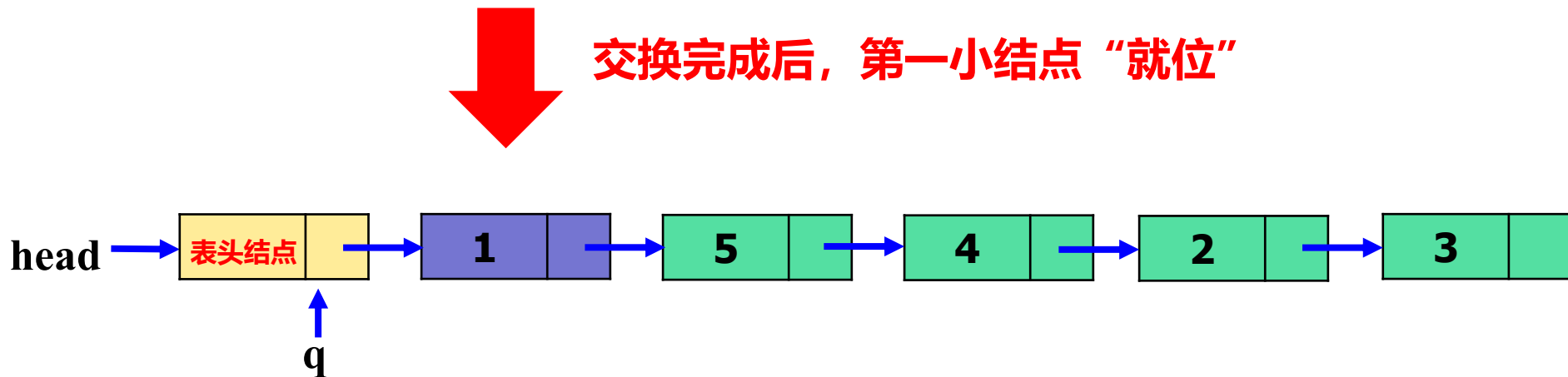
注：**q** 总是**指向**下一趟要扫描范围内的第一个结点的前驱，然后 **p** 从 $q \rightarrow next$ 开始向后扫描。

③ 交换 $q \rightarrow \text{next}$ 和 pmin 两个指针所指的结点，从而本趟范围内的最小结点就“就位”了。

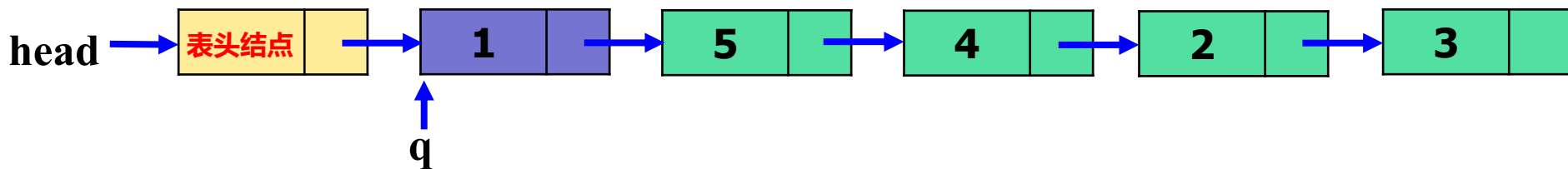
交换 $q \rightarrow \text{next}$ 所指的结点和 pmin 所指的结点



交换完成后，第一小结点“就位”



④ **$q = q \rightarrow next$** : q 指向下一个结点, 做好下一次循环的准备。



第一趟循环处理完成

开始第二趟循环处理: 与上面基本一样的**扫描/交换**过程。

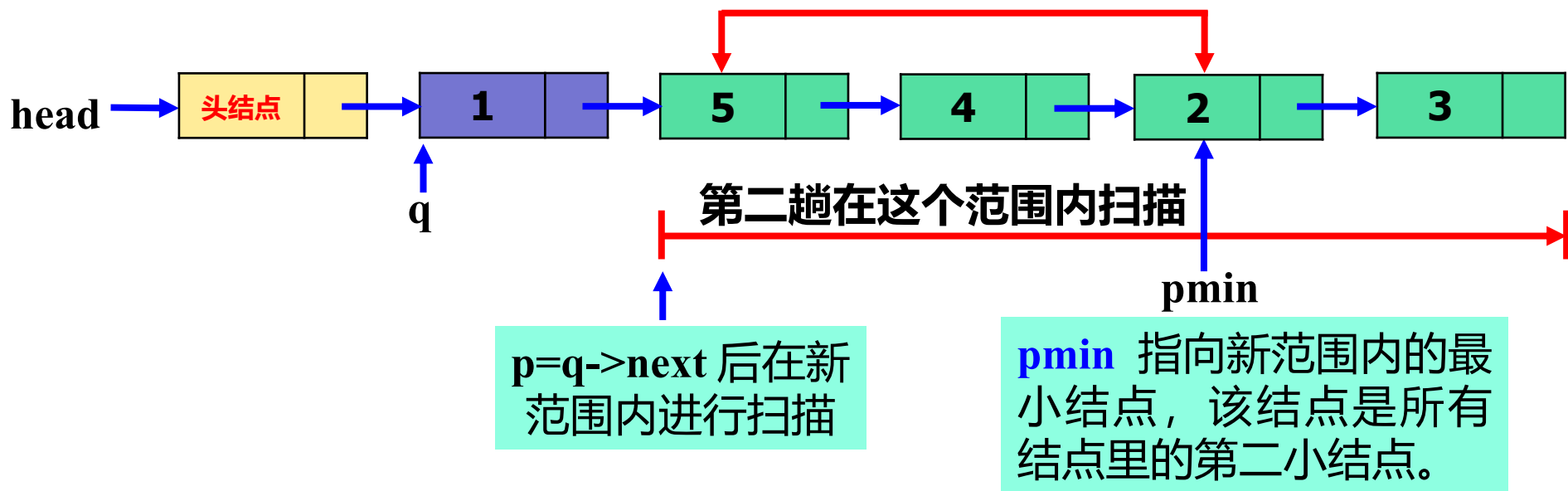
基于 q 现在所指的结点位置, 令 **$p = q \rightarrow next$** , **重新在现在的 $q \rightarrow next \sim$ 尾结点范围内**查找其中的“**最小**”结点 (这个结点也就是所有 n 个结点里面的**第二小结点**)。之后**把这个第二小结点和当前的 $q \rightarrow next$ 所指的结点交换**, 从而第二小结点也就“**就位**”了。

再之后, 再令 **$q = q \rightarrow next$** , 重复上面的循环处理.....。

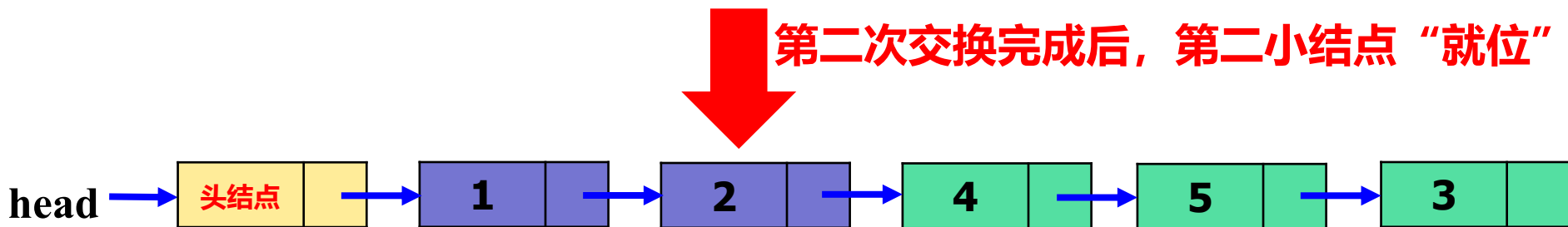
每次循环, 一个“小”元素就位, 总共 **$n-1$** 次后排序完成。

如**第二趟**循环处理：

交换 $q \rightarrow next$ 所指的结点和 $pmin$ 所指的结点



第二次交换完成后，第二小结点“就位”



```
void selectsort(struct lsnode * head)
```

```
{ struct lsnode *p, *q, *pmin, *r, *rm;
```

```
q=head;
```

两个前驱指针

```
while(q->next) {
```

```
    r=q; p=q->next; 设好 p 和它的前驱指针 r 的初始值
```

```
    rm=q; pmin=q->next;
```

```
    while(p) { 设好pmin和它的前驱指针rm的初始值
```

```
        if(p->data<pmin->data) { rm=r; pmin=p; }
```

```
        r=p; p=p->next;
```

```
    } 平移 r 和 p, 继续向后搜索
```

```
    if(pmin!=q->next) {
```

```
        t=q->next->next;
```

```
        rm->next=q->next;
```

```
        q->next->next =pmin->next;
```

```
        pmin->next=t;
```

```
        q->next=pmin;
```

在找到 pmin 后, 如果 pmin 不是当前搜索范围内的第一个结点, 就交换 pmin 和 q->next。否则不用动。

```
}
```

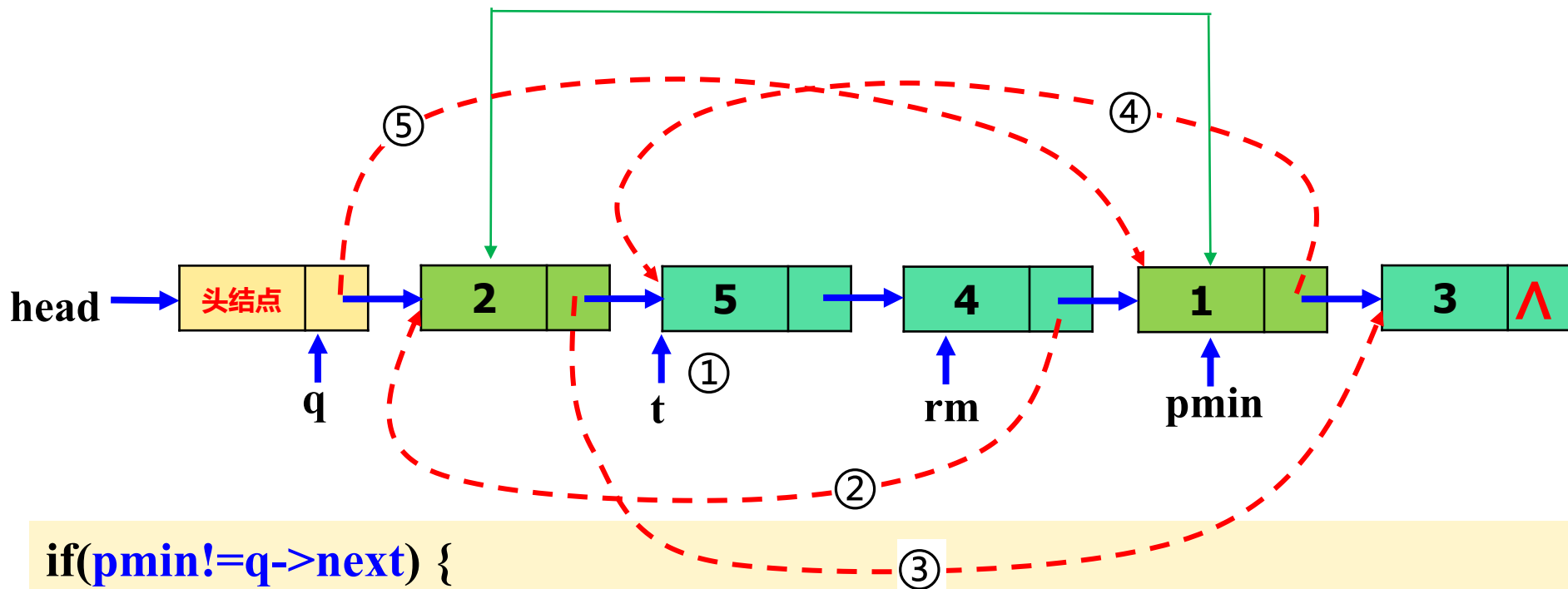
```
q=q->next; 继续下一趟的循环
```

```
}
```

说明: 为了实现 **q->next** 和 **pmin** 的交换, 程序中还需要在记下 pmin 的同时, 记下 pmin 的前驱结点。为此引入 **rm 指针指向 pmin 的前驱结点**。而为了在搜索过程中在找到 pmin 的同时也记下 rm, 程序中同时引入 **r 指针指向 p 的前驱结点**。在 p 向后搜索的过程中, r 同步向后移动, 而在 p 找到一个更小的结点时, r 也就是该结点的前驱。此时令 **rm=r; pmin=p;** 就同时记下当前最小结点及其前驱结点了。

如下图，交换 $q \rightarrow \text{next}$ 和 pmin 两个指针所指的结点。

交换 $q \rightarrow \text{next}$ 所指的结点和 pmin 所指的结点



```
if( $\text{pmin} \neq q \rightarrow \text{next}$ ) {
```

```
    ①  $t = q \rightarrow \text{next} \rightarrow \text{next};$ 
```

```
    ②  $\text{rm} \rightarrow \text{next} = q \rightarrow \text{next};$ 
```

```
    ③  $q \rightarrow \text{next} \rightarrow \text{next} = \text{pmin} \rightarrow \text{next};$ 
```

```
    ④  $\text{pmin} \rightarrow \text{next} = t;$ 
```

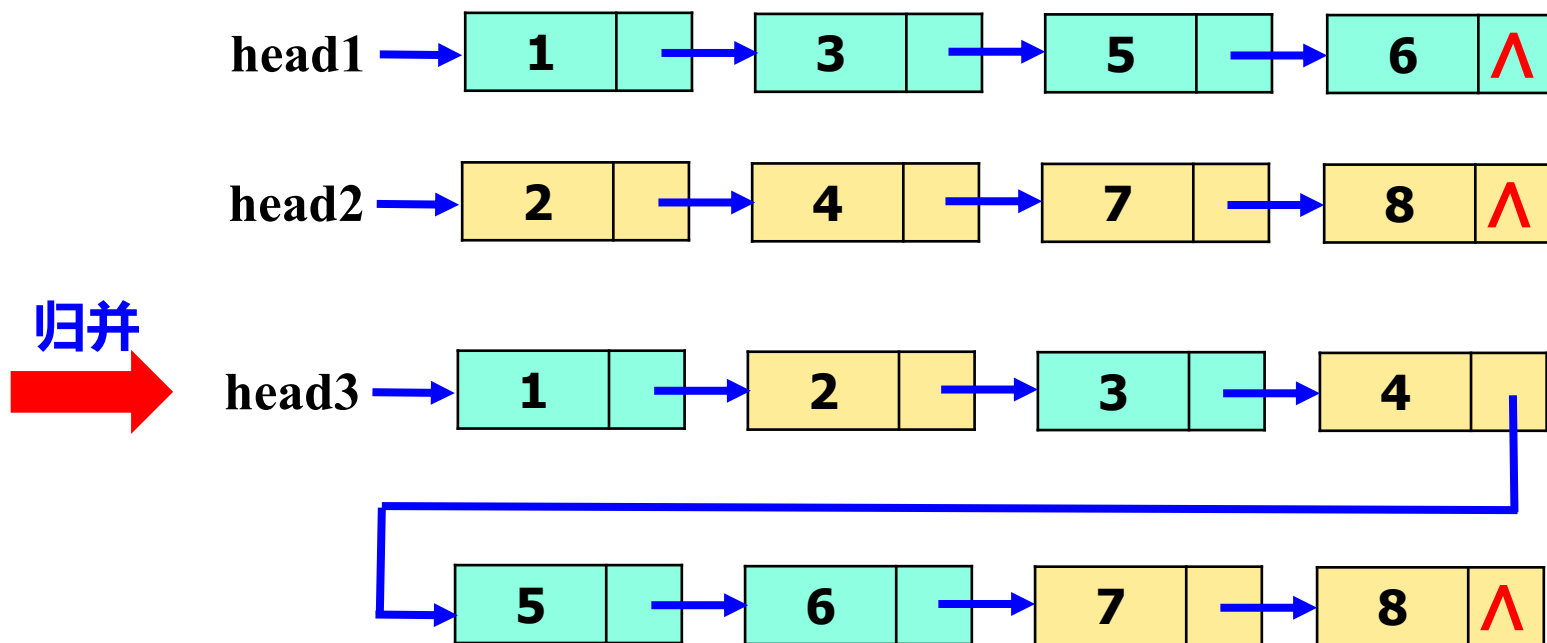
```
    ⑤  $q \rightarrow \text{next} = \text{pmin};$ 
```

```
}
```

} 这两句顺序不能颠倒

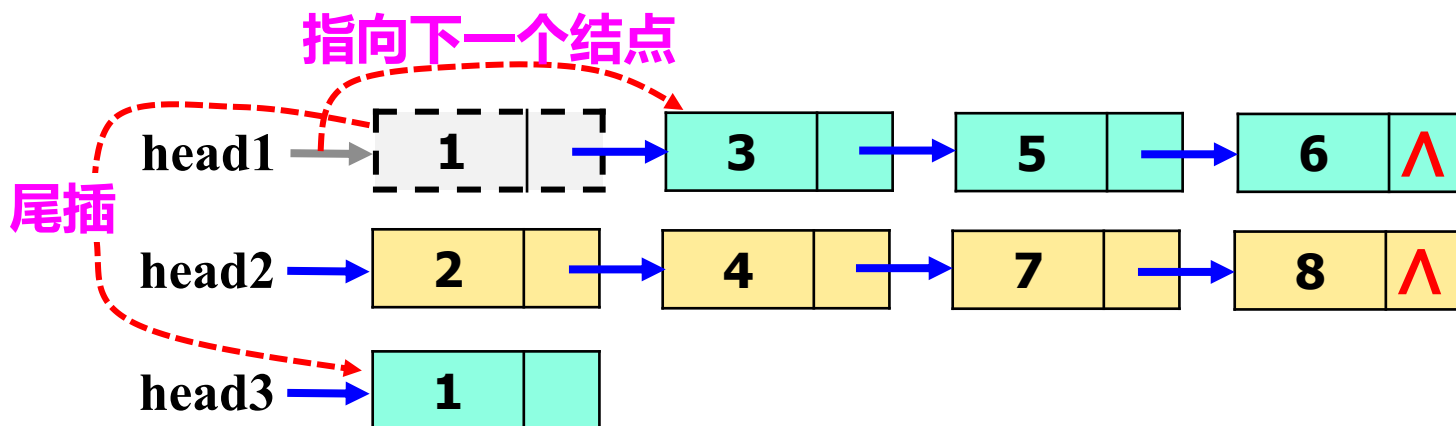
6、归并链表

问题描述：设有两个已经按递增排好序的链表，头指针分别为**head1**和**head2**（都不带表头结点），**合并两个链表而生成一个包含两个链表所有结点且递增有序的新链表head3。**



策略：

- (1) **比较两个头指针当前所指的两个结点的大小，将小的结点从原链表中摘除，并以尾插法插入到新链表head3中；而小结点从原链表中摘除后，该链表的头指针指向原先的第二个结点。**



- (2) 然后**重复(1)**，继续比较两个头指针当前所指的两个结点的大小，每次都把一个小结点从原链表中摘除并尾插到新链表中。
当某个链表变空后，直接将另一个链表还剩下一些 **“大” 结点** 接到head3链表的后面，算法结束。**程序见P318，自行学习。**

9.8.6 循环单链表

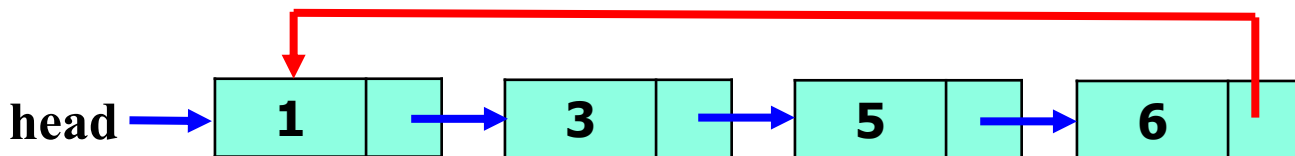
单链表中，每个结点的指针指向其直接后继，最后一个结点指针为空（NULL）。



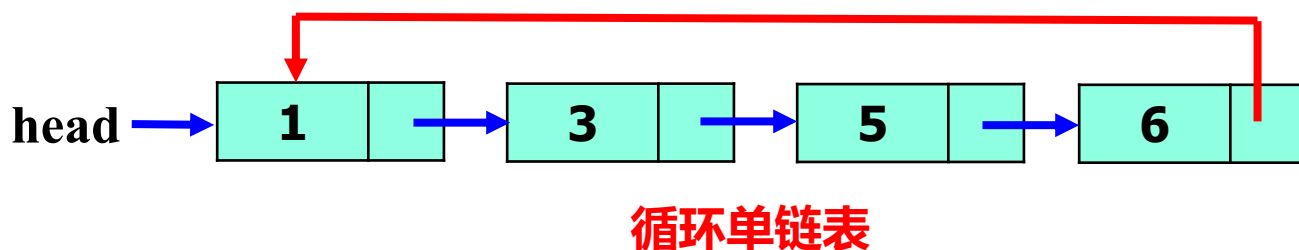
单链表

最后一个结点的next == NULL

循环单链表：如果把单链表中**最后一个结点的 next 指针改为指向第一个结点**（把首结点（的地址）赋给尾结点的 next 指针），就形成一个“**环**”，这样的链表称为**循环单链表**。



循环单链表



循环单链表的特点：

- ◆ 循环单链表中**没有空指针域**；
- ◆ 遍历循环单链表时，可以从任何一个结点出发，而不必一定从首结点出发，**沿 next 指针在链表中“打转”**，就可以访问到链表中的所有结点。

循环单链表使用的注意事项：

在增、删结点时注意保持其**“循环链表”**的结构特征即可。

循环单链表**“最后一个结点”**的判定： **$p \rightarrow next == head$**

9.8.7 双向链表

双向链表：结点的指针域包含**两个自引用指针**，逻辑上看，一个**指向后继**，一个**指向前驱**，用这样的结点构建的链表称为**双向链表**。

1、双向链表的结点类型声明

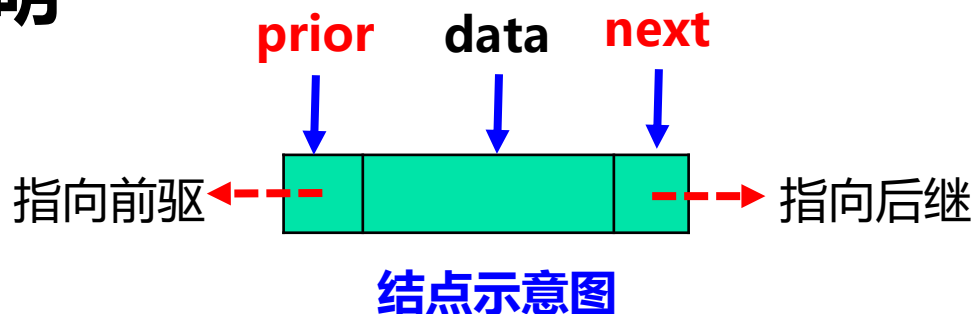
```
struct lnode{
```

```
    int data;
```

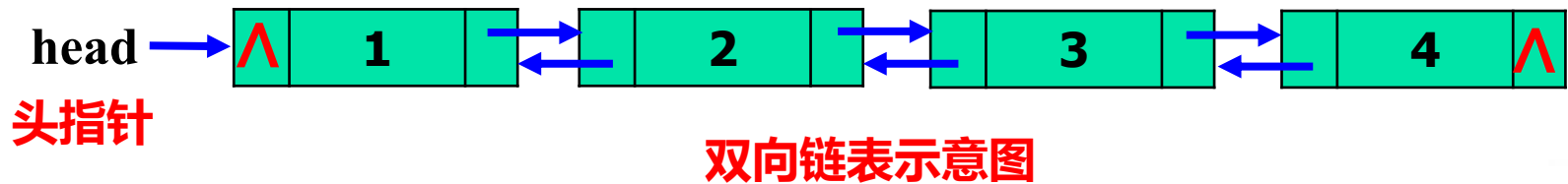
```
    struct lnode * prior; /*前驱指针*/
```

```
    struct lnode * next;  /*后继指针*/
```

```
};
```



2、非循环双向链表

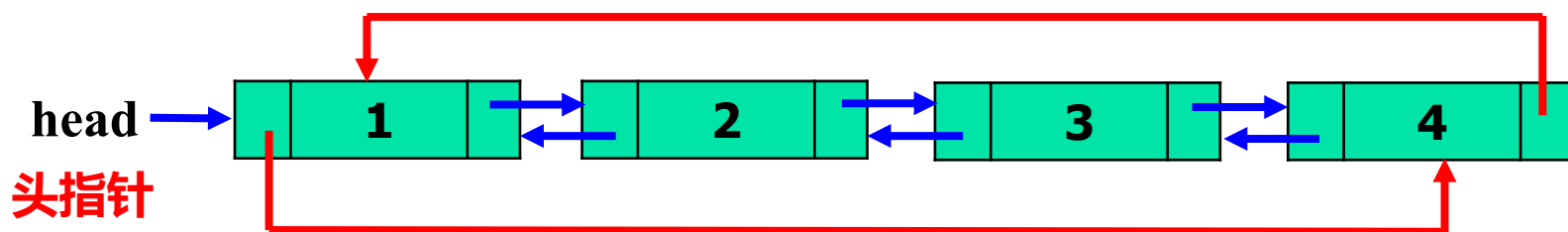


双向链表的特点：

- ◆ 双向链表也由**头指针head** “指示”。
- ◆ 每个结点中有两个指针，**next 指向后继**，**prior 指向前驱**。
- ◆ **首结点的 prior 指针为空**，**尾结点的 next 指针为空**。
- ◆ **双向搜索**：利用后继指针（next）可以向后搜索，利用前驱指针（prior）可以向前搜索。

3、双向循环链表

令双向链表中的尾结点的 **next** 指针指向头结点，头结点的 **prior** 指针指向尾结点，由此构成**双向循环链表**。



双向循环链表示意图

双向循环链表的特点：

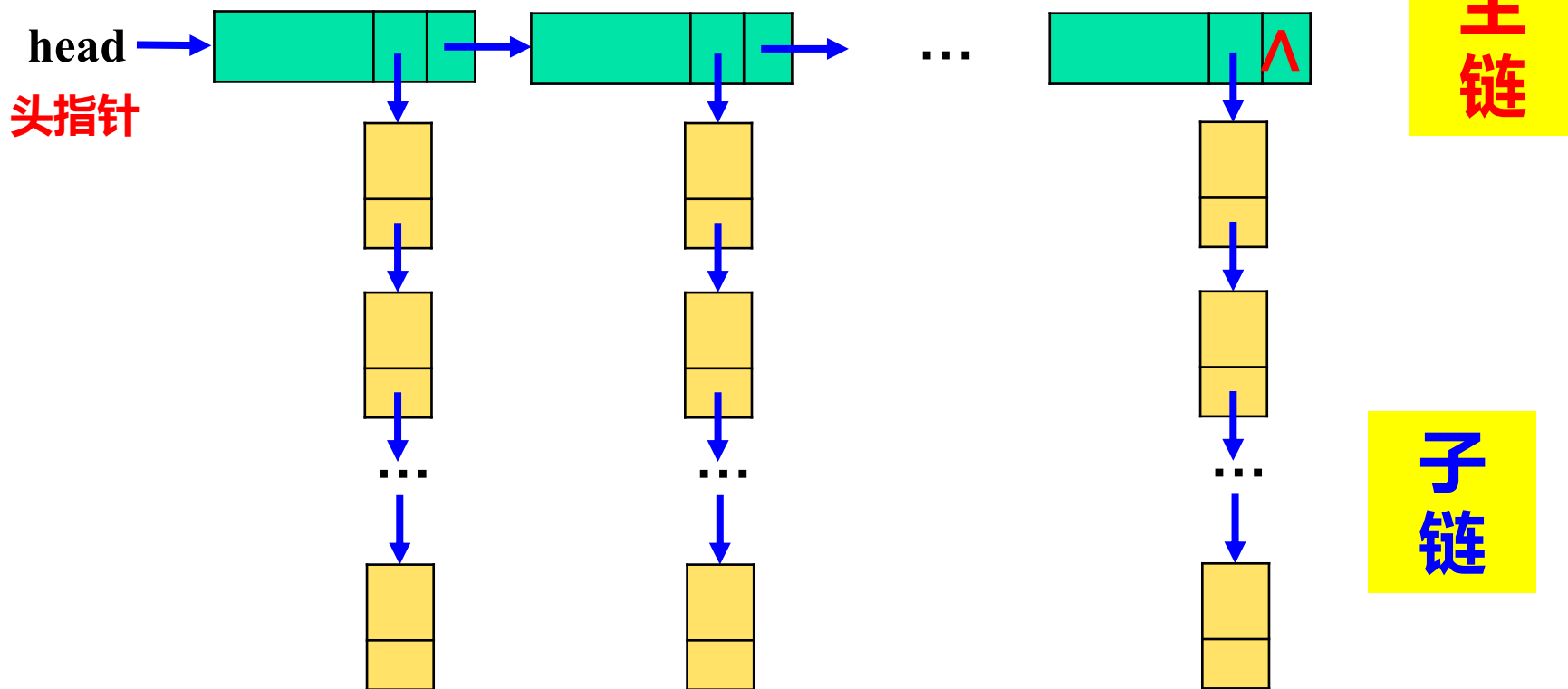
双向循环链表中没有空指针域；遍历链表时，可以从任何一个结点出发，**沿 next 指针或 prior 指针在链表中“打转”**，都可以访问到链表中的所有结点。

4、双向链表的操作

基于**双向（循环）链表的定义和形态特征**，参考单向链表的操作方法，可以**创建新链表、插入结点、删除结点、查找结点、遍历链表、释放链表**等，也可以创建**有表头结点的双向（循环）链表**。对双向（循环）链表的具体操作，查阅资料自行学习。

9.8.8 十字交叉链表

形如如下形式的链表称为**十字交叉链表**。



- ◆ **十字链表**中，每个结点（至少）有**两个指针**，一个指向后继，用于建立**主链**；另一个作为**子链头指针**，指向一个**子链**。
- ◆ 每个**主结点**都有自己的子链。

例 利用十字交叉链表实现学生基本信息和学生学习成绩的录入、遍历、存盘、加载等操作功能。

学生基本信息表

学号	姓名	性别	年龄	家庭住址	联系电话	备注
0001	aaa	m	18	hubei,wuhan	12345678	Abc def
0002	bbb	f	18	hunan,changsha	76545678	Aaaaaa bbb
...						
00xx	zzz	m	19	hubei,honghu	32145678	Nnn kkk

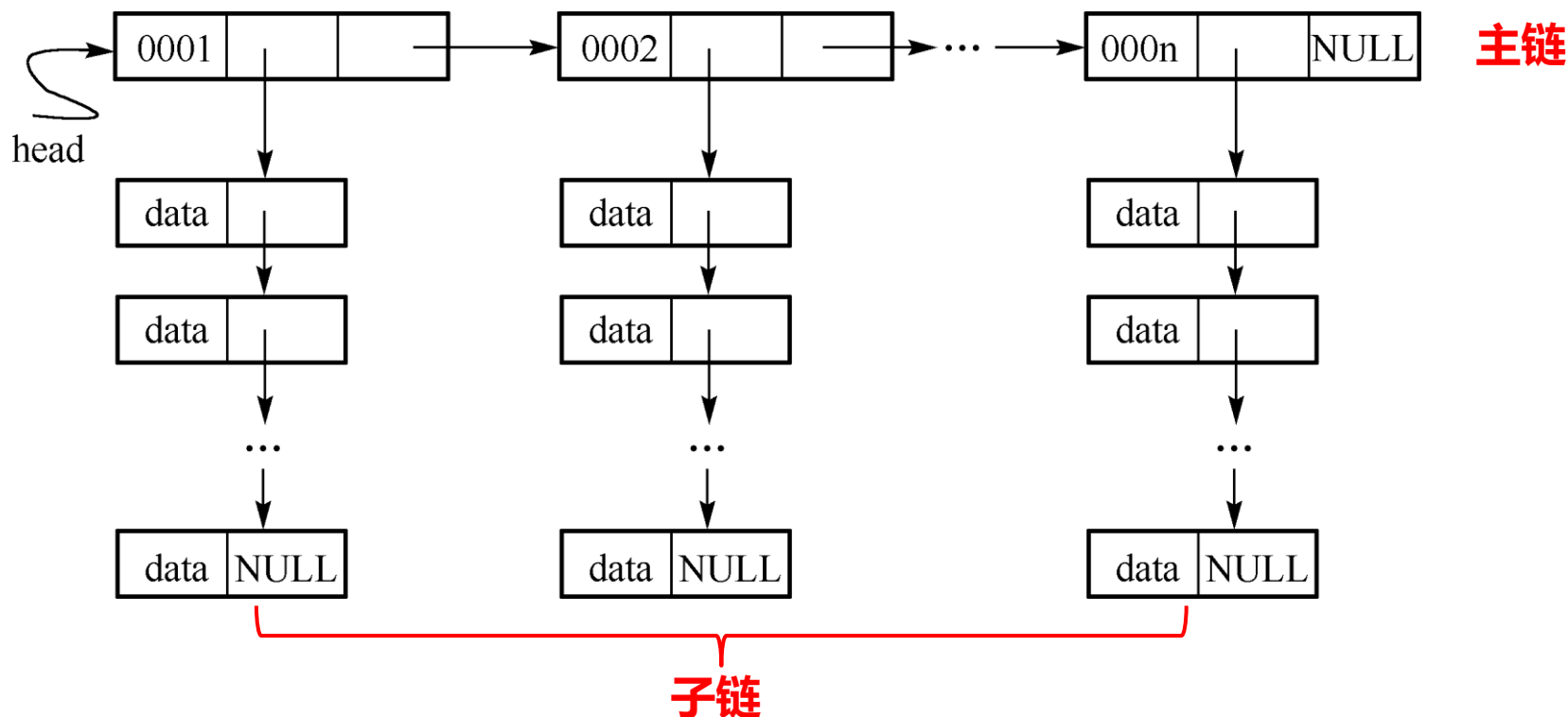
学生学习成绩表

学号	高等数学	普通物理	电工基础	中国语文	法语	茶艺
0001	86	85	73	×	80	×
0002	77	83	76	82	87	75
0003	87	82	81	×	×	86
...						
00xx	89	87	85	×	×	×

除必修课外，学生有不同的选修，课程信息不一

用**十字交叉链表**描述学生数据：

首先创建一个**主单链表**，保存**学生的基本信息**，该链表中的每个结点记录一个同学的基本信息，并有一个用于建立保存学生各科成绩的**成绩子链表**的**头指针**，将指向这个学生的**成绩子链**。



学生基本信息主表和成绩子表的十字交叉链表表示

1、十字交叉链表的数据结构

(1) 课程成绩子链结点的结构类型 **courses**

```
typedef struct scorelnode{  
    char    num[5];           /* 学号 */  
    char    course[20];       /* 课程名称 */  
    int     score;            /* 成绩 */  
    struct scorelnode * next; /* 有一个自引用指针，用于构建  
                                成绩子单链表*/  
} courses;
```

(2) 学生基本信息**主链结点**的结构类型 **studs**

```
typedef struct studlnode{  
    char num[5];          /* 学号 */  
    char name[10];        /* 姓名 */  
    char sex;              /* 性别 */  
    int  age;              /* 年龄 */  
    char addr[30];         /* 家庭住址 */  
    char phone[12];        /* 联系电话 */  
    char * memo;           /* 备注字段 */  
    courses * head_score; /* 指向成绩子链的头指针 */  
    struct studlnode * next; /* 自引用指针, 用于构建主单链表 */  
} studs;
```

□ 以下是本应用中设计的一些函数的原型声明

- **void create_cross_list(studs **head);**
 - 输入数据并创建一个十字交叉链表，head带出表头指针；
- **void save_cross_list(studs *head);**
 - 用fwrite函数将十字交叉链表中的结点数据存盘；
- **void load_cross_list(studs **head);**
 - 用fread函数读数据文件，并用读入的数据创建一个学生基本信息和课程成绩的十字交叉链表；
- **void traverse_cross_list(studs *head);**
 - 遍历十字交叉链表。

□ 程序需包含的头文件：

```
#include "stdio.h"
```

```
#include "stdlib.h"
```

```
#include "string.h"
```

```

void main(void)
{
    studs *head = NULL;  char ch;
lop:
    do {  /* 构造提示菜单，供用户选择 */
        printf("\t\t1: input and create the cross list\n");
        printf("\t\t2: save the cross list\n");
        printf("\t\t3: load the cross list\n");
        printf("\t\t4: traverse the cross list\n");
        scanf("%c",&ch);
        getchar();
    }while(ch < '1' || ch > '4');
    switch(ch){  /* 根据输入的选择转移 */
        case '1': create_cross_list(&head);  break;
        case '2': save_cross_list(head);  break;
        case '3': load_cross_list(&head);  break;
        case '4': traverse_cross_list(head);
    }
    printf("coutinue choice? yes or no?\n"); /* 是否继续 */
    ch = getchar();  getchar();
    if(ch == 'y' || ch == 'Y')
        goto lop; /* 继续,转lop */
}

```

显示一个**菜单**，用户可以通过输入功能编号1、2、3、4，然后根据功能号，调用相应的函数进行操作。

/* 创建十字交叉链表处理 */
 /* 保存十字交叉链表 */
 /* 读入十字交叉链表 */
 /* 遍历十字交叉链表 */


```
void create_cross_list(studs **head)    /* 创建十字交叉链表的函数 */
```

```
{
```

```
    studs *hp=NULL,*p;    /* hp是用于下面创建主链的 “临时” 头指针 */
```

```
    courses * pcrs;
```

```
    char s[80]="", ch;
```

```
loop:
```

```
    p = (studs *)malloc(sizeof(studs));    /* 动态申请主链中学生基本信息的结点 */
```

```
    printf("input num,name,sex,age,addr,phone\n");
```

```
    scanf("%s %s %c %d %s %s", p->num, p->name, &p->sex,&p->age,  
                                                p->addr,p->phone);    /*输入结点数据*/
```

```
    getchar();    /* 读scanf 中的换行符 */
```

```
    p->head_score = NULL;    /* 置成绩子链头指针为NULL */
```

```
    p->next = hp;  
    hp = p;    }    /* “头插法” 入链*/
```

```
    printf("continue input data of students? yes or no\n");    /*是否继续*/
```

```
    ch = getchar(); getchar();
```

```
    if(ch == 'y' || ch == 'Y')    goto loop;    /* 要继续, 则继续创建下一个结点 */
```

```
    (*head) = hp;    /* 将 “临时” 头指针赋给参数head, 以便带出链表到主调函数中*/
```

```

p = (*head);
while(p != NULL){      /* 创建每个学生的学习成绩子链 */
    printf("input %s student's score? yes or no?\n",p->num);
    ch = getchar(); getchar();
    while(ch == 'y' || ch == 'Y'){
        pcrs = (courses *)malloc(sizeof(courses));      /* 创建成绩结点 */
        printf("input course, score\n");
        scanf("%s %d", pcrs->course, &pcrs->score);
        getchar();
        strcpy(pcrs->num, p->num);
        pcrs->next = p->head_score;
        p->head_score = pcrs;
        /* 输入成绩和学号 */
        /* 头插法将成绩结点入子链 */
        printf("input %s student's score? yes or no?\n",p->num); /* 是否继续*/
        ch = getchar();
        getchar();
    }
    p = p->next;
}
}

```

```

void traverse_cross_list(studs *head) /* 遍历十字交叉链表的函数 */
{
    studs * p = head;
    courses * pcrs;
    while(p != NULL){ /* 遍历学生基本信息主链，输出每个学生的信息*/
        printf("%s\t%s\t%c\t%d\t%s\t%s\n",p->num,p->name,
                p->sex, p->age,p->addr,p->phone);
        pcrs = p->head_score; /*pcrs指向成绩子链*/
        while(pcrs != NULL){ /* 遍历当前结点的成绩子链并输出各科成绩*/
            printf("%s\t%s\t%d\n", pcrs->num, pcrs->course, pcrs->score);
            pcrs = pcrs->next;
        }
        p = p->next;
    }
}

```

```

void save_cross_list(studs * head)    /* 保存十字交叉链表的函数，保存至文件 */
{
    FILE *out1, *out2;
    studs * p = head;
    courses * pcrs;
    if((out1 = fopen("c:\\student.dat","wb")) == NULL)    /* 打开基本信息文件 */
        exit(-1);
    if((out2 = fopen("c:\\score.dat","wb")) == NULL)    /* 打开成绩文件 */
        exit(-1);
    while(p != NULL){
        fwrite(p,sizeof(studs),1,out1);    /* 写学生基本信息记录 */
        pcrs = p->head_score;
        while(pcrs != NULL){
            fwrite(pcrs,sizeof(courses),1,out2);
            pcrs = pcrs->next;
        }
        p = p->next;    /* 指向下一个学生基本信息结点 */
    }
    fclose(out1);    /* 关闭基本信息文件 */
    fclose(out2);    /* 关闭成绩文件 */
}

```

/* 遍历学生成绩链，将成绩数据写入文件 */

```

void load_cross_list(studs **head)          /* 从文件读入十字交叉链表的函数 */
{
    FILE *in1,*in2;
    studs * hp=NULL,*p;
    courses * pcrs;
    if((in1 = fopen("c:\\student.dat","rb")) == NULL) exit(-1); /* 打开基本信息文件 */
    if((in2 = fopen("c:\\score.dat","rb")) == NULL) exit(-1); /* 打开成绩文件 */
    while( ! feof(in1) ){
        p = (studs *)malloc(sizeof(studs)); /* 创建基本信息结点 */
        fread(p,sizeof(studs),1,in1);      /* 读一条基本信息记录到结点中 */
        if(!feof(in1)){ p->head_score = NULL; /* 置成绩链头指针为NULL */
                        p->next = hp;
                        hp = p; } /* “头插法” 入链*/
    }
    (*head) = hp; /* 将 “临时” 头指针赋给参数head, 以便带出链表到主调函数中*/
    while( ! feof(in2) ){
        pcrs = (courses *)malloc(sizeof(courses));
        fread(pcrs,sizeof(courses),1,in2); /* 读一条成绩记录到新结点 */
        if(!feof(in2)) { p = (*head);
                        while(p != NULL) { /*遍历学生基本信息链 */
                            if(!strcmp(p->num , pcrs->num)){ /* 找该学号的主结点 */
                                pcrs->next = p->head_score;
                                p->head_score = pcrs; } /* “头插法” 将成绩结点插入子链*/
                            break; }
                        else p = p->next; }
    }
}
fclose(in1); /* 关闭基本信息文件 */
fclose(in2); /* 关闭成绩文件 */
}

```

*9.9 联合

联合类型也是一种**构造类型**。

◆ **联合**和**结构** “**形式上**” 的异与同：

■ **只有一点不同**：声明类型的时候所用的关键字不同

➤ 联合用：**union**

➤ 结构用：**struct**

■ 除上述一点不同外，**联合类型的声明、使用在形式上与结构看起来完全一样。**

9.9.1 联合类型的声明和使用

1、联合类型的声明

例，如下声明了一个有3个数据项的**联合类型**。

```
union chl {  
    char c;      /*字符成员*/  
    short h;     /*短整型成员*/  
    long l;      /*长整型成员*/  
};
```

其中，

- ◆ **union**：关键字，必须有；
- ◆ **chl**：联合类型名，合法标识符。
- ◆ 上述声明定义了一种联合类型：**union chl**，其中的 **c**、**h**、**l** 是该联合类型的**数据成员**。数据成员各有自己的类型。

2、联合变量、联合指针变量的声明、联合成员的引用

- ◆ **联合变量**的声明: `union chl u;`
- ◆ **联合类型指针变量**的声明: `union chl v,*pv = &v;`
- ◆ **取联合变量的地址**: `&u`
- ◆ **间访联合类型指针**: `*pv`
- ◆ **联合成员的引用**
 - 基于**联合体**用 `.` 引用成员: `u.c`、`u.h`、`u.l`
 - 基于**联合指针**用 `->` 引用成员: `pv->c`、`pv->h`、`pv->l`
 - 或**间访指针**并引用成员: `(*pv).c`、`(*pv).h`、`(*pv).l`

形式上看与结构完全一致

◆ 定义 **union chl** 联合类型的同时声明联合变量

如: **union chl**{
 char c;
 short h;
 long l;
} **v**, ***pv** = &**v**;

3、联合变量的初始化: **union chl u**={**'a'**}; /*初值表*/

4、为成员赋值: **u.c**='a';
 u.h=123;
 u.l=123456789;

根据成员类型赋值

5、联合类型的数组: **union chl u**[6];

6、函数: 参数和返回值都可以是联合类型,

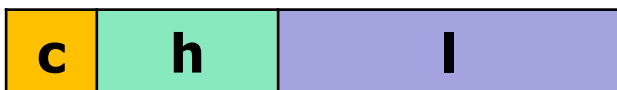
联合类型的指针可以做形参, 采用址传递等。

形式上看与结构完全一致

9.9.2 联合与结构的本质不同：所有数据成员共享空间

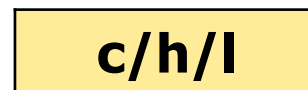
对比以下声明的结构类型 `struct s_chl` 和结构变量 `s`、联合类型 `union chl` 和联合变量 `u`：

```
struct s_chl{  
    char c;  
    short h;  
    long l;  
} s;
```



结构变量 `s` 的存储，7个字节

```
union chl{  
    char c;  
    short h;  
    long l;  
} u;
```



联合变量 `u` 的存储，4个字节

- 结构变量的每个成员在内存中都有自己独立的存储单元；
- 联合变量的所有成员在内存中都没有自己独立的存储单元，而是共享一个共同的存储区域。

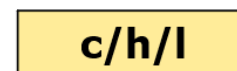
- ◆ 结构类型的大小是其所有成员大小之和。
- ◆ 而**联合类型的大小由联合中占字节数最多的成员决定**；如：union chl 的存储大小为 **4字节**，因为成员 long l 最长。

```
struct s_chl{  
    char c;  
    short h;  
    long l;  
} s;
```



结构变量 s 的存储，7个字节

```
union chl{  
    char c;  
    short h;  
    long l;  
} u;
```



联合变量 u 的存储，4个字节

- ◆ **为什么会有联合类型：**
 - 基本目的：**节省空间**
 - 特殊用途：**同一存储区的内容可以根据成员做不同的解释。**

◆ 联合类型适用的基本场景

联合类型适用于这样的一种场景：其成员各有用途，但**同一时间成员又不会同时存在**：如一个时刻用字符 c 成员，下一时刻用短整型 h 成员，再下一时刻用长整型 l 成员。

如果像结构那样定义它们，则每个成员都要分配独立的存储空间，但又不同时使用，**任何时刻没被用到的成员仍会占据空间而造成空间浪费**。所以把它们组合在“联合”里，**大家公用一个空间保存数据**：

- (1) **节约总空间**；
- (2) **错时使用**，不会冲突

(自行设想具体应用场景)。

注：对特殊应用场景
要另作解释。

9.9.2 联合类型的存储

共享存储：联合类型的所有成员共享一个存储空间，对任何一个成员赋值，**该成员的值都存放到这个空间从第一个字节开始的、长度等于该成员类型长度的若干字节中**，这些字节原有的内容被覆盖，但**超出长度范围、没用到的字节内容不变**。如：

- ◆ 先赋值：**u.i=0x12345678**



- ◆ 再赋值：**u.c= 'a'**



- ◆ 再赋值：**u.h=0xFF**



```
union chl{  
    char c;  
    short h;  
    long l;  
} u;
```

注：不论给哪个成员赋值，总空间大小是不变的。

联合成员的地址：由于所有成员的存储都是从空间的第一个字节开始，所以用 **&** 取任何成员的地址，**得到的地址值都相同**，都是该空间第一个字节的地址；但**由于成员的类型不同，成员的地址类型也就不同**，这些地址的类型是**各自成员类型的指针类型**。

如以下联合体 **u**，假设从地址 **0x00ffa080** 开始，则有：

```
union chl{
    char c;
    short h;
    long l;
} u;
```



表达式	值	类型
&u.l	0x00ffa080	Int *
&u.c	0x00ffa080	char *
&u.h	0x00ffa080	short *

9.9.3 联合变量成员的引用

引用联合体的成员时，仅依据**成员的类型**获取联合体存储空间中从第一个字节开始的长度等于成员类型长度的若干字节使用，**超出范围的字节访问不到**。

如：**u.c='a'**：仅对 u 的 4 字节里的**第一个字节**赋字符数据 'a'，其余的字节保持不变；

u.h=0xFF：将对**前 2 字节**赋短型整数 **0xFF**，后两个字节的内容保持不变；

u.l=0x12345678：将用到所有 **4 个字节**，并赋长型整数 **0x12345678**。

读也是一样，仅读取从第一个字节开始的长度等于成员类型长度的若干字节内容，并按成员类型解释其值。

如：若有 **u.l=0x12345678**；

char a=u.c；

则 **a==0x78**

short s=u.h；

h==0x5678

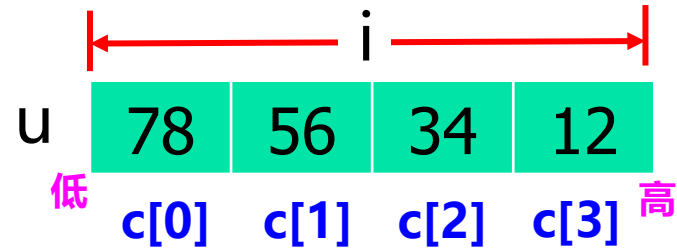


特殊应用实例1：利用联合类型输出一个整数的 4 个字节的值 (32位系统)。

```
#include "stdio.h"

union _ui {
    char c[4];
    int i;
};

int main(void)
{
    union _ui u;
    u.i=0x12345678;
    for(i=0;i<4;i++)
        printf("%0x ", u.c[i]);
    return 0;
}
```



输出：
78 56 34 12

特殊应用实例2：利用联合类型来判断系统的数据表示是大端还是小端。

```
#include "stdio.h"
```

```
int main(void)
```

```
{
```

```
    union {
```

```
        char c;
```

```
        int a;
```

```
    }u;
```

```
    u.a=1;
```

```
    if(u.c)
```

```
        printf("小端系统");
```

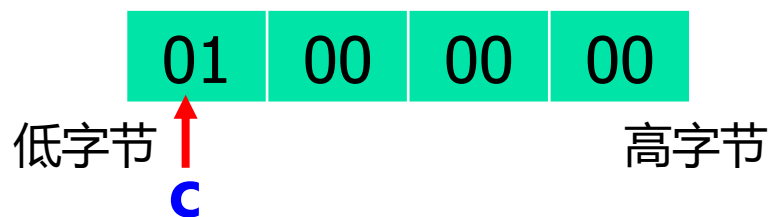
```
    else
```

```
        printf("大端系统");
```

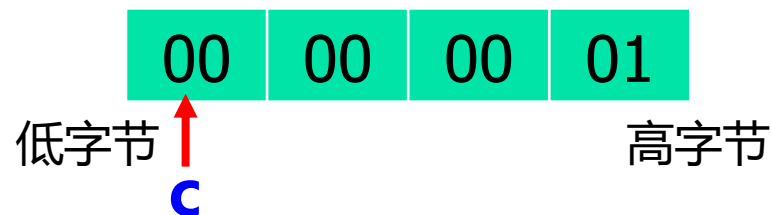
```
    return 0;
```

```
}
```

小端系统中 u 的值



大端系统中 u 的值



两个问题：

1、联合变量的初始化

- **联合变量的初始化**形式上与结构变量的初始化类似：**初值表**；
但早期C语言标准规定，**联合变量初始化时只能对联合体的第 1 个成员进行初始化**，即初值表中只能包含类型与第 1 个成员数据类型相同的一个初值。

如： `union chl v = {'9'};`

但从C99标准开始，可以使用**成员指示符**指定对联合变量的某个成员进行初始化。

如： `union chl v = { .l = 0x12345678};`

指定对 l 成员进行初始化

2、使用联合要注意数据与操作的相容性问题

例，在 `union chl` 中增加一个成员 `float f`;

执行：`u.l=0x31323334L`;

`printf("float format: %f\n",u.f );`

输出结果：`0.00` 为什么？

```
union chl{
    char c;
    short h;
    long l;
    float f;
} u;
```

字节0 字节1 字节2 字节3

0x34	0x33	0x32	0x31
------	------	------	------

按IEEE 754单精度浮点数来解释

联合中允许存储不同类型的数据，但对不同时刻存储的数据，允许的操作可能不相容（字节内容和操作不兼容）而出现错误。

注：由于语法上合法，编译器对这种情况不会报错，但运算的结果却不正确，所以使用时要格外小心。

上次课主要内容 (2024.12.10)

1、交换链表的结点：只交换data、交换结点实体

(1) **交换相邻的两个结点**

基于链表的finadmax过程和**冒泡排序**

(2) **交换不相邻的两个结点**

选择排序算法：基于数组和基于链表的实现

2、归并链表操作

3、**循环单链表、双向链表、双向循环链表**

4、**十字链表**：带有多指针域，**一个指针域一个指针域处理**

5、**联合**：**成员共享数据空间**，使用上对照结构的相关内容

*9.10 字段结构

字：一般指**两字节数据**，C语言对应 **short int** 类型。

问题的提出：如 **short int** 型数据（一个**字**）有 16 位，可以表示一个大整数。但若仅用于存储小整数，就有点浪费了。如只表示 **0~15**，只要 4 位就够。如果应用程序中有很多**小整数数据**，每个位数都不多，如若都用**长型**整数存储，就会**浪费很多字节**。

一种节约存储空间思路：大整型的量（如 short int 型变量）中有很多位（如 16 位），**把这些位分成“若干段”，每段有若干位，可以保存一个小范围内的整数**，这样这些小整数就被“**压缩存储**”在一个大整型的字节空间内，从而达到节约空间的效果。

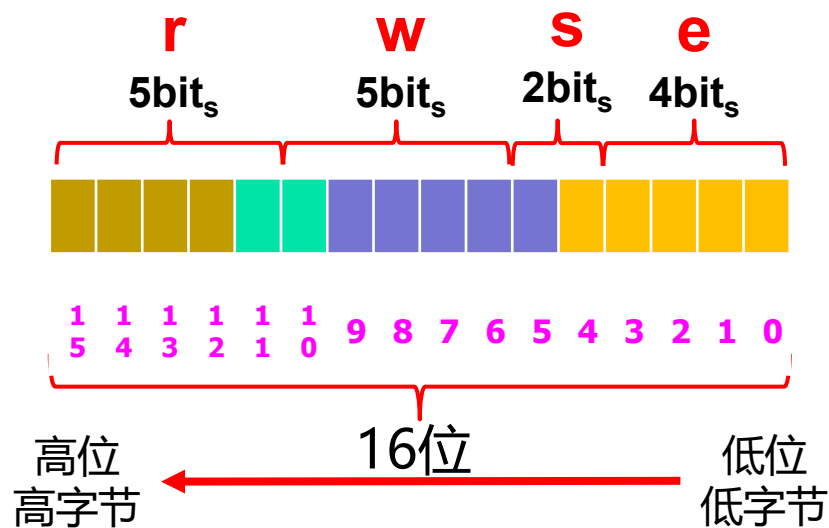
——采用**字段结构**来实现。

1、**字段**：连续相邻的若干二进制位称为一个**字段**。

字段的位数称为**字段宽度**。设一个字段宽度为 n ，则这个字段可以表示 $0 \sim 2^n - 1$ 范围内的“小整数”。

2、**字段结构**：以**字段为成员**定义的结构，称为**字段结构**。

如：struct t {
 unsigned short int e:4;
 unsigned short int s:2;
 unsigned short int w:5;
 unsigned short int r:5;
};



e、**s**、**w**、**r**是字段成员名，**4**、**2**、**5**、**5**分别是4个字段的宽度。

3、对字段结构类型声明的说明

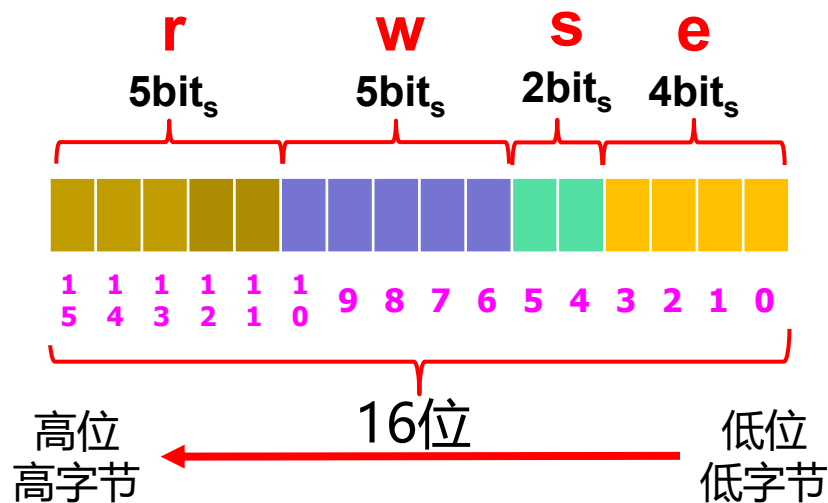
首先确定存储字段的“**主类型**”。主类型**必须是整型**，通常说明为 **unsigned**；然后以

主类型 字段成员名 : 宽度

的方式声明**字段结构成员**，其中宽度是一个非负整型常量表达式。

然后在**主类型的“位空间”**内，按照字段成员声明的顺序和宽度将主类型的**位空间“划分成段”**。如：

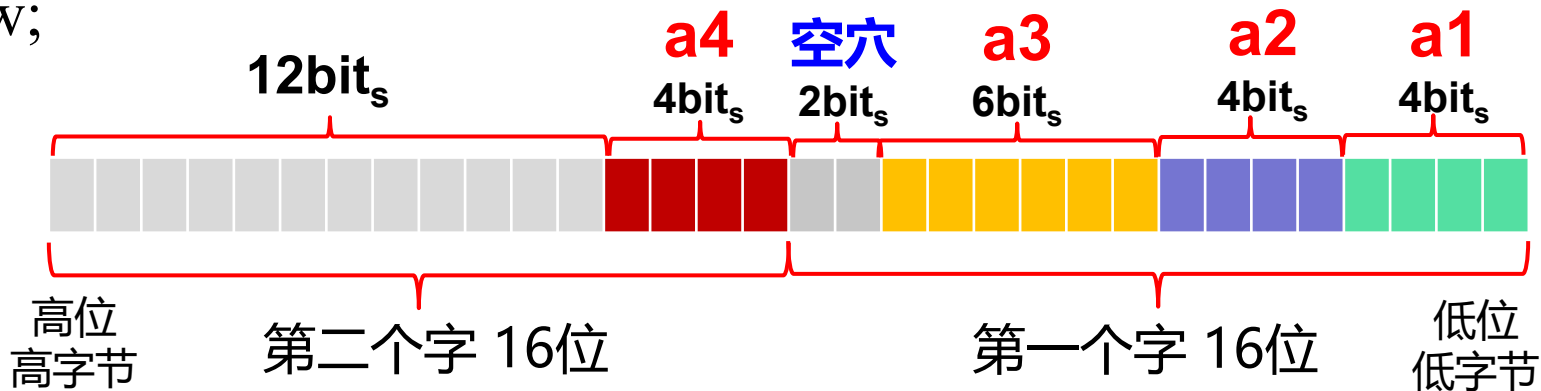
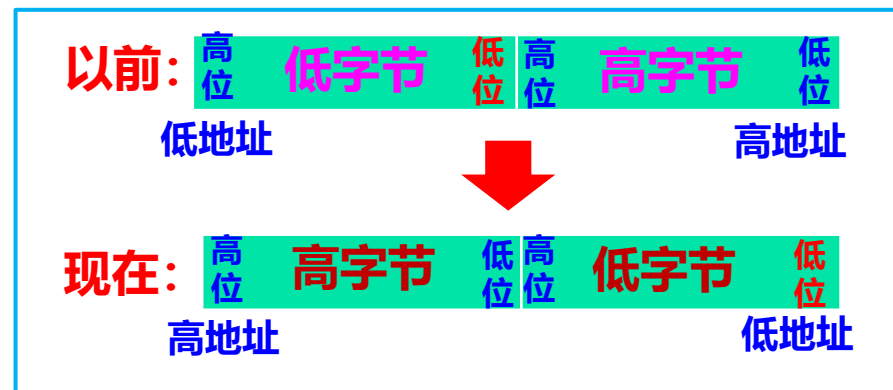
如： **struct t** {
 unsigned short int **e:4**;
 unsigned short int **s:2**;
 unsigned short int **w:5**;
 unsigned short int **r:5**;
} **a**;



说明：（1）**字段不能跨越一个字的边界**

即**主类型的位空间按“字”（16位）组织**，**一个字段的位只能在一个“双字节”范围内划分**。如果一个字剩余的位数不足，则下一字段将从相邻的下一个字开始存放，这就使得上一个字中留下一些未用的空位，称为**空穴**。

如：`struct {`
 unsigned short **a1:4**;
 unsigned short **a2:4**;
 unsigned short **a3:6**;
 unsigned short **a4:4**;
};



(2) 字段可以没有名字。

无名字段的作用主要是用**空位**占据那些不用的二进制位；

(3) **宽度为0的无名字段**强制下一个字段从**新的字边界**开始存储。

如: **struct** {

unsigned short **a1:4**;

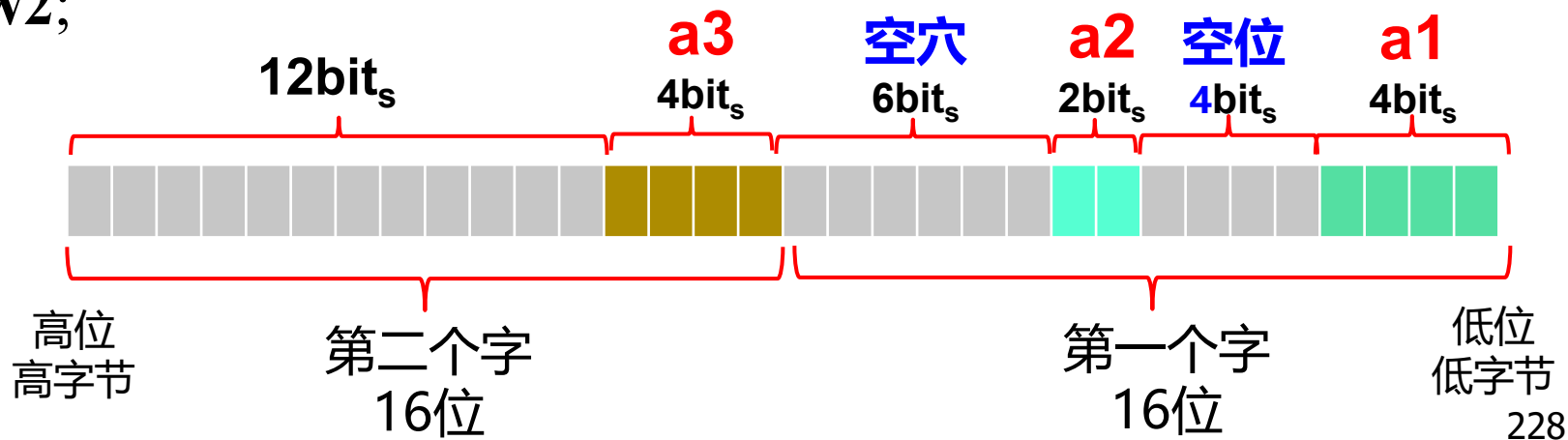
unsigned short **:4**; /***无名字段, 形成空位***/

unsigned short **a2:2**;

unsigned short **:0**; /***宽度为0**的字段*/

unsigned short **a3:4**;

} **w2**;



3、字段结构类型的使用

字段结构也是一种**构造类型**，本身也是一种**结构类型**。除了定义和常规的结构类型有以上的区别外，其余的**使用方式和常规结构类型大致相同**，如：

- ◆ 声明字段结构类型的变量：**struct t a;**
- ◆ 声明字段结构类型的指针：**struct t *pa = &a;**
- ◆ 访问字段结构变量的成员：**a.e、a.s、a.w、a.r;**
pa->e、(*pa).s
- ◆ 间访字段结构指针：***pa;**

◆ 为字段成员赋值： **a.e=7**、 **pa->e=7**

注：字段成员只限在其 **“位内”** 存储数据（小整型数）。如果赋的数据二进制位数超出它的字段宽度，则只存储 **“位宽”** 以内的低位，超出的高位会被舍弃。所以对字段成员赋值时要注意不要超过其位宽。

如：a.e=17;

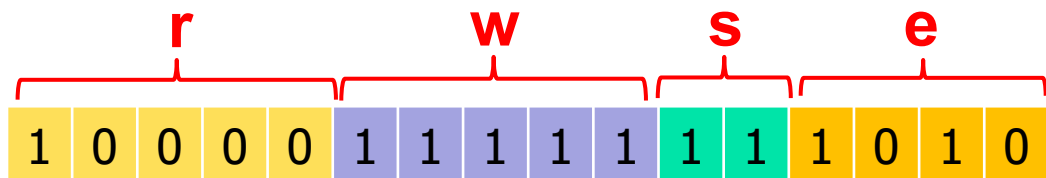


printf("%d", a.e); **输出： 1**

```
struct t {  
    unsigned short int e:4;  
    unsigned short int s:2;  
    unsigned short int w:5;  
    unsigned short int r:5;  
};
```

◆ 字段成员的值：只限定其 **“位内”** 表示的值。

如：a.e=5; a.s=3; a.w=31; a.r=1;



高位
高字节

低位
低字节

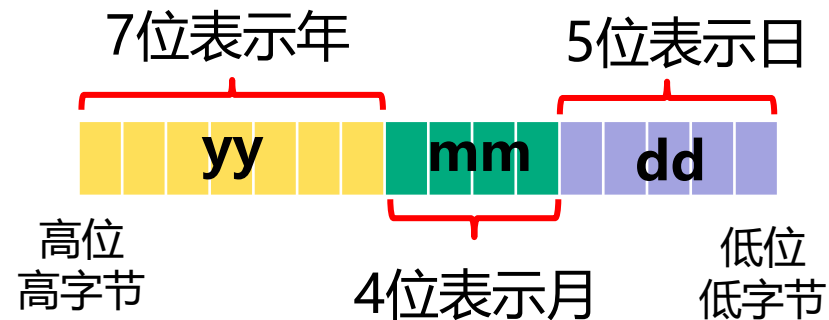
```
printf("%d %d %d %d",  
       a.e, a.s, a.w, a.r);
```

输出： **5 3 31 1**

- 字段结构提供了一种**紧凑存储数据**的方法，起到了节约数据存储空间的效果。 注：由于空穴的存在，使用字段有时未必节约空间。

回顾：把表示21世纪日期的日、月、年3个整数压缩成1个16位的整数。例2.24用移位操作实现。现在可以用字段结构实现：

```
struct dmy
{
    unsigned short int day:5;
    unsigned short int month:4;
    unsigned short int year:7;
} b;
```



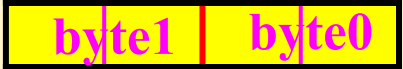
```
b.day=10;
b.month=12;
b.year=24
```

- **字段结构并不节约时间**：字段成员参与运算的时候，其值要被“拆解”出来，还原成 int 型数据后再参与后续运算。

9.8.3 字段结构与联合的应用

例 用字段和联合访问一个16位字中的各个**字节**和**半字节**。

```
struct w16_bytes {  
    unsigned short byte0:8;  
    unsigned short byte1:8;  
};
```



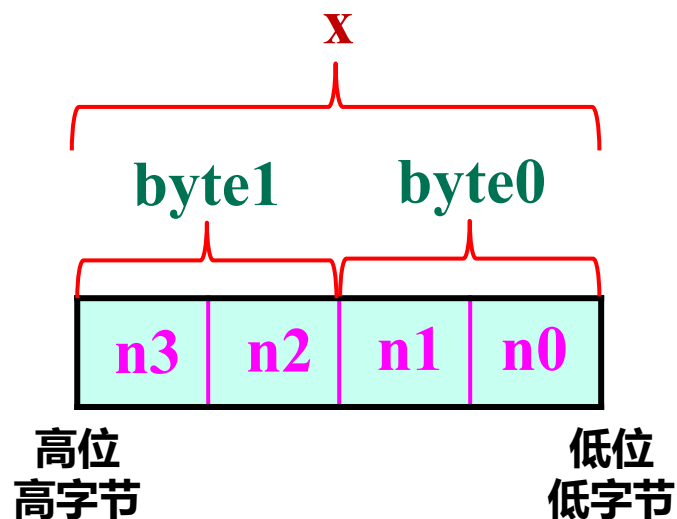
A diagram showing a horizontal rectangle divided into two equal yellow sections. The left section is labeled 'byte1' and the right section is labeled 'byte0'.

```
struct w16_nibbles {  
    unsigned short n0:4;  
    unsigned short n1:4;  
    unsigned short n2:4;  
    unsigned short n3:4;  
};
```



A diagram showing a horizontal rectangle divided into four equal yellow sections. The sections are labeled from left to right as 'n3', 'n2', 'n1', and 'n0'.

```
union w16 {  
    short x;  
    struct w16_bytes byte;  
    struct w16_nibbles nibble;  
};
```



```
#include <stdio.h>
```

```
struct w16_bytes {  
    unsigned short byte0:8;  
    unsigned short byte1:8;  
};
```

```
struct w16_nibbles {  
    unsigned short n0:4;  
    unsigned short n1:4;  
    unsigned short n2:4;  
    unsigned short n3:4;  
};
```

```
union w16 {  
    short x;  
    struct w16_bytes byte;  
    struct w16_nibbles nibble;  
};
```

```
int main()
```

```
{  
    union w16 w={0x1234};  
    printf("1st byte of %#x is %x\n", w.x, w.byte.byte0);  
    printf("2nd byte of %#x is %x\n", w.x, w.byte.byte1);  
    printf("1st nibble of %#x is %x\n", w.x, w.nibble.n0);  
    printf("2nd nibble of %#x is %x\n", w.x, w.nibble.n1);  
    printf("3rd nibble of %#x is %x\n", w.x, w.nibble.n2);  
    printf("4th nibble of %#x is %x\n", w.x, w.nibble.n3);  
}
```

```
1st byte of 0x1234 is 34  
2nd byte of 0x1234 is 12  
1st nibble of 0x1234 is 4  
2nd nibble of 0x1234 is 3  
3rd nibble of 0x1234 is 2  
4th nibble of 0x1234 is 1
```

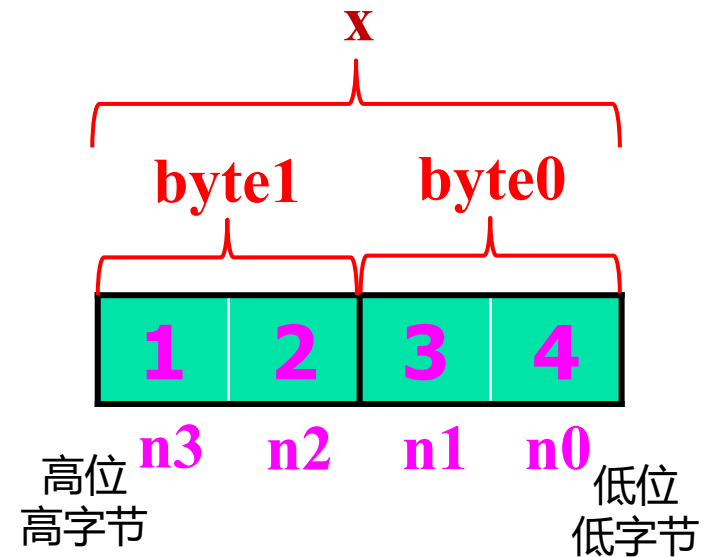
```
#include <stdio.h>
```

```
struct w16_bytes {  
    unsigned short byte0:8;  
    unsigned short byte1:8;  
};
```

```
struct w16_nibbles {  
    unsigned short n0:4;  
    unsigned short n1:4;  
    unsigned short n2:4;  
    unsigned short n3:4;  
};
```

```
union w16 {  
    short x;  
    struct w16_bytes byte;  
    struct w16_nibbles nibble;  
};
```

```
1st byte of 0x1234 is 34  
2nd byte of 0x1234 is 12  
1st nibble of 0x1234 is 4  
2nd nibble of 0x1234 is 3  
3rd nibble of 0x1234 is 2  
4th nibble of 0x1234 is 1
```



对 w 的高字节取反，然后交换最高半字节和最低半字节的值（自学）

```
int main()
```

```
{
```

```
    union w16 w = {0x1234};
```

```
    w.byte.byte1 = ~ w.byte.byte1; /* 高字节取反， w.x为0xed34 */
```

```
    w.nibble.n0 ^= w.nibble.n3;
```

```
    w.nibble.n3 ^= w.nibble.n0;
```

```
    w.nibble.n0 ^= w.nibble.n3;
```



```
    /* 交换最高和最低半字节， w.x为0x4d3e */
```

```
    printf("w.x= %#hx\n", w.x); /* 以十六进制输出整个16位字的值 */
```

```
    printf("Low byte: %#hx\n", w.byte.byte0); /* 输出16位字的低字节 */
```

```
    printf("Low nibble : %#hx\n", w.nibble.n0); /* 输出16位字的最低半字节 */
```

```
    return 0;
```

```
}
```

```
w.x= 0x4d3e
Low byte: 0x3e
Low nibble : 0xe
```


交换两个整形变量x和y的值：

方法一：

```
t=y;
```

```
y=x;
```

```
x=t;
```

方法二：

```
x=x^y;
```

```
y=y^x
```

```
x=x^y;
```

本章小结

1. 结构类型的声明，结构变量的声明及初始化；
2. 结构体的引用，结构指针和基于结构指针的成员引用。
3. 结构数组的定义和使用。
4. 结构类型作为函数的参数和返回值。。
5. 链表的定义和基本操作，动态链表。
6. 联合类型的定义，联合变量的声明，以及联合成员的引用，了解联合类型的存储结构特征。
7. 字段的定义，字段结构的定义、字段结构类型变量的声明及成员的引用

第九章习题：

■ 9.1, 9.4, 9.5, 9.7, 9.9 (单数小题) 9.12 (1、3、5小题) ,
9.15

■ 11.5: 1、3、5小题

■ 11.6: 1、2、4小题

■ 11.9

■ 14.4 (以Ctrl+Z结束输入) 14.6

只做可以用文字描述的部分，
凡是涉及需要“上机操作”的
部分可以不用做。